An Algebra of Hierarchical Lazy DataFrame Reduction Trees

Hyeondong Lee

November 2026

Abstract

A table with d grouping columns can be summarized by $d!$ reduction trees, one for each order of the columns: the root holds all rows, each level splits by one more column, and summaries are merged upward, as in `GROUP BY ROLLUP`. All of these trees have the same root and the same leaves and differ only in between, at the levels that are displayed as bar charts and heat maps. We ask which order to choose and which views are worth displaying. Our measure is the variance of the group means that a set of columns explains. We show that it is monotone on the lattice of grouping sets and obeys a chain rule, so every order splits the same total among its levels, and orders differ only in how early the variance appears. We give an exact dynamic program over the 2^d grouping sets for the order that reveals variance earliest, show that the greedy order is optimal when the columns act additively and can be far from optimal when they interact, and separate three effects of a single column: its main, total, and Shapley effect. Views of one or two axes are ranked by the same quantity, and flat views identify columns that can be dropped. Everything is computed from the summaries of a single `GROUP BY`, by merging.

These results rest on an algebra of reduction trees, which we develop in full. An aggregate is a homomorphism from the free commutative monoid of bags into a summary space, followed by a finalizer that is applied once. A summary-valued bag keeps a set of columns and squashes all other columns into the summary space, with one summary for each value of the kept columns; the row count of a bag projection is the smallest such space in use, and the bag of the squashed values the largest. Squashing can be continued in any number of steps and cannot be undone, which fixes the direction of every merge. A count on the summary space lets a summary still tell how many rows it stands for, and fibers, the squashes into the free summary space, are restored from finer fibers by stacking. For trees with arbitrary filter predicates, an exchange theorem states exactly when merging the children of a node reproduces the summary of the rows that reach the node, with its consequences for idempotent summaries, additive summaries, and summaries with a count, and for deferring the finalizer. Dense trees compute `ROLLUP`, the levels of a level-uniform tree commute, and the $d!$ orders of a dense tree together compute `CUBE`.

Contents

1 Introduction **3**

2 Preliminaries **5**

2.1 Monoids 5

2.2 Bags 6

3 Mergeable aggregation **7**
3.1 Partial aggregation . . . 7
3.2 Aggregates as a homomorphism followed by a finalizer . . . 8
3.3 Grouped aggregation . . . 9
3.4 Related work . . . 9

4 Relations and squashes **10**
4.1 Relations . . . 10
4.2 Summary-valued bags: kept and squashed attributes . . . 12
4.3 Fibers . . . 20

5 Reduction trees **23**
5.1 Reduction tree and downward operator . . . 24
5.2 Upward operator . . . 25
5.3 From downward selection to upward aggregation . . . 27
5.4 Dense layers and dense trees: `ROLLUP` . . . 37

6 Reordering the levels **40**
6.1 Level-uniform trees and level permutations . . . 40
6.2 Summaries on the level lattice . . . 44
6.3 All orders of a dense tree: `CUBE` . . . 45

7 Choosing the order of levels **47**
7.1 Explained variance on the level lattice . . . 48
7.2 Orders . . . 49
7.3 The most affecting column . . . 51
7.4 Views . . . 52
7.5 The algorithm . . . 53

8 Conclusion **55**

1 Introduction

Consider a table of measurements, for example request latencies annotated with operating system, workload, region, and software version. A typical analysis builds a tree. The root holds all rows. Each child is obtained by a filter (`os = 'linux'`), each grandchild by a further filter (`workload = 'write'`), and so on. At the leaves we compute statistics; we then merge statistics upward, so that each parent holds a small table with one row per child.

With d columns there are $d!$ such trees, one for each order of the levels. They agree at the two ends: the root summarizes all rows, and the leaves are the groups of `GROUP BY` on all columns. They differ at every level in between, and these are the levels that an analyst looks at, because a page shows one axis (a bar chart) or two (a heat map). The order of the levels is usually chosen by habit. This paper chooses it by the data, and answers three questions:

(Q1) **Order.** In which order should the columns be split so that the variation of the aggregated values appears as early as possible?
(Q2) **Column.** Which column affects the aggregated values most?
(Q3) **Views.** Which views of one or two axes expose drastic variation, which suggests a root cause, and which show none, which tells us that a column is irrelevant?

The answers are exact only if two questions about the trees themselves are settled, which are usually taken for granted:

(P1) **Correctness of merging.** When is a parent summary that is merged from the summaries of the children equal to the summary computed directly on the parent's rows?
(P2) **Freedom to reorder.** When may tree levels be swapped, for example from `os` $\to$ `workload` to `workload` $\to$ `os`, and what do the two trees then have in common?

Sections 2 to 6 develop an algebra of reduction trees that answers (P1) and (P2), for trees with arbitrary filter predicates and for every mergeable summary. Section 7 uses it to answer (Q1) to (Q3).

Our contributions are the following.

- *Explained variance on the level lattice* (Section 7.1). For a set I of columns, $\nu(I)$ is the variance of the means of the groups of `GROUP BY` I, weighted by their sizes. It is computed from counts and sums, it is monotone in I, it is bounded by the total variance of the measure, and it obeys a chain rule (Theorem 7.2).
- *Orders* (Section 7.2). Every order splits the same total among its levels (Proposition 7.4), so orders are compared by how early the variance appears. The most revealing order is found exactly by a recursion over the 2^d grouping sets; a greedy order needs $d(d+1)/2$ sets, is optimal when the columns act additively, and can miss an interaction entirely (Theorem 7.6, Example 7.7).
- *Columns* (Section 7.3). The effect of a column depends on the columns before it. We separate its main, total, and Shapley effect and show that they rank columns differently (Proposition 7.9).
- *Views* (Section 7.4). Every view, for every choice of pinned columns and axes, is a level of the cube and is computed from the leaf summaries of one tree; views are ranked by the variance they show (Proposition 7.12).
- *Algorithms* (Section 7.5). Algorithm 1 reads the table once, as one `GROUP BY` on all

columns, and returns the most revealing order with the levels of its tree, the three effects of every column, and the ranked views; Algorithm 2 is the greedy variant.

The algebra underneath consists of the following.

- *Mergeable aggregation* (Section 3). An aggregate function is a homomorphism from the free commutative monoid of bags into a summary space, followed by a finalizer; Table 1 gives the standard SQL aggregates in this form.
- *Summary-valued bags* (Section 4.2). For a set X of kept attributes, an S-bag over X holds, for each value of X, one element of the summary space S, into which the attributes of $\Sigma \setminus X$ have been squashed. The bag projection is the case $S = \mathbb{N}_0$, and the family of fibers the case in which S is free. Squashing can be continued and cannot be undone (Propositions 4.8 and 4.9), which fixes the direction of every merge in the paper. A count on the summary space lets a summary tell whether its group has a row and how many; every summary space can be given one (Proposition 4.17), and the squash then determines the bag projection (Proposition 4.15).
- *Fibers* (Section 4.3). The squash into the free summary space loses nothing. Fibers over fewer kept attributes are restored from fibers over more by stacking, and only in this direction (Theorem 4.19).
- *Reduction trees and the exchange theorem* (Sections 5.1 to 5.3). A tree is a prefix-free set of paths of filter predicates, with a downward selection and an upward merge. Both the direct summary and the upward summary of a node are homomorphic images of the bag of the relation, and they agree for every summary space exactly when every row that reaches the node reaches exactly one leaf below it (Theorem 5.12, Corollary 5.13). Idempotent summaries tolerate overlapping children, additive summaries have an explicit discrepancy, and a count certifies the merge (Corollaries 5.16 and 5.17, Proposition 5.18). The finalizer can be deferred to the node at which a value is reported (Theorem 5.21), and one level is a *two-step merge*: child aggregation followed by the squash of the label attribute (Proposition 5.26).
- *`ROLLUP` and `CUBE`* (Sections 5.4 and 6). A dense layer is one `GROUP BY` on one column, and a dense tree computes `ROLLUP` (Theorem 5.31). The levels of a level-uniform tree commute: a node is determined by the set of levels above it and not by their order (Theorem 6.4), and the $d!$ orders of a dense tree together compute `CUBE` (Theorem 6.8).

Reading guide. Section 7 uses only a few statements of the sections before it: the squash and its calculus (Definition 4.4, Propositions 4.8, 4.9, and 4.15), the exchange theorem (Theorem 5.12 with Corollary 5.13), and the identification of the dense trees with `ROLLUP` and `CUBE` (Theorems 5.31 and 6.8). A reader who is interested in the order of levels can read these and go on to Section 7.

Relation to prior work. The classification of aggregate functions into *distributive*, *algebraic*, and *holistic*, and the operators `ROLLUP` and `CUBE`, go back to Gray et al. [8]; our summary monoids are a restatement of the first two classes, and our trees are the chains of the cube. Merging partial aggregates by an associative and commutative operation is the combiner of MapReduce [3], and aggregation with values in commutative monoids over annotated relations is studied by Amsterdamer, Deutch, and Tannen [1]; Section 3.4 gives a fuller account. The upward summary of a tree is defined by recursion on the tree, a fold in the sense of Meijer, Fokkinga, and Paterson [11]. The decomposition of variance is the law of total variance; the Shapley value of explained variance as an importance

measure is due to Owen [12] and, for dependent inputs as they occur in a table, to Song, Nelson, and Staum [15]. The geometric mean, which Fleming and Wallace [6] argue for when normalized benchmark ratios are summarized, is covered by a transformation of the measure (Remark 7.14). On the systems side, lazy engines such as Polars can evaluate several related queries jointly and apply common-subplan elimination [14], and SQL engines such as DuckDB expose explicit control over CTE materialization [4]. Our contribution is not a new engine but a semantics that tells such engines, and their users, which rewrites are safe. What we add is the connection: the lattice on which the variance is decomposed is the lattice of grouping sets of the cube, an order of tree levels is a maximal chain in it, and every quantity is a function of summaries that are merged and never recomputed.

2 Preliminaries

2.1 Monoids

Definition 2.1 (Semigroup, monoid)**.** A *semigroup* $(S, \oplus)$ is a set S together with a binary operation $\oplus : S \times S \to S$ that is *associative*:

$$(a \oplus b) \oplus c = a \oplus (b \oplus c) \qquad \text{for all } a, b, c \in S.$$

A *monoid* $(S, \oplus, \mathbf{0}_S)$ is a semigroup with a distinguished *identity* element $\mathbf{0}_S \in S$ satisfying

$$\mathbf{0}_S \oplus a = a = a \oplus \mathbf{0}_S \qquad \text{for all } a \in S.$$

A semigroup or monoid is *commutative* if, in addition, $a \oplus b = b \oplus a$ for all $a, b \in S$.

Now, we define structure-preserving maps between monoids as follows.

Definition 2.2 (Monoid homomorphism)**.** Let $(M, \odot, e)$ and $(S, \oplus, \mathbf{0}_S)$ be monoids. A function $h : M \to S$ is a *monoid homomorphism* if

$$h(a \odot b) = h(a) \oplus h(b) \quad \text{for all } a, b \in M, \qquad \text{and} \qquad h(e) = \mathbf{0}_S.$$

A monoid $F \supseteq X$ is *free* on a set X if, for every monoid $(S, \oplus, \mathbf{0}_S)$, every function $\alpha : X \to S$ extends uniquely to a homomorphism $\widehat{\alpha} : F \to S$. The standard example is the set Σ^* of finite strings over an alphabet Σ, with concatenation as the operation and the empty string ε as the identity. Every string is a unique product of symbols, so the extension is forced to be

$$\widehat{\alpha}(a_1 a_2 \cdots a_n) = \alpha(a_1) \oplus \alpha(a_2) \oplus \cdots \oplus \alpha(a_n), \qquad \widehat{\alpha}(\varepsilon) = \mathbf{0}_S.$$

A *free commutative monoid* is defined in the same way, with F and S ranging over commutative monoids only.

Multiples, idempotent and conical monoids. In a commutative monoid $(S, \oplus, \mathbf{0}_S)$, write $n \cdot s$ for the n-fold sum $s \oplus \cdots \oplus s$ of $s \in S$, for $n \in \mathbb{N}_0$, with $0 \cdot s = \mathbf{0}_S$. Then

$$(n + n') \cdot s = n \cdot s \oplus n' \cdot s, \qquad n \cdot (s \oplus s') = n \cdot s \oplus n \cdot s', \qquad (nn') \cdot s = n \cdot (n' \cdot s), \quad (1)$$

and every monoid homomorphism h satisfies $h(n \cdot s) = n \cdot h(s)$. A commutative monoid is *idempotent* if $s \oplus s = s$ for all $s \in S$, so that $n \cdot s = s$ for all $n \geq 1$, and *conical* if $s \oplus s' = \mathbf{0}_S$ implies $s = s' = \mathbf{0}_S$, so that a sum is $\mathbf{0}_S$ only if every summand is. The monoids $(\mathbb{N}_0, +, 0)$ and $(\mathbb{N}_0, \max, 0)$ are conical, and the second is idempotent; a nontrivial group, such as $(\mathbb{Z}, +, 0)$, is neither.

2.2 Bags

Relational tables and dataframes are naturally modeled as *bags* (multisets) of rows, which, unlike sets, record how many times each element occurs. Formally, a bag over a set U is a function $B : U \to \mathbb{N}_0$ with finite support, where $B(u)$ is the multiplicity of u. We write

$$\mathrm{Bag}(U) = \{B : U \to \mathbb{N}_0 \mid \mathrm{supp}(B) \text{ is finite}\}, \qquad \mathrm{supp}(B) = \{u \in U \mid B(u) > 0\},$$

for the collection of all bags over U. The empty bag $\varnothing_{\mathrm{bag}}$ assigns multiplicity zero to every element, and the bag union of $B_1, B_2 \in \mathrm{Bag}(U)$ adds multiplicities pointwise:

$$(B_1 \uplus B_2)(u) = B_1(u) + B_2(u) \qquad \text{for all } u \in U.$$

We write $\{\{u\}\}$ for the bag that contains one occurrence of u and nothing else.

This makes the triple $(\mathrm{Bag}(U), \uplus, \varnothing_{\mathrm{bag}})$ a free commutative monoid on U, which gives us a good property related to extensibility. For an arbitrary summary space S and its commutative monoid $(S, \oplus, \mathbf{0}_S)$, every function $\alpha : U \to S$ uniquely extends to a homomorphism $\widehat{\alpha} : \mathrm{Bag}(U) \to S$ with $\widehat{\alpha}(\{\{u\}\}) = \alpha(u)$. Explicitly,

$$\widehat{\alpha}(B) = \bigoplus_{u \in \mathrm{supp}(B)} \underbrace{\alpha(u) \oplus \cdots \oplus \alpha(u)}_{B(u) \text{ copies}} = \bigoplus_{u \in \mathrm{supp}(B)} B(u) \cdot \alpha(u), \tag{2}$$

with the empty sum defined to be $\mathbf{0}_S$. We read α as the *contribution* of a single row and $\widehat{\alpha}$ as the resulting aggregation over a table, in which every occurrence of a row contributes separately.

For example, let $U = \{\mathrm{KR}, \mathrm{US}\} \times \mathbb{N}_0$ be the set of all rows $[\mathit{region}, \mathit{value}]$, and let

$$B = \{\{\ [\mathrm{KR}, 10],\ [\mathrm{KR}, 10],\ [\mathrm{US}, 7]\ \}\},$$

so that $B([\mathrm{KR}, 10]) = 2$, $B([\mathrm{US}, 7]) = 1$, and $B(u) = 0$ otherwise. Summing the values corresponds to $(S, \oplus, \mathbf{0}_S) = (\mathbb{N}_0, +, 0)$ and $\alpha([\mathit{region}, \mathit{value}]) = \mathit{value}$, which gives

$$\widehat{\alpha}(B) = 2 \cdot \alpha([\mathrm{KR}, 10]) + 1 \cdot \alpha([\mathrm{US}, 7]) = 2 \cdot 10 + 1 \cdot 7 = 27.$$

Selection and image. Two operations of relational algebra are defined for bags over any set. For a predicate φ on U, the *selection* keeps the elements that satisfy φ, with their multiplicities:

$$\sigma_\varphi(B)(u) := B(u) \cdot [\varphi(u)],$$

where $[\,\cdot\,]$ is the Iverson bracket. For a function f on U we write $\sigma_{f=c}$ for the selection by the predicate $f(u) = c$. For a function $f : U \to U'$, the *image* of B adds the multiplicities of the elements that f identifies:

$$f_*B \in \mathrm{Bag}(U'), \qquad (f_*B)(u') := \sum_{\substack{u \in \mathrm{supp}(B) \\ f(u) = u'}} B(u).$$

It is the extension of the contribution $u \mapsto \{\{f(u)\}\}$ with values in $\mathrm{Bag}(U')$.

Lemma 2.3 (Calculus of bags)**.** *Let $f : U \to U'$ and $g : U' \to U''$ be functions, φ, ψ predicates on U, χ a predicate on U', and $\alpha' : U' \to S$ a contribution.*

(1) Homomorphisms. *The maps σ_φ and f_* are monoid homomorphisms.*

(2) Composition. $\sigma_\varphi \circ \sigma_\psi = \sigma_{\varphi \wedge \psi}$, $(g \circ f)_* = g_* \circ f_*$, *and* $\sigma_\chi \circ f_* = f_* \circ \sigma_{\chi \circ f}$*: a selection that reads only the image may be applied before or after* f_*.
(3) Aggregation. $\widehat{\alpha'} \circ f_* = \widehat{\alpha' \circ f}$.
(4) Partition. $B = \biguplus_{u' \in \mathrm{supp}(f_* B)} \sigma_{f=u'}(B)$ *for every* $B \in \mathrm{Bag}(U)$*, and* $(f_* B)(u')$ *is the number of elements of* $\sigma_{f=u'}(B)$*, counted with multiplicity.*

Proof. (1) The selection multiplies each multiplicity by 0 or 1, and f_* is the extension of a contribution. (2) The first identity holds pointwise. In the other two, both sides are homomorphisms out of the free monoid $\mathrm{Bag}(U)$, and they agree on a singleton $\{\{u\}\}$: the values are $\{\{g(f(u))\}\}$, and $\{\{f(u)\}\}$ if $\chi(f(u))$ and $\varnothing_{\mathrm{bag}}$ otherwise. (3) Both sides are homomorphisms with the value $\alpha'(f(u))$ on $\{\{u\}\}$. (4) Every $u \in \mathrm{supp}(B)$ has exactly one image $f(u)$, and this image lies in $\mathrm{supp}(f_* B)$; hence $\sum_{u'} B(u) \cdot [f(u) = u'] = B(u)$. The second claim is the definition of $f_* B$. □

For bags of rows, the selection is the selection of relational algebra, and the image under a map that forgets attributes is its projection under bag semantics, which adds the multiplicities of rows that become identical (Section 4.1). Part (4) is the reason for which grouping loses no row: the groups $\sigma_{f=u'}(B)$ partition B.

3 Mergeable aggregation

Section 2.2 extends every contribution $\alpha : U \to S$ into a commutative monoid $(S, \oplus, \mathbf{0}_S)$ to a unique homomorphism $\widehat{\alpha} : \mathrm{Bag}(U) \to S$ (2). We call S the *summary space* and its elements *summaries*. This section states what the homomorphism property provides. A table can be aggregated in parts, with only summaries exchanged (Section 3.1); an aggregate function is a homomorphism into a summary space followed by a map that is applied once, at the end (Section 3.2); and a grouped aggregation is the same construction, carried out for each key (Section 3.3). Section 3.4 relates this to prior work.

3.1 Partial aggregation

Because $\widehat{\alpha} : \mathrm{Bag}(U) \to S$ is a homomorphism,

$$\widehat{\alpha}(B_1 \uplus B_2) = \widehat{\alpha}(B_1) \oplus \widehat{\alpha}(B_2), \qquad \widehat{\alpha}(\varnothing_{\mathrm{bag}}) = \mathbf{0}_S,$$

as depicted in Figure 1. Consequently, if a table is stored as partitions $B = B_1 \uplus \cdots \uplus B_k$, then

$$\widehat{\alpha}(B) = \widehat{\alpha}(B_1) \oplus \cdots \oplus \widehat{\alpha}(B_k),$$

and each partition can be aggregated independently, with only the summaries $\widehat{\alpha}(B_i) \in S$ exchanged. This is the algebraic basis of partial aggregation in MapReduce [3]: the map phase applies α to each row, a combiner folds the results with $\oplus$ within a partition, and the reduce phase merges the partial summaries and applies γ once. Associativity and commutativity of $\oplus$ make the result independent of both the partitioning and the order in which partial summaries arrive, and are accordingly the conditions MapReduce imposes on a reduce function that is also used as a combiner [3]. Lin [10] formulates the same observation as a design principle, namely that the intermediate values of a MapReduce computation should form a monoid.

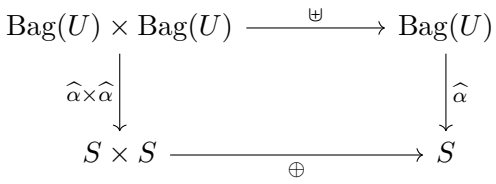


Figure 1: The aggregation homomorphism $\widehat{\alpha}$ preserves bag union: merging two partitions and then aggregating (right, then down) yields the same summary as aggregating each partition and then merging the summaries (down, then right).

3.2 Aggregates as a homomorphism followed by a finalizer

An aggregate function is a composite of a monoid homomorphism $\widehat{\alpha}$ and a *finalizer* γ:

$$\mathrm{Bag}(U) \xrightarrow{\widehat{\alpha}} S \xrightarrow{\gamma} V \cup \{\texttt{NULL}\},$$

where S is the summary space, the set of intermediate summaries, and γ extracts the reported value. The finalizer is applied to a merged summary and never before a merge: no function of two means computes the mean of the union. The arithmetic mean `AVG`, for instance, is not a homomorphism, but it factors through the summary $(\mathit{sum}, \mathit{count}) \in \mathbb{N}_0^2$ under componentwise addition, followed by a division. The composite is well defined provided that $(S, \oplus, \mathbf{0}_S)$ is a commutative monoid and γ is total on S; in particular, $\gamma(\mathbf{0}_S)$ determines the result on the empty bag. Where the combining operation has no identity, or where its identity differs from the value SQL reports for an empty group, we adjoin one: for a commutative semigroup $(T, *)$, we write $(T, *)_\perp$ for the commutative monoid $(T \cup \{\perp\}, *, \perp)$ and set $\gamma_\perp(\perp) = \texttt{NULL}$ and $\gamma_\perp(t) = t$ for $t \in T$.

Table 1 expresses the standard SQL aggregates in this form.

Aggregate	$(S, \oplus, \mathbf{0}_S)$	$\alpha([r, x])$	$\widehat{\alpha}(B)$	γ	$\gamma(\widehat{\alpha}(B))$
`COUNT`, `COUNT_BIG`	$(\mathbb{N}_0, +, 0)$	1	3	$n \mapsto n$	3
`SUM`	$(\mathbb{N}_0, +)_\perp$	x	27	$\gamma_\perp$	27
`PRODUCT`	$(\mathbb{N}_0, \times)_\perp$	x	700	$\gamma_\perp$	700
`MIN`	$(\mathbb{N}_0, \min)_\perp$	x	7	$\gamma_\perp$	7
`MAX`	$(\mathbb{N}_0, \max)_\perp$	x	10	$\gamma_\perp$	10
`AVG`	$(\mathbb{N}_0^2, +, (0, 0))$	$(x, 1)$	$(27, 3)$	γ_{avg}	9
`VARP`, `STDEVP`	$(\mathbb{N}_0^3, +, (0, 0, 0))$	$(1, x, x^2)$	$(3, 27, 249)$	γ_{varp}, $\sqrt{\gamma_{\mathrm{varp}}}$	2, $\sqrt{2}$
`VAR`, `STDEV`	$(\mathbb{N}_0^3, +, (0, 0, 0))$	$(1, x, x^2)$	$(3, 27, 249)$	γ_{var}, $\sqrt{\gamma_{\mathrm{var}}}$	3, $\sqrt{3}$
`COUNT(DISTINCT)`	$(\mathcal{P}_{\mathrm{fin}}(\mathbb{N}_0), \cup, \emptyset)$	$\{x\}$	$\{7, 10\}$	$A \mapsto \lvert A\rvert$	2
`APPROX_COUNT_DISTINCT`	$(\mathbb{N}_0^m, \max, \mathbf{0})$	$\rho(x)\, e_{i(x)}$	sketch	γ_{HLL}	≈ 2
`CHECKSUM_AGG`	$(\{0,1\}^{32}, \mathrm{xor}, \mathbf{0})$	$\mathrm{bits}(x)$	$\mathrm{bits}(7)$	bits^{-1}	7
`MEDIAN`, `PERCENTILE_*`, `QUANTILE`	$(\mathrm{Bag}(\mathbb{N}_0), \uplus, \varnothing_{\mathrm{bag}})$	$\{\{x\}\}$	$\{\{7, 10, 10\}\}$	γ_{med}	10
`STRING_AGG`	$(\mathrm{Bag}(\Sigma^*), \uplus, \varnothing_{\mathrm{bag}})$	$\{\{\mathrm{str}(x)\}\}$	$\{\{7, 10, 10\}\}$	γ_{join}	`7,10,10`

Table 1: Aggregates as composites $\gamma \circ \widehat{\alpha}$, where x denotes the value of a row $[r, x]$ and B is the example bag of Section 2.2. Addition and maximum on tuples are componentwise.

Cost of the summary space. The summary space S also determines the cost of this decomposition, and thereby recovers the taxonomy of Gray et al. [8]. An aggregate is *distributive* if γ is the identity, as for `COUNT`; *algebraic* if S consists of tuples of fixed size, as for `AVG` and `VAR`; and *holistic* if no summary of bounded size suffices, as for `MEDIAN`, where S must retain the bag of values and partial aggregation reduces no communication.

3.3 Grouped aggregation

Grouped aggregation is an instance of the same construction. For a key function $\kappa : U \to K$, assign to each key k the summary $\widehat{\alpha}(\sigma_{\kappa=k}(B))$ of the rows with that key. Since $\sigma_{\kappa=k}$ and $\widehat{\alpha}$ are homomorphisms,

$$\widehat{\alpha}\big(\sigma_{\kappa=k}(B_1 \uplus B_2)\big) = \widehat{\alpha}\big(\sigma_{\kappa=k}(B_1)\big) \oplus \widehat{\alpha}\big(\sigma_{\kappa=k}(B_2)\big) :$$

the group summaries of a partitioned table are merged key by key, and a key that does not occur in a partition has the summary $\mathbf{0}_S$ there. The groups partition the table (Lemma 2.3(4)), so merging all group summaries gives the summary of the table,

$$\bigoplus_{k \in \operatorname{supp}(\kappa_* B)} \widehat{\alpha}(\sigma_{\kappa=k}(B)) = \widehat{\alpha}(B).$$

Section 4.2 develops this for relations, where the key is a set X of attributes: the result keeps X and holds, for each key value, one element of the summary space S, into which all other attributes have been squashed.

3.4 Related work

The observation that only part of an aggregate needs to be a homomorphism is old and has been made independently in several communities.

- **Databases.** Gray et al. [8] classify aggregate functions as *distributive*, *algebraic*, or *holistic*: a distributive function can be recomputed from its own values on a partition of the input, an algebraic function can be recomputed from a fixed-size tuple of such sub-aggregates followed by a final function, and a holistic function admits no constant-size summary.
- **MapReduce combiners.** The combiner of MapReduce [3] is sound only when the reduction is associative and commutative.
- **Decomposable aggregation.** Yu, Gunda, and Isard [16] make the general case explicit by calling an aggregation *decomposable* when it splits into an initial reduction, an associative and commutative combine step, and a final reduction.
- **Sawzall.** Sawzall [13] takes the same view at the language level by restricting programs to a per-record emit phase feeding predefined aggregators, which Lämmel [9] reconstructs as list homomorphisms into monoids.
- **List homomorphisms.** Lämmel's reading goes back to the Bird–Meertens formalism [2], in which a homomorphism out of the free monoid factors as a map followed by a reduce; the third homomorphism theorem [7] characterises the functions that admit such a factorisation.
- **Semiring aggregators.** Emoto, Fischer, and Hu [5] strengthen the requirement on the aggregator to a semiring homomorphism in order to derive efficient MapReduce programs.
- **Aggregation over annotated relations.** Closest to our setting, Amsterdamer, Deutch, and Tannen [1] model SQL aggregation over annotated relations by commutative monoids.

We follow these lines in isolating the homomorphic core, but we state it over bags rather than lists, so that commutativity is built into the domain instead of being a side condition on the combine operator, and we keep the finalizer as a separate, non-homomorphic map. We also treat the empty group explicitly, since the identity of the combining operation need not coincide with the value SQL reports there.

4 Relations and squashes

A relation is a bag of rows over a schema. This section keeps a set X of its attributes and squashes all others into a summary space. It defines the objects that result, the summary-valued bags, and states what can and what cannot be computed from them without the rows.

4.1 Relations

Let $\mathcal{R}$ be a relation over a schema Σ, which is a finite set of attributes (columns). For each attribute $A \in \Sigma$, let $\mathrm{Dom}(A)$ denote its value domain. For a subset $X \subseteq \Sigma$,

$$\mathrm{Row}(X) := \prod_{A \in X} \mathrm{Dom}(A)$$

is the set of possible row values over the attributes in X, and for $r \in \mathrm{Row}(X)$ and $X' \subseteq X$ we write $r[X'] \in \mathrm{Row}(X')$ for the restriction of r to X'. The set $\mathrm{Row}(\emptyset)$ has a single element, the empty row.

Throughout this paper, we mainly focus on the following aspects of relations: the partition of the schema, the order on sets of filtered attributes, the relation as a bag with its groups, and the lattice of groupings. The summaries and the fibers of the groups follow in Sections 4.2 to 4.3.

Schema partitioned by keys Any hierarchical reduction query defines a three-way disjoint partition of the schema Σ:

$$\Sigma = \mathcal{K} \sqcup \mathcal{V} \sqcup \mathcal{F}$$

1. **Grouping Attributes ($\mathcal{K}$)**: The columns over which equivalence classes (groups) are formed or along which downward tree levels branch. In a tree of depth d where each level $i \in \{1, \ldots, d\}$ branches on a disjoint subset of key columns $K_i \subseteq \mathcal{K}$, the grouping attributes are further partitioned into an ordered **filtration of key sets**:
$$\mathcal{K} = K_1 \sqcup K_2 \sqcup \cdots \sqcup K_d.$$
2. **Value Attributes ($\mathcal{V}$)**: Numeric columns mapped into an algebraic summary space via an aggregation homomorphism.
3. **Auxiliary Filter Attributes ($\mathcal{F}$)**: All the other columns.

Subfilters A key value $x \in \mathrm{Row}(X)$ stands for the filter $X = x$, a conjunction of one equality for each attribute of X. A filter that includes the conditions of another filter selects a subset of its rows. We order filters accordingly, with the notation used for subtypes and subtraits: the more restrictive filter is the smaller one.

Definition 4.1 (Subfilter order)**.** For $X, X' \subseteq \Sigma$ we say that X is a *subfilter* of X', and X' a *superfilter* of X, and write

$$X <: X' \quad :\Longleftrightarrow \quad X' \subseteq X.$$

For key values $x \in \mathrm{Row}(X)$ and $x' \in \mathrm{Row}(X')$ we write $x <: x'$ if $X <: X'$ and $x[X'] = x'$; the filter $X = x$ then includes every condition of $X' = x'$ and implies it. For $X <: X'$ the *upcast*

$$\mathrm{up}^X_{X'} : \mathrm{Row}(X) \to \mathrm{Row}(X'), \qquad x \mapsto x[X'],$$

forgets the conditions on $X \setminus X'$; it is the unique map with $x <: \mathrm{up}^X_{X'}(x)$.

The relation $<:$ is a partial order: $X <: X$, and $X <: X'$, $X' <: X''$ imply $X <: X''$, with

$$\mathrm{up}_X^X = \mathrm{id}, \qquad \mathrm{up}_{X''}^{X'} \circ \mathrm{up}_{X'}^{X} = \mathrm{up}_{X''}^{X}\,. \tag{3}$$

The empty filter $\emptyset$ is the top element and Σ the bottom element: $\Sigma <: X <: \emptyset$ for every X. The greatest lower bound of X and Y is $X \cup Y$, the filter with the conditions of both. As for a subtrait, which includes the requirements of its supertraits and therefore has fewer implementors, a subfilter includes more conditions and selects fewer rows. An upcast always exists and loses the conditions it forgets; a downcast from X' to X needs the values of the attributes in $X \setminus X'$.

Relations as bags, and their groups We model the relation $\mathcal{R}$ as a finite bag

$$B_{\mathcal{R}} \in \mathrm{Bag}(\mathrm{Row}(\Sigma)),$$

where $B_{\mathcal{R}}(r) \in \mathbb{N}_0$ denotes the multiplicity of the full row value r. The operations of relational algebra under bag semantics are the selection and the image of Section 2.2. The selection $\sigma_\varphi(\mathcal{R})$ by a predicate φ over Σ has the bag $\sigma_\varphi(B_{\mathcal{R}})$; it keeps the rows that satisfy φ, with their multiplicities. For a key value $x \in \mathrm{Row}(X)$ we write $\sigma_{X=x}$ for the selection by the filter $r[X] = x$. The projection onto $X \subseteq \Sigma$ has the bag

$$B_X := (\mathrm{up}_X^{\Sigma})_*\, B_{\mathcal{R}} \in \mathrm{Bag}(\mathrm{Row}(X)), \qquad B_X(x) = \sum_{\substack{r \in \mathrm{supp}(B_{\mathcal{R}}) \\ r[X]=x}} B_{\mathcal{R}}(r), \tag{4}$$

the *bag projection* onto X, the image of $B_{\mathcal{R}}$ under the upcast; it adds the multiplicities of rows that become identical. Section 4.2 places it among the squashes onto X: it is the one whose summary space is $\mathbb{N}_0$.

Two rows *agree on* X, written $r \sim_X r'$, if $r[X] = r'[X]$. For each key value $x \in \mathrm{Row}(X)$, define its *group*, or *projection fiber*, by

$$\mathrm{Fib}_X(x) := \{r \in \mathrm{supp}(B_{\mathcal{R}}) \mid r[X] = x\}\,.$$

Thus, $\mathrm{Fib}_X(x)$ contains all distinct full row values present in $\mathcal{R}$ that agree on the grouping attributes X with key value x, and $B_X(x)$ counts the row occurrences in the group identified by x. The non-empty groups are precisely the equivalence classes of $\sim_X$ restricted to $\mathrm{supp}(B_{\mathcal{R}})$: for every $r \in \mathrm{supp}(B_{\mathcal{R}})$,

$$[r]_X := \{r' \in \mathrm{supp}(B_{\mathcal{R}}) \mid r' \sim_X r\} = \mathrm{Fib}_X(r[X]).$$

Consequently,

$$\mathrm{supp}(B_{\mathcal{R}})/{\sim_X} = \{\mathrm{Fib}_X(x) \mid x \in \mathrm{supp}(B_X)\}\,,$$

and $[r]_X \mapsto r[X]$ defines a canonical bijection between the quotient set of groups $\mathrm{supp}(B_{\mathcal{R}})/{\sim_X}$ and the set of observed grouping-key values $\mathrm{supp}(B_X)$.

Lemma 4.2 (Refinement)**.** *For all $X, Y \subseteq \Sigma$,*

$$\sim_{X \cup Y} = \sim_X \cap \sim_Y.$$

Hence $X \mapsto \sim_X$ is an order-preserving map from the subsets of Σ under the subfilter order to the lattice of equivalence relations on $\mathrm{Row}(\Sigma)$ *ordered by inclusion, and it sends the*

meet $X \cup Y$ to the meet: a subfilter refines the grouping. In particular, if $X <: X'$, then for every $x' \in \mathrm{Row}(X')$,

$$\mathrm{Fib}_{X'}(x') = \bigsqcup_{\substack{x \in \mathrm{supp}(B_X) \\ x <: x'}} \mathrm{Fib}_X(x), \qquad B_{X'}(x') = \sum_{\substack{x \in \mathrm{supp}(B_X) \\ x <: x'}} B_X(x), \tag{5}$$

so that $\mathrm{Fib}_X(x) \subseteq \mathrm{Fib}_{X'}(x')$ and $B_X(x) \leq B_{X'}(x')$ whenever $x <: x'$.

Proof. Rows agree on $X \cup Y$ iff they agree on X and on Y. For the decomposition, each $r \in \mathrm{supp}(B_{\mathcal{R}})$ with $r[X'] = x'$ lies in exactly one group over X, namely $\mathrm{Fib}_X(r[X])$, and $r[X] <: x'$ because $r[X][X'] = r[X'] = x'$. Summing $B_{\mathcal{R}}$ over the disjoint union gives the identity for $B_{X'}(x')$. □

Lattice Homomorphism For a set $X \subseteq \mathcal{K}$ of keys, let $\Pi(X)$ denote the partition of the rows of $\mathcal{R}$ obtained by grouping rows that agree on all keys in X; its blocks are the non-empty groups $\mathrm{Fib}_X(x)$. Adding keys makes this partition finer: if $X <: X'$, that is, $X' \subseteq X$, then $\Pi(X) \preceq \Pi(X')$. Thus, the map $X \mapsto \Pi(X)$ preserves the subfilter order, reverses inclusion, and maps the union of key sets to the meet of their partitions (Lemma 4.2):

$$\Pi(X \cup Y) = \Pi(X) \wedge \Pi(Y).$$

Here, the meet is the coarsest partition that refines both $\Pi(X)$ and $\Pi(Y)$; its blocks are the nonempty intersections of blocks from the two partitions.

Equivalently, selecting records that satisfy both $X = x$ and $Y = y$ gives the same result regardless of which condition is applied first:

$$\sigma_{X=x \wedge Y=y}(\mathcal{R}) = \sigma_{X=x}(\sigma_{Y=y}(\mathcal{R})) = \sigma_{Y=y}(\sigma_{X=x}(\mathcal{R}))\,.$$

Consequently, if the level key sets $K_1, \ldots, K_d$ together cover $\mathcal{K}$, applying them in any order produces the same final partition. For every permutation π of $\{1, \ldots, d\}$,

$$\Pi\Big(K_{\pi(1)} \cup \cdots \cup K_{\pi(d)}\Big) = \Pi(\mathcal{K}).$$

In other words, changing the order of the levels may change the intermediate groupings, but it does not change which records end up together at the leaves.

4.2 Summary-valued bags: kept and squashed attributes

A grouped aggregation keeps a set X of attributes and reduces everything else to the summary space. `GROUP BY` X returns one row for each key value. The rows of a group agree on X, so all that distinguishes them lies in the remaining attributes $\Sigma \setminus X$, and the aggregate replaces this information by one element of a commutative monoid S, computed from the attributes outside X of the rows of the group. The attributes of X are *kept*, and the attributes of $\Sigma \setminus X$ are *squashed* into S. The letter S stands for *summary*: we call $(S, \oplus, \mathbf{0}_S)$ the summary monoid, its carrier S the *summary space*, and its elements summaries.

What is left of the squashed attributes is decided by the summary space and the contribution, and by nothing else. The bag projection B_X of (4) is the case $S = \mathbb{N}_0$: of each group it retains the number of row occurrences. This number is a summary like any other, and $\mathbb{N}_0$ is one summary space among many; a squash does not reduce the attributes of $\Sigma \setminus X$

to a number unless S consists of numbers. With $S = \mathbb{N}_0 \times \mathbb{R}$ a group is reduced to a count and a sum, with a sketch as S to a sketch, and with $S = \mathrm{Bag}(\mathrm{Row}(\Sigma \setminus X))$ to the bag of its residuals, of which nothing is lost. This subsection defines the squash and the objects it produces.

Kept and squashed attributes For $X \subseteq \Sigma$ write

$$X^{\mathsf{c}} := \Sigma \setminus X.$$

A row $r \in \mathrm{Row}(\Sigma)$ is determined by its two restrictions $r[X]$ and $r[X^{\mathsf{c}}]$. For $x \in \mathrm{Row}(X)$ and $t \in \mathrm{Row}(X^{\mathsf{c}})$ we write $(x, t) \in \mathrm{Row}(\Sigma)$ for the row with these restrictions, so that $\mathrm{Row}(\Sigma) = \mathrm{Row}(X) \times \mathrm{Row}(X^{\mathsf{c}})$. With respect to X, we call $r[X]$ the *key part* of r and $r[X^{\mathsf{c}}]$ its *residual*.

Definition 4.3 (Fiber)**.** Let $X \subseteq \Sigma$ and $x \in \mathrm{Row}(X)$. The *fiber* of $\mathcal{R}$ over x is the bag of the residuals of the rows with key part x:

$$\mathrm{Fib}_X^{X^{\mathsf{c}}}(x) \in \mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}})), \qquad \mathrm{Fib}_X^{X^{\mathsf{c}}}(x)(t) := B_{\mathcal{R}}(x, t).$$

The fiber over x is the group of x without its key part, which is the same in all rows of the group: the selection $\sigma_{X=x}(B_{\mathcal{R}})$ has the multiplicity $\mathrm{Fib}_X^{X^{\mathsf{c}}}(x)(t)$ at the row (x, t) and 0 at every row with another key part. The fiber has $B_X(x)$ elements, counted with multiplicity, and it is empty unless $x \in \mathrm{supp}(B_X)$. The superscript names the attributes of which the fiber retains the values; Section 4.3 admits other sets in its place. Nothing is lost by passing to the fibers, since $B_{\mathcal{R}}(x, t) = \mathrm{Fib}_X^{X^{\mathsf{c}}}(x)(t)$: the relation is the family of its fibers, labelled by the key values.

The squash A fiber is a bag, and a bag is aggregated by the extension $\widehat{\alpha}$ of a contribution (Section 2.2). Aggregating every fiber gives one summary for each key value.

Definition 4.4 (Squash, S-bag)**.** Let $X \subseteq \Sigma$, let $(S, \oplus, \mathbf{0}_S)$ be a commutative monoid, and let $\alpha : \mathrm{Row}(\Sigma) \to S$ be a contribution. The *squash of $\mathcal{R}$ onto X* is the function

$$G_X : \mathrm{Row}(X) \to S, \qquad G_X(x) := \widehat{\alpha}(\sigma_{X=x}(B_{\mathcal{R}})) = \bigoplus_{t \in \mathrm{supp}(\mathrm{Fib}_X^{X^{\mathsf{c}}}(x))} \mathrm{Fib}_X^{X^{\mathsf{c}}}(x)(t) \cdot \alpha(x, t).$$

It keeps the attributes X, whose values index its entries, and it squashes the attributes X^{c} into the summary space S: the entry at x is the summary of the fiber over x, an element of S. We write G_X^{α} when the contribution has to be named.

An *S-bag over X* is a function $G : \mathrm{Row}(X) \to S$ with finite support

$$\mathrm{supp}(G) := \{x \in \mathrm{Row}(X) \mid G(x) \neq \mathbf{0}_S\}.$$

We write $S\langle X \rangle$ for the set of all S-bags over X. It is a commutative monoid under the pointwise operation $(G \oplus G')(x) := G(x) \oplus G'(x)$, whose identity is the S-bag with the constant value $\mathbf{0}_S$.

The two expressions for $G_X(x)$ agree by (2). Written with the contribution $\alpha_x(t) := \alpha(x, t)$ of a residual at the key value x, the definition reads

$$G_X(x) = \widehat{\alpha_x}(\mathrm{Fib}_X^{X^{\mathsf{c}}}(x)), \qquad \widehat{\alpha_x} : \mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}})) \to S. \tag{6}$$

The entry at x is the image of the fiber over x under a monoid homomorphism. This is the precise sense in which X^{c} is squashed into the summary space: the bag of the residuals of a group, an element of the free commutative monoid $\mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}}))$, is replaced by its image in S, and of the residuals the summary retains what the homomorphism retains, and nothing else.

We say that a contribution α *reads* a set $W \subseteq \Sigma$ of attributes if $\alpha(r) = \beta(r[W])$ for a map $\beta : \mathrm{Row}(W) \to S$, and that it *does not read* an attribute a if it reads $\Sigma \setminus \{a\}$. In the standard situation α reads value attributes and X consists of key and filter attributes. Then α reads X^{c}, the contribution α_x is the same for all x, and one homomorphism squashes every fiber. If α also reads a kept attribute, its value enters the summary as a parameter that is constant on the fiber; no aggregation over it takes place.

Examples For `COUNT` the summary space is $\mathbb{N}_0$, with $S = (\mathbb{N}_0, +, 0)$ and $\alpha(r) = 1$, and the squash is the bag projection,

$$G_X = B_X,$$

which retains of each fiber the number of its elements. The bag projection is therefore not a separate construction: it is the squash onto X into the summary space $\mathbb{N}_0$, and a multiplicity is a summary. A bag over $\mathrm{Row}(X)$ is an $\mathbb{N}_0$-bag over X: $\mathrm{Bag}(\mathrm{Row}(X)) = \mathbb{N}_0\langle X\rangle$, with $\uplus$ as the pointwise operation. We keep the word *bag* for this case and say *S-bag*, a bag of summaries, otherwise. For `SUM` over a value attribute w, $G_X(x)$ is the sum of $r[w]$ over the row occurrences of the group, and similarly for every row of Table 1: G_X is the result of `GROUP BY` X before the finalizer is applied.

In the example of Section 2.2, $\Sigma = \{\mathit{region}, \mathit{value}\}$, the bag is $\{\{[\mathrm{KR}, 10], [\mathrm{KR}, 10], [\mathrm{US}, 7]\}\}$, and `SUM` has the contribution $\alpha(r) = r[\mathit{value}]$. The three squashes are

$$\begin{aligned}
X = \{\mathit{region}\} : &\qquad G_X(\mathrm{KR}) = 20, &\qquad G_X(\mathrm{US}) = 7,\\
X = \emptyset : &\qquad G_\emptyset = 27,\\
X = \Sigma : &\qquad G_\Sigma([\mathrm{KR}, 10]) = 20, &\qquad G_\Sigma([\mathrm{US}, 7]) = 7.
\end{aligned}$$

The fiber over KR is the bag $\{\{10, 10\}\}$ of values, and the squash onto $\{\mathit{region}\}$ replaces it by its sum. The last two lines are the extreme cases, which occur for every relation. For $X = \emptyset$ every attribute is squashed: $\mathrm{Row}(\emptyset)$ has a single element, we identify $S\langle\emptyset\rangle$ with S, and $G_\emptyset = \widehat{\alpha}(B_\mathcal{R})$ is the summary of the table. For $X = \Sigma$ no attribute is squashed: the fiber over a row r consists of $B_\mathcal{R}(r)$ copies of the empty row, and

$$G_\Sigma(r) = B_\mathcal{R}(r) \cdot \alpha(r), \tag{7}$$

so that only the duplicates of a row are merged.

S-bags are squashes The squash of a relation is an S-bag, and every S-bag arises in this way.

Lemma 4.5 (S-bags are squashes)**.** *Let X be a set of attributes and S a commutative monoid.*

(1) *For every relation $\mathcal{R}$ over a schema $\Sigma \supseteq X$ and every contribution $\alpha : \mathrm{Row}(\Sigma) \to S$, the squash G_X is an S-bag over X, and $\mathrm{supp}(G_X) \subseteq \mathrm{supp}(B_X)$.*

(2) *Conversely, let $G \in S\langle X\rangle$. Let $\Sigma_S := X \sqcup \{a_S\}$ for a new attribute a_S with $\mathrm{Dom}(a_S) = S$, let $\mathcal{R}_G$ be the relation over Σ_S that has exactly one row $(x, G(x))$ for each $x \in \mathrm{supp}(G)$, and let $\alpha(r) := r[a_S]$. Then the squash of $\mathcal{R}_G$ onto X is G.*

Proof. (1) If no row of $\mathrm{supp}(B_{\mathcal{R}})$ has the key part x, then $\sigma_{X=x}(B_{\mathcal{R}})$ is empty and $G_X(x) = \widehat{\alpha}(\varnothing_{\mathrm{bag}}) = \mathbf{0}_S$. Hence $\mathrm{supp}(G_X) \subseteq \mathrm{supp}(B_X)$, which is finite. (2) The fiber of $\mathcal{R}_G$ over $x \in \mathrm{supp}(G)$ is the single residual $G(x)$, with the contribution $G(x)$, and all other fibers are empty. □

By part (2), the general notion adds nothing to the squash. An S-bag over X is a relation of which the attributes outside X have been squashed, written with the summaries in the place of the squashed attributes. A table that was aggregated in advance is given in this form. In the inclusion of part (1), a key value of $\mathrm{supp}(B_X)$ is missing from $\mathrm{supp}(G_X)$ when the summary of a nonempty fiber is $\mathbf{0}_S$; Proposition 4.15 states two conditions that exclude this.

Operations The operations on S-bags are those that can be carried out on a relation after a part of its attributes has been squashed.

Definition 4.6 (Operations on S-bags)**.** Let $G \in S\langle X\rangle$.

- *Merge.* For $X' \subseteq X$, that is, $X <: X'$, the S-bag $(\mathrm{up}^X_{X'})_* G \in S\langle X'\rangle$ merges the summaries of the key values with the same upcast:

$$((\mathrm{up}^X_{X'})_* G)(x') := \bigoplus_{\substack{x \in \mathrm{supp}(G) \\ x <: x'}} G(x).$$

  It squashes the attributes $X \setminus X'$ in addition to those that G has squashed already.
- *Total.* $\mathrm{tot}(G) := \bigoplus_{x \in \mathrm{supp}(G)} G(x) \in S$ is the merge of all summaries. It is the merge for $X' = \emptyset$, which squashes every attribute.
- *Selection.* For a predicate φ over X, the S-bag $\sigma_\varphi(G) \in S\langle X\rangle$ keeps the summaries of the key values that satisfy φ:

$$\sigma_\varphi(G)(x) := \begin{cases} G(x), & \varphi(x), \\ \mathbf{0}_S, & \text{otherwise.} \end{cases}$$

- *Change of summary.* For a monoid homomorphism $h : S \to S'$, the composite $h \circ G$ is an S'-bag over X.

For bags, the merge is the image under the upcast and the selection is the selection of Section 2.2. Two operations are missing on purpose. A selection is defined only for a predicate over the kept attributes, and the merge only leads from X to a subset of X: there is no operation that returns a squashed attribute to the kept ones.

Lemma 4.7 (Calculus of S-bags)**.** *Let $X'' \subseteq X' \subseteq X$, let φ, ψ be predicates over X and χ a predicate over X', which is also a predicate over X by $\chi(x) := \chi(x[X'])$, and let $h : S \to S'$ be a monoid homomorphism.*

(1) Homomorphisms. *The maps* $(\mathrm{up}^X_{X'})_*$, tot, σ_φ, *and* $G \mapsto h \circ G$ *are monoid homomorphisms.*
(2) Squashing in steps. $(\mathrm{up}^{X'}_{X''})_* \circ (\mathrm{up}^X_{X'})_* = (\mathrm{up}^X_{X''})_*$, $(\mathrm{up}^X_X)_* = \mathrm{id}$, *and* $\mathrm{tot} \circ (\mathrm{up}^X_{X'})_* = \mathrm{tot}$.
(3) Selection. $\sigma_\varphi \circ \sigma_\psi = \sigma_{\varphi \wedge \psi}$, *and* $\sigma_\chi \circ (\mathrm{up}^X_{X'})_* = (\mathrm{up}^X_{X'})_* \circ \sigma_\chi$*: a selection on attributes that remain kept may be applied before or after the others are squashed.*
(4) Change of summary. $h \circ ((\mathrm{up}^X_{X'})_* G) = (\mathrm{up}^X_{X'})_* (h \circ G)$, $h \circ \sigma_\varphi(G) = \sigma_\varphi(h \circ G)$, *and* $h(\mathrm{tot}(G)) = \mathrm{tot}(h \circ G)$.

Proof. All sums below are finite, and a sum over $\mathrm{supp}(G)$ may be taken over any finite set that contains it, since the additional terms are $\mathbf{0}_S$. (1) The maps are defined pointwise or by finite sums of values, and they commute with $\oplus$ by associativity and commutativity; h sends $\mathbf{0}_S$ to $\mathbf{0}_{S'}$, so $h \circ G$ has finite support. (2) By (3), a key value x satisfies $x <: x''$ if and only if $x <: x'$ and $x' <: x''$ for $x' = x[X']$, so the iterated sum is the sum of $G(x)$ over all $x <: x''$. The total is the case $X'' = \emptyset$. (3) The first identity holds pointwise. For the second, $\chi(x) = \chi(x')$ for every $x <: x'$, so both sides have the value $\bigoplus_{x<:x'} G(x)$ at x' if $\chi(x')$ and $\mathbf{0}_S$ otherwise. (4) A homomorphism commutes with finite sums and sends $\mathbf{0}_S$ to $\mathbf{0}_{S'}$. □

Squashing a relation For the squashes of a relation, the operations are the relational operations that they stand for.

Proposition 4.8 (Squashing a relation)**.** *Let $X' \subseteq X \subseteq \Sigma$, and let G_X be the squashes of $\mathcal{R}$ for a contribution $\alpha : \mathrm{Row}(\Sigma) \to S$.*

(1) Squashing in steps. *The squash onto X' is the merge of the squash onto X: $G_{X'} = (\mathrm{up}^X_{X'})_* G_X$, that is,*

$$G_{X'}(x') = \bigoplus_{\substack{x \in \mathrm{supp}(G_X) \\ x <: x'}} G_X(x) \qquad \text{for every } x' \in \mathrm{Row}(X').$$

In particular, $G_X = (\mathrm{up}^\Sigma_X)_* G_\Sigma$ with G_Σ as in (7)*, and $\mathrm{tot}(G_X) = G_\emptyset = \widehat{\alpha}(B_\mathcal{R})$ for every X.*

(2) Selection on kept attributes. *Let φ be a predicate over X, so that $\varphi(r)$ depends on $r[X]$ only. Then the squash of $\sigma_\varphi(\mathcal{R})$ onto X is $\sigma_\varphi(G_X)$.*

(3) Change of summary. *For a monoid homomorphism $h : S \to S'$, $h \circ G^\alpha_X = G^{h\circ\alpha}_X$.*

(4) Additivity. *If $B_\mathcal{R} = B_1 \uplus B_2$, then G_X is the pointwise merge of the squashes of B_1 and of B_2 onto X. Thus the squash onto X is a monoid homomorphism from* $\mathrm{Bag}(\mathrm{Row}(\Sigma))$ *to $S\langle X\rangle$.*

Proof. (1) Let $x' \in \mathrm{Row}(X')$. The groups over X partition the bag $\sigma_{X'=x'}(B_\mathcal{R})$ by Lemma 2.3(4), applied to this bag and the upcast up^Σ_X, whose image is supported in $\mathrm{supp}(B_X)$. Since $r[X] = x$ implies $r[X'] = x[X']$, the composite $\sigma_{X=x} \circ \sigma_{X'=x'}$ is $\sigma_{X=x}$ if $x <: x'$ and has the value $\varnothing_{\mathrm{bag}}$ otherwise. Hence

$$\sigma_{X'=x'}(B_\mathcal{R}) = \biguplus_{\substack{x \in \mathrm{supp}(B_X) \\ x <: x'}} \sigma_{X=x}(B_\mathcal{R}),$$

and applying the homomorphism $\widehat{\alpha}$ gives $G_{X'}(x') = \bigoplus_{x\in\mathrm{supp}(B_X),\, x<:x'} G_X(x)$. The terms with $x \notin \mathrm{supp}(G_X)$ are $\mathbf{0}_S$. For $X = \Sigma$ the bag $\sigma_{\Sigma=r}(B_\mathcal{R})$ consists of $B_\mathcal{R}(r)$ copies of r, which gives (7); for $X' = \emptyset$ the merge is the total, and $\sigma_{\emptyset=x'}$ selects every row. (2) Since φ depends on $r[X]$ only, $\sigma_{X=x}(\sigma_\varphi(B_\mathcal{R}))$ is $\sigma_{X=x}(B_\mathcal{R})$ if $\varphi(x)$ and $\varnothing_{\mathrm{bag}}$ otherwise. (3) The homomorphisms $h \circ \widehat{\alpha}$ and $\widehat{h \circ \alpha}$ agree on singletons. (4) The selection $\sigma_{X=x}$ and $\widehat{\alpha}$ are homomorphisms. □

Part (1) is the reason for which a squash produces summaries in a monoid and not reported values: a summary of the attributes X^c becomes a summary of the larger set X'^c by merging, without the rows. The attributes may therefore be squashed in any number of steps and in any order, with the same result, and every squash lies between G_Σ, in which

only duplicates are merged, and $G_\emptyset$, the summary of the table. The second identity of (5) is the case of `COUNT`. In the terms of Definition 4.1, a superfilter keeps fewer attributes and squashes more: the merge follows the upcast.

What is squashed is gone The proposition lists what a squash still allows. The two operations that are missing in Definition 4.6 are in fact impossible.

Proposition 4.9 (No way back)**.** *Let $X \subseteq \Sigma$, and let $a \in X^\mathsf{c}$ be an attribute with at least two values that α does not read. Suppose that α is not the constant $\mathbf{0}_S$. Then there are relations $\mathcal{R}_1$ and $\mathcal{R}_2$ over Σ with the same squash onto X such that*

(1) *their squashes onto X' differ, for every $X' \subseteq \Sigma$ that contains a; and*
(2) *the squashes of $\sigma_\varphi(\mathcal{R}_1)$ and $\sigma_\varphi(\mathcal{R}_2)$ onto X differ, for a predicate φ over $\{a\}$.*

Hence G_X determines neither the squash onto a set that keeps a nor the squash of a selection that reads a. In particular, if every attribute has at least two values and α reads X'^c, then G_X determines $G_{X'}$ for every relation over Σ if and only if $X' \subseteq X$.

Proof. Choose a row r_1 with $\alpha(r_1) \neq \mathbf{0}_S$ and a row r_2 that differs from r_1 exactly in the attribute a; then $\alpha(r_2) = \alpha(r_1)$. Let $\mathcal{R}_i$ have the bag $\{\{r_i\}\}$. Since $a \notin X$, the two rows have the same key part over X, and both squashes onto X have the single nonzero entry $\alpha(r_1)$ at this key value. (1) If $a \in X'$, then $r_1[X'] \neq r_2[X']$. The squash of $\mathcal{R}_1$ onto X' has the entry $\alpha(r_1) \neq \mathbf{0}_S$ at $r_1[X']$, and the squash of $\mathcal{R}_2$ has the entry $\mathbf{0}_S$ there. (2) For $\varphi \equiv (a = r_1[a])$, $\sigma_\varphi(\mathcal{R}_1) = \mathcal{R}_1$ and $\sigma_\varphi(\mathcal{R}_2)$ is empty. For the last claim, if $X' \subseteq X$, then $G_{X'}$ is the merge of G_X by Proposition 4.8(1); otherwise apply (1) to an attribute $a \in X' \setminus X$. □

Table 2 collects the two propositions. A filter on an attribute that is to be squashed must therefore be applied before the squash. It need not be applied to the relation: it can be squashed together with its attribute, by a contribution that is $\mathbf{0}_S$ on the rows that fail the filter (Lemma 5.10). A regrouping by a squashed attribute is possible only if the summary has kept the values of that attribute, as the fibers of Section 4.3 do.

Operation on $\mathcal{R}$	On the squash G_X	Reference
Squash further, onto $X' \subseteq X$	merge $(\mathrm{up}^X_{X'})_*$	Proposition 4.8(1)
Select by a predicate over X	selection σ_φ	Proposition 4.8(2)
Aggregate with $h \circ \alpha$ instead of α	change of summary $h \circ G_X$	Proposition 4.8(3)
Add rows	pointwise merge	Proposition 4.8(4)
Select by a predicate that reads $a \in X^\mathsf{c}$	not determined by G_X	Proposition 4.9(2)
Squash onto a set that contains $a \in X^\mathsf{c}$	not determined by G_X	Proposition 4.9(1)

Table 2: What can be computed from the squash G_X of a relation, without the relation. In the last two lines a is an attribute that the contribution does not read.

Corollary 4.10 (A relation as an S-bag)**.** *Let $Z \subseteq \Sigma$. For every predicate φ over Z and every $X \subseteq Z$, the squash of the selected relation $\sigma_\varphi(\mathcal{R})$ onto X is*

$$(\mathrm{up}^Z_X)_* \, \sigma_\varphi(G_Z).$$

Hence every summary $\widehat{\alpha}(\sigma_{\varphi \wedge X=x}(B_{\mathcal{R}}))$ in which φ and X read attributes of Z only is computed from G_Z, without $B_{\mathcal{R}}$.

Proof. By Proposition 4.8(2), the squash of $\sigma_\varphi(\mathcal{R})$ onto Z is $\sigma_\varphi(G_Z)$; apply part (1) to $\sigma_\varphi(\mathcal{R})$. □

The corollary gives the partition $\Sigma = \mathcal{K} \sqcup \mathcal{V} \sqcup \mathcal{F}$ of the schema its meaning. A contribution reads the value attributes, which are therefore squashed in every S-bag that we consider. The filter attributes are read by predicates and must be kept until the predicates have been applied. The key attributes are kept for as long as the result is to be grouped by them. For $Z = \mathcal{K} \sqcup \mathcal{F}$ the squash G_Z is the relation with its value columns squashed and nothing else: an S-bag with one entry for each distinct combination of keys and filter attributes, on which every query that filters and groups on $\mathcal{K} \sqcup \mathcal{F}$ can be evaluated.

Moving attributes into the summary The monoid $S\langle Z \rangle$ can itself serve as a set of summaries. This separates the two things that a merge does to an attribute: it removes the attribute from the key, and it aggregates over its values.

Lemma 4.11 (Nesting)**.** *Let X and Z be disjoint sets of attributes. The map $G \mapsto (x \mapsto G(x, \cdot))$ is an isomorphism of monoids*

$$S\langle X \sqcup Z \rangle \;\cong\; \big(S\langle Z \rangle\big)\langle X \rangle,$$

under which the merge $(\mathrm{up}_X^{X \sqcup Z})_*$ becomes the change of summary along $\mathrm{tot} : S\langle Z \rangle \to S$.

Proof. For $G \in S\langle X \sqcup Z \rangle$, each $G(x, \cdot)$ has finite support and is the constant $\mathbf{0}_S$ for all but finitely many x; the inverse map sends H to $(x, z) \mapsto H(x)(z)$, and both maps preserve the pointwise operation. The total of $G(x, \cdot)$ is $\bigoplus_z G(x, z)$, the value of the merge at x. □

Read from left to right, the isomorphism moves the attributes Z from the key into the summary without squashing them: the summary of x is now a table over Z. The change of summary along tot then squashes them. Every merge is the composite of these two steps, and the levels of a reduction tree have this form (Proposition 5.26). For $S = \mathbb{N}_0$ and $X \sqcup Z = \Sigma$ the isomorphism is

$$\mathrm{Bag}(\mathrm{Row}(\Sigma)) \;\cong\; \mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}}))\langle X \rangle, \qquad B_{\mathcal{R}} \longmapsto \mathrm{Fib}_X^{X^{\mathsf{c}}},$$

the passage from the relation to the family of its fibers: the attributes X^{c} are moved into the summary, and none of them is squashed. The family of fibers is itself the squash onto X for the contribution $r \mapsto \{\{r[X^{\mathsf{c}}]\}\}$, whose summary space $\mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}}))$ is free and loses nothing. By (6), every squash onto X is obtained from this family by a homomorphism in each entry, from the free summary space into S; the bag projection is the case in which this homomorphism is the cardinality, into $\mathbb{N}_0$.

Remark 4.12 (Derived attributes). A grouping by a function of the rows is a squash as well. For a function $\kappa : \mathrm{Row}(\Sigma) \to D$, add to the schema a *derived attribute* a_κ with $\mathrm{Dom}(a_\kappa) = D$, replace every row r by $(r, \kappa(r))$, and let the extended row have the contribution $\alpha(r)$. The squash of the extended relation onto $\{a_\kappa\}$ has the entries $\widehat{\alpha}(\sigma_{\kappa=k}(B_{\mathcal{R}}))$, $k \in D$; it is the grouped aggregation by the key function κ of Section 3.3.

Remark 4.13 (What does not carry over from bags). Two properties of bags are lost in general. First, $\mathrm{Bag}(\mathrm{Row}(X))$ is free, and $S\langle X\rangle$ is not; this is why the input of a squash remains a bag, and S-bags appear as its images under the homomorphisms of Proposition 4.8. Second, the support of a bag is the set of elements that are present, and the support of an image is the image of the support. For an S-bag only $\mathrm{supp}\big((\mathrm{up}^X_{X'})_* G\big) \subseteq \mathrm{up}^X_{X'}(\mathrm{supp}(G))$ holds: in $(\mathbb{Z}, +, 0)$ the summaries 1 and -1 of two key values cancel when the merge identifies them. The paragraph on counts that follows states the condition under which the squash of a relation still tells which key values are present, and how many rows each of them stands for.

Counts A squash replaces the fiber of each key value by an element of the summary space S. The fiber tells whether its group has any row and how many, and the bag projection B_X, the squash into the summary space $\mathbb{N}_0$, retains exactly this. A squash into another summary space need not. In $(\mathbb{R}, +, 0)$ the rows 5 and -5 have the summary 0, which is also the summary of the empty fiber. In $(\mathbb{N}_0, \min)_\perp$ the summary 7 is reached by one row and by a hundred. What is missing in both cases is a homomorphism from the summary space to $\mathbb{N}_0$.

Definition 4.14 (Count, counting contribution)**.** A *count* on a commutative monoid $(S, \oplus, \mathbf{0}_S)$ is a monoid homomorphism $\#\colon S \to (\mathbb{N}_0, +, 0)$. A contribution $\alpha\colon U \to S$ is *counting* if $\#\alpha(u) = 1$ for every $u \in U$: each row counts once.

The identity is a count on $(\mathbb{N}_0, +, 0)$, and the cardinality $|B| := \sum_u B(u)$ is a count on $\mathrm{Bag}(U)$; for both, the contributions $u \mapsto 1$ and $u \mapsto \{\{u\}\}$ are counting. The summaries of `AVG` and `VAR` in Table 1 have the count as one of their components, and their contributions are counting. For a counting contribution,

$$\#\widehat{\alpha}(B) = |B| \qquad \text{for every } B \in \mathrm{Bag}(U), \tag{8}$$

because both sides are monoid homomorphisms $\mathrm{Bag}(U) \to \mathbb{N}_0$ with the value 1 on the bags $\{\{u\}\}$: the count of a summary is the number of row occurrences it summarizes. In particular, $\widehat{\alpha}(B) = \mathbf{0}_S$ if and only if B is empty, since $\#\mathbf{0}_S = 0$.

Proposition 4.15 (Counts of a squash)**.** *Let $X \subseteq \Sigma$, and let G_X be the squash of $\mathcal{R}$ onto X for a contribution $\alpha : \mathrm{Row}(\Sigma) \to S$.*

(1) *If $\#$ is a count on S and α is counting, then*

$$\# \circ G_X = B_X \qquad \textit{and} \qquad \mathrm{supp}(G_X) = \mathrm{supp}(B_X)\,:$$

the count of the summary of a fiber is the number of its row occurrences, and a key value has a nonzero summary if and only if its fiber has a row.

(2) *If S is conical and $\alpha(r) \neq \mathbf{0}_S$ for every row r, then* $\mathrm{supp}(G_X) = \mathrm{supp}(B_X)$.

Proof. (1) By Proposition 4.8(3) for the homomorphism $\#\colon S \to \mathbb{N}_0$, $\# \circ G^\alpha_X = G^{\# \circ \alpha}_X$, and $\# \circ \alpha$ is the constant 1, whose squash is B_X. If $B_X(x) > 0$, then $\#G_X(x) > 0$ and therefore $G_X(x) \neq \mathbf{0}_S$; with Lemma 4.5(1) the supports agree. (2) The summary of a nonempty fiber is a nonempty sum of nonzero elements, which is not $\mathbf{0}_S$ in a conical monoid; with Lemma 4.5(1) this gives the claim. □

By part (1), the bag projection is recovered from the squash and need not be kept beside it. The count is a change of summary from S to $\mathbb{N}_0$, and it carries the squash into S to

the squash into $\mathbb{N}_0$. The summary space S then lies between the free summary space of the fibers and $\mathbb{N}_0$: by (6) and (8), the cardinality of the fiber over x factors as

$$\mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}})) \xrightarrow{\widehat{\alpha_x}} S \xrightarrow{\#} \mathbb{N}_0.$$

The supports of the squashes of a relation then behave as those of bags, in contrast to Remark 4.13: for $X' \subseteq X$, $\mathrm{supp}(G_{X'}) = \mathrm{supp}(B_{X'})$ is the image of $\mathrm{supp}(G_X) = \mathrm{supp}(B_X)$ under the upcast, and no group with rows disappears in a merge.

Every summary space can be given a count, by adding one.

Definition 4.16 (Counting completion)**.** For a commutative monoid $(S, \oplus, \mathbf{0}_S)$, the *counting completion* is $S^{\#} := \mathbb{N}_0 \times S$ with the componentwise operation, the identity $(0, \mathbf{0}_S)$, and the count $\#(n, s) := n$. For a contribution $\alpha\colon U \to S$, put $\alpha^{\#}(u) := (1, \alpha(u))$, and let $\mathrm{pr}\colon S^{\#} \to S$, $\mathrm{pr}(n, s) := s$.

Proposition 4.17 (Counting completion)**.** *Let S be a commutative monoid and $\alpha\colon U \to S$ a contribution.*

(1) *$\alpha^{\#}$ is a counting contribution, and $\widehat{\alpha^{\#}}(B) = (|B|, \widehat{\alpha}(B))$ for every $B \in \mathrm{Bag}(U)$.*
(2) *pr is a monoid homomorphism and $\mathrm{pr} \circ \widehat{\alpha^{\#}} = \widehat{\alpha}$. Hence every aggregate $\gamma \circ \widehat{\alpha}$ equals $(\gamma \circ \mathrm{pr}) \circ \widehat{\alpha^{\#}}$: counting loses nothing, and it costs one integer for each summary.*

Proof. In both parts, the two sides are monoid homomorphisms on $\mathrm{Bag}(U)$ that agree on the bags $\{\{u\}\}$. □

For `SUM`, the completion $(\mathbb{R}, +)^{\#}$ holds a count and a sum, as the summary of `AVG` does: the rows 5 and -5 have the summary $(2, 0)$, which differs from the summary $(0, 0)$ of the empty fiber. For `MIN`, $(\mathbb{N}_0, \min)^{\#}_{\perp}$ holds the minimum together with the number of rows.

A count has two limits. First, it excludes idempotence: if $s \oplus s = s$, then $\#s = 2\#s$ and $\#s = 0$, so by (8) the summary of a nonempty bag under a counting contribution is never idempotent, and $n \cdot \alpha(u) = m \cdot \alpha(u)$ only for $n = m$. The summaries of `MIN`, `MAX`, and `COUNT(DISTINCT)` therefore admit no counting contribution as they stand, and their completions are no longer idempotent; Proposition 5.18 states what this costs in a tree. Second, a count tells how many rows a summary stands for only when the summary was computed from rows by a counting contribution. For an S-bag that is given as input, such as a table that was aggregated in advance, this needs the additional condition that $\#s = 0$ implies $s = \mathbf{0}_S$: in $\mathbb{N}_0 \times \mathbb{R}$ the entry $(0, 5)$ is nonzero and has no row behind it. We do not impose this condition, because every S-bag in the sections that follow is the squash of a relation.

4.3 Fibers

A summary in a fixed-width monoid condenses a fiber. The fiber itself is a summary as well, for the free monoid, and it is the one summary into which nothing is squashed. Between the two lie the summaries that retain the values of some attributes and squash the others.

Definition 4.18 (Fiber map)**.** Let $X, Y \subseteq \Sigma$, where X is the set of attributes that are kept as the key and Y the set of attributes whose values are kept in the summary. The *fiber map* Fib^{Y}_{X} is the squash of $\mathcal{R}$ onto X for the monoid $\big(\mathrm{Bag}(\mathrm{Row}(Y)), \uplus, \varnothing_{\mathrm{bag}}\big)$ and the

contribution $r \mapsto \{\{r[Y]\}\}$. Its value at $x \in \mathrm{Row}(X)$, the *fiber over x with values in Y*, is the bag

$$\mathrm{Fib}_X^Y(x) = (\mathrm{up}_Y^\Sigma)_* \, \sigma_{X=x}(B_\mathcal{R}) \in \mathrm{Bag}(\mathrm{Row}(Y)), \qquad \mathrm{Fib}_X^Y(x)(y) = \sum_{\substack{r[X]=x \\ r[Y]=y}} B_\mathcal{R}(r),$$

the bag of $\pi_Y(\sigma_{X=x}(\mathcal{R}))$. Fix a total order on $\mathrm{supp}(B_\mathcal{R})$, and let $(r_1, \ldots, r_n)$ enumerate the row occurrences of the group $\mathrm{Fib}_X(x)$ in this order, with each distinct row value r repeated exactly $B_\mathcal{R}(r)$ times; then $n = B_X(x)$. The *fiber tuple* is

$$\overrightarrow{\mathrm{Fib}}_X^Y(x) := (r_1[Y], \ldots, r_n[Y]) \in \mathrm{Row}(Y)^{B_X(x)}.$$

Thus $\overrightarrow{\mathrm{Fib}}_X^Y$ is a function on $\mathrm{Row}(X)$ whose value at x is a tuple of $B_X(x)$ rows over Y,

$$\overrightarrow{\mathrm{Fib}}_X^Y : \ \mathrm{Row}(X) \ni x \ \longmapsto \ \overrightarrow{\mathrm{Fib}}_X^Y(x) \in \mathrm{Row}(Y)^{B_X(x)},$$

and the bag of entries of $\overrightarrow{\mathrm{Fib}}_X^Y(x)$ is $\mathrm{Fib}_X^Y(x)$.

The fiber map splits the schema into three parts. The attributes of X are kept as the key. The attributes of Y are kept in the summary, whose summary space is $\mathrm{Bag}(\mathrm{Row}(Y))$. The attributes outside $X \cup Y$ are squashed into this space: of their values, the fiber retains only the multiplicities. For $Y = X^\mathsf{c}$ the fiber map is the family of fibers of Definition 4.3, into which nothing is squashed, and the same holds whenever $X \cup Y = \Sigma$. For $Y = \emptyset$ the summary space is $\mathrm{Bag}(\mathrm{Row}(\emptyset)) = \mathbb{N}_0$ and the fiber map is the bag projection B_X, into which everything outside X is squashed.

The cardinality is a count on $\mathrm{Bag}(\mathrm{Row}(Y))$ and the contribution $r \mapsto \{\{r[Y]\}\}$ is counting, so by Proposition 4.15(1) the fiber over x has $B_X(x)$ elements, counted with multiplicity; this is the length of the fiber tuple. The group is the support of the fiber with all attributes kept, $\mathrm{Fib}_X(x) = \mathrm{supp}(\mathrm{Fib}_X^\Sigma(x))$.

For a value attribute $w \in \mathcal{V}$ with values in $\mathbb{R}$, the fiber tuple is the *fiber value vector*

$$F_{X,w} : \mathrm{Row}(X) \to \mathbb{R}^*, \qquad F_{X,w}(x) = (r_1[w], \ldots, r_n[w]),$$

where $\mathbb{R}^*$ denotes the set of finite real-valued vectors, including the empty vector. For an empty fiber, $F_{X,w}(x)$ is the empty vector. More generally, for an ordered list $W = (w_1, \ldots, w_k) \in \mathcal{V}^k$ of value attributes, the fiber tuple $\overrightarrow{\mathrm{Fib}}_X^W(x)$, written as a matrix with one row for each row occurrence, is the *fiber value matrix*

$$F_{X,W}(x) = \begin{pmatrix} r_1[w_1] & \cdots & r_1[w_k] \\ \vdots & & \vdots \\ r_n[w_1] & \cdots & r_n[w_k] \end{pmatrix} \in \mathbb{R}^{B_X(x) \times k}.$$

For an empty fiber, $F_{X,W}(x)$ is the unique $0 \times k$ matrix. The grouping attributes X therefore identify each group, while the selected value attributes W supply its aggregation inputs; auxiliary filter attributes in $\mathcal{F}$ remain available in the full rows for selection predicates.

Theorem 4.19 (Fibers preserve and restore)**.** *Let $X, X', Y \subseteq \Sigma$.*

(1) Preserve. *The family of fibers labelled by their key values is the projection of $\mathcal{R}$ onto $X \cup Y$:*

$$B_{X\cup Y}(t) = \mathrm{Fib}_X^Y(t[X])(t[Y]) \qquad \text{for all } t \in \mathrm{Row}(X \cup Y).$$

In particular, if $X \cup Y = \Sigma$, the labelled family $\left(x, \mathrm{Fib}_X^Y(x)\right)_{x\in\mathrm{supp}(B_X)}$ is the relation $\mathcal{R}$ itself.

(2) Restore. *If $X <: X'$, the fibers over the superfilter X' are restored from the fibers over X by stacking: for every $x' \in \mathrm{Row}(X')$,*

$$\mathrm{Fib}_{X'}^Y(x') = \biguplus_{\substack{x\in\mathrm{supp}(B_X)\\ x<:x'}} \mathrm{Fib}_X^Y(x).$$

For tuples, $\overrightarrow{\mathrm{Fib}}_{X'}^Y(x')$ is a rearrangement of the concatenation of the tuples $\overrightarrow{\mathrm{Fib}}_X^Y(x)$ with $x <: x'$.

(3) Only in this direction. *Suppose that every attribute has at least two values. The labelled family of fibers over X determines the labelled family of fibers over X', for every relation over Σ, if and only if $X' \subseteq X \cup Y$: a regrouping is possible exactly by the attributes that have not been squashed. In particular, when X and X' are sets of key attributes and Y is a set of value attributes, this holds if and only if $X <: X'$.*

(4) Universal. *Let $\alpha : \mathrm{Row}(\Sigma) \to S$ be a contribution that reads Y, that is, $\alpha(r) = \beta(r[Y])$ for a map $\beta : \mathrm{Row}(Y) \to S$. Then*

$$G_X^\alpha = \widehat{\beta} \circ \mathrm{Fib}_X^Y, \qquad \text{that is,} \qquad G_X^\alpha(x) = \widehat{\beta}\big(\mathrm{Fib}_X^Y(x)\big) \quad \text{for every } x \in \mathrm{Row}(X) :$$

every squash onto X that reads only the attributes kept in the fiber is a change of summary of the fiber map.

Proof. (1) A row r satisfies $r[X \cup Y] = t$ if and only if $r[X] = t[X]$ and $r[Y] = t[Y]$, so $B_{X\cup Y}(t) = \sum_{r[X\cup Y]=t} B_{\mathcal{R}}(r)$ is the stated multiplicity.

(2) This is Proposition 4.8(1) for the fiber map, whose support is $\mathrm{supp}(B_X)$ by Proposition 4.15(1). Two tuples with the same bag of entries are rearrangements of each other.

(3) If $X' \subseteq X \cup Y$, the family over X determines $B_{X\cup Y}$ by (1), and the fibers over X' with values in Y are computed from $B_{X\cup Y}$ because $X' \cup Y \subseteq X \cup Y$. Conversely, let $a \in X' \setminus (X \cup Y)$. Choose rows r_0, r_1 that differ exactly in the attribute a, and consider the relations with bags $\{\{r_0, r_0\}\}$ and $\{\{r_0, r_1\}\}$. Since r_0 and r_1 agree on $X \cup Y$, both relations have the same family over X: one key, with fiber $\{\{r_0[Y], r_0[Y]\}\}$. Over X' the first relation has one key with two rows and the second has two keys with one row each. For the last claim, key and value attributes are disjoint, so $X' \subseteq X \cup Y$ means $X' \subseteq X$, that is, $X <: X'$.

(4) The extension $\widehat{\beta} : \mathrm{Bag}(\mathrm{Row}(Y)) \to S$ is a monoid homomorphism with $\widehat{\beta}(\{\{r[Y]\}\}) = \beta(r[Y]) = \alpha(r)$; apply Proposition 4.8(3) to the fiber map and $h = \widehat{\beta}$. □

In part (2) the tuples are equal, and not only rearrangements, when the fixed order of row occurrences sorts by $r[X]$ first and the blocks are concatenated in that order. The first identity of (5) is part (2) for $Y = \Sigma$, stated for supports.

Part (2) also describes the group to which a key contributes. For $X <: X'$ and $x \in \mathrm{supp}(B_X)$, the fiber over the upcast key,

$$\overrightarrow{\mathrm{Fib}}_{X'}^Y(\mathrm{up}_{X'}^X(x)) \in \mathrm{Row}(Y)^{B_{X'}(x[X'])},$$

contains the fiber $\overrightarrow{\mathrm{Fib}}^{Y}_{X}(x)$ of x itself and is the stack of the fibers of x and of its siblings, the keys $\tilde{x}$ with the same upcast. This expression is defined only when $X <: X'$, since otherwise x has no upcast to a key over X'.

Parts (1) and (4) separate the fiber from a fixed-width summary. A fixed-width summary preserves a statistic of the group; the fiber preserves its rows, and with them every statistic. For example, the bags $\{\{1,3\}\}$ and $\{\{2,2\}\}$ have the same $(\mathrm{count}, \mathrm{sum})$ summary $(2,4)$, so no map recovers the rows from that summary. Part (3) is the counterpart of Proposition 4.9 for a summary that keeps the values of Y: the attributes of Y can still be returned to the key, and the squashed ones cannot.

Three squashes of the same attributes Let α read X^{c}, with $\alpha(x,t) = \beta(t)$, and let $\#$ be a count on S and α counting. The fibers, the summaries, and the counts over X are then changes of summary of one another (part (4), Proposition 4.15(1), and Lemma 4.11):

$$\mathrm{Bag}(\mathrm{Row}(\Sigma)) \xrightarrow{\ \cong\ } \mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}}))\langle X\rangle \xrightarrow{\ \widehat{\beta}\circ\ } S\langle X\rangle \xrightarrow{\ \#\circ\ } \mathrm{Bag}(\mathrm{Row}(X)),$$

which sends $B_{\mathcal{R}} \mapsto \mathrm{Fib}^{X^{\mathsf{c}}}_{X} \mapsto G_X \mapsto B_X$. All three are squashes onto X: they keep X and reduce X^{c} to the summary spaces $\mathrm{Bag}(\mathrm{Row}(X^{\mathsf{c}}))$, S, and $\mathbb{N}_0$. They differ only in what these spaces retain: the fiber retains the residuals themselves, the summary what $\widehat{\beta}$ retains of them, and the bag projection the number of row occurrences. The first arrow only regroups the rows; each of the other two discards information that no later step recovers.

Remark 4.20 (Direction of restoration)*.* Restoration follows upcasts. Passing from X to a superfilter X' removes the attributes $X \setminus X'$ from the key: it stacks fibers and needs no information beyond the labels. Passing to a subfilter splits a fiber and needs, for each row, the values of the added attributes, which are available only if those attributes are kept in Y; this is the content of part (3).

In a reduction tree (Section 5) the key attributes along a path form a chain

$$X_d <: X_{d-1} <: \cdots <: X_1 <: X_0 = \emptyset,$$

each level a subfilter of the one above it, and the key values satisfy $u_j <: u_{j-1}$. The fibers over X_j therefore restore the fibers over every X_i with $i \le j$, and in an exhaustively branching tree the leaves, which are subfilters of all their ancestors, restore every level. Two nodes whose key sets are incomparable under $<:$, such as the depth-j nodes of two different level orderings (Remark 6.5), do not restore one another; both are restored from any common subfilter, in particular from the leaves.

5 Reduction trees

A reduction tree filters a relation on the way down and merges summaries on the way up. This section defines the two passes, states exactly when they agree, which answers (P1), and identifies the tree that occurs most often, one `GROUP BY` column per level, with `ROLLUP`.

5.1 Reduction tree and downward operator

Definition 5.1 (Reduction tree)**.** A *reduction tree* over a relation $\mathcal{R}$ is a finite, nonempty, prefix-free collection E of root-to-leaf *paths*. Each *path* is a finite sequence

$$p = (e_1, \ldots, e_d) \in E, \qquad e_i = (\varphi_i, U_i, \ell_i),$$

where each edge record consists of

- a *downward predicate* φ_i;
- an *upward reducer* U_i, mapping the child's result table to a fixed-shape contribution; and
- a *label* ℓ_i, unique among sibling edges, identifying both the edge and its contribution under the parent vertex.

The nodes are the prefixes of paths in E:

$$V := \{q \mid q \text{ is a prefix of some } p \in E\}.$$

Identical prefixes denote the same node, and the empty prefix ε is the root.

Definition 5.2 (Downward operator)**.** For a path $p = (e_1, \ldots, e_d)$, write

$$v_0 = \varepsilon, \qquad v_j = (e_1, \ldots, e_j) \quad (1 \le j \le d),$$

so that the path traverses $v_0 \to v_1 \to \cdots \to v_d$. The *downward operator* at v_j is the cumulative filter

$$D(v_j) := \sigma_{\varphi_j} \circ \cdots \circ \sigma_{\varphi_1} = \sigma_{\bigwedge_{i=1}^{j} \varphi_i}, \qquad D(v_0) := \sigma_{\text{true}}.$$

Thus, the relation (sub-dataframe) reaching v_j is $D(v_j)(\mathcal{R})$.

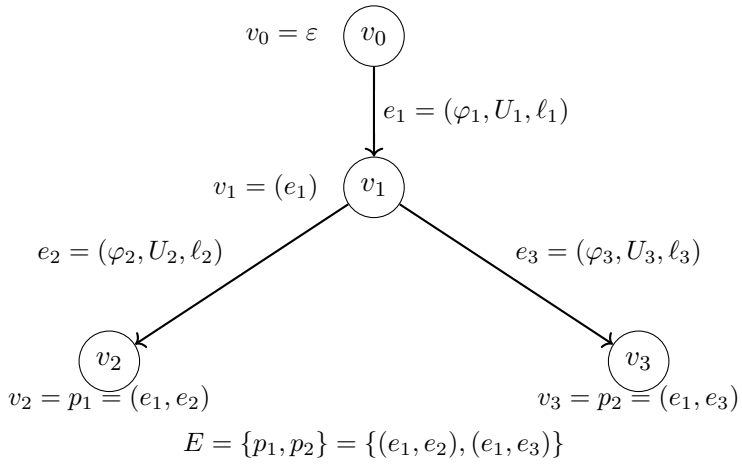


Figure 2: A reduction tree represented by root-to-leaf paths. The paths share the prefix (e_1). Each edge carries a downward predicate, an upward reducer, and a label that uniquely distinguishes it among siblings. Each internal vertex serves as the child key for its outgoing labels. Arrows indicate the downward direction, while reducers propagate contributions upward.

5.2 Upward operator

Throughout this subsection, fix the input relation $\mathcal{R}$ with bag $B_{\mathcal{R}}$, an ordered list $W = (w_1, \ldots, w_k) \in \mathcal{V}^k$ of value attributes, and a number $z \in \mathbb{N}_0$ of statistics. For a row r, write $r[W] := (r[w_1], \ldots, r[w_k]) \in \mathbb{R}^{1\times k}$.

Key chains For a node $v = (e_1, \ldots, e_j) \in V$ with $e_i = (\varphi_i, U_i, \ell_i)$, define its cumulative predicate and its *key attributes* by

$$\Phi_v := \bigwedge_{i=1}^{j} \varphi_i, \qquad X_v := \mathrm{attr}(\Phi_v) = \bigcup_{i=1}^{j} \mathrm{attr}(\varphi_i),$$

where $\mathrm{attr}(\varphi) \subseteq \Sigma$ is the set of attributes occurring in φ. Thus $D(v) = \sigma_{\Phi_v}$, $\Phi_\varepsilon = \mathrm{true}$, and $X_\varepsilon = \emptyset$.

Assumption 5.3 (Key-determined selection). For every node $v \in V$, we have $X_v \subseteq \mathcal{K}$, and there is a (necessarily unique) key value $u_v \in \mathrm{Row}(X_v)$ such that

$$\Phi_v(r) \iff r[X_v] = u_v \qquad \text{for all } r \in \mathrm{Row}(\Sigma).$$

This holds, for instance, when every edge predicate is a conjunction of equality atoms $a = c$ with $a \in \mathcal{K}$ and every Φ_v is satisfiable. Under Assumption 5.3, the relation $D(v)(\mathcal{R})$ reaching v has bag $r \mapsto B_{\mathcal{R}}(r) \cdot [\, r[X_v] = u_v \,]$ (with $[\,\cdot\,]$ the Iverson bracket), support $\mathrm{Fib}_{X_v}(u_v)$, exactly $B_{X_v}(u_v)$ row occurrences, and value matrix $F_{X_v,W}(u_v)$. For a summary monoid S and a contribution α, its summary is the entry $G_{X_v}(u_v)$ of the squash of $\mathcal{R}$ onto X_v (Definition 4.4): a node keeps its key attributes X_v, and what it holds for its key value u_v is the fiber over X_v^{c} or a summary into which these attributes have been squashed.

For a path $p = (e_1, \ldots, e_d) \in E$ with nodes $v_0, \ldots, v_d$, abbreviate $X_j := X_{v_j}$ and $u_j := u_{v_j}$. The key attributes are cumulative and the key values are compatible:

$$\emptyset = X_0 \subseteq X_1 \subseteq \cdots \subseteq X_d \subseteq \mathcal{K}, \qquad u_j[X_{j-1}] = u_{j-1} \quad (1 \le j \le d),$$

the latter because Φ_{v_j} implies $\Phi_{v_{j-1}}$. In the subfilter order of Definition 4.1 this is the chain

$$X_d <: X_{d-1} <: \cdots <: X_1 <: X_0 = \emptyset, \qquad u_j <: u_{j-1} \quad (1 \le j \le d) :$$

each node is a subfilter of its parent. A step down the path adds the attributes $X_j \setminus X_{j-1}$ to the kept ones, and a step up squashes them again. By Lemma 4.2, the group of u_{j-1} is the disjoint union of the groups of the keys $x \in \mathrm{supp}(B_{X_j})$ with $x <: u_{j-1}$, and by Proposition 4.8(1) its summary is the merge of their summaries.

Matrix operations Let $\mathbb{R}_\perp := \mathbb{R} \cup \{\perp\}$, where $\perp$ denotes an undefined value (NULL), and let

$$\mathcal{M}_k := \bigcup_{n\in\mathbb{N}_0} \mathbb{R}^{n\times k}$$

be the set of *raw value matrices* with k columns. For $M \in \mathbb{R}^{n\times k}$, write $M_{\bullet h} \in \mathbb{R}^n$ for its h-th column.

- *Stacking* (UNION ALL). For $M \in \mathbb{R}^{n\times k}$ and $M' \in \mathbb{R}^{n'\times k}$,

$$M \uplus M' := \begin{pmatrix} M \\ M' \end{pmatrix} \in \mathbb{R}^{(n+n')\times k}.$$

Stacking is associative, with the $0 \times k$ matrix as identity; $\biguplus_{i=1}^{m} M_i$ denotes the iterated stack.

- *Row equivalence.* $M \simeq M'$ iff $M' = PM$ for some permutation matrix P, i.e. M and M' contain the same bag of rows.
- *Statistics.* Fix functions $s_1, \dots, s_z : \mathbb{R}^* \to \mathbb{R}_\perp$, each *symmetric*: for every permutation π of $\{1, \dots, n\}$,

$$s_t(a_{\pi(1)}, \dots, a_{\pi(n)}) = s_t(a_1, \dots, a_n).$$

The *statistics block* of $M \in \mathbb{R}^{n \times k}$ is

$$\mathrm{Stat}(M) := \big(s_t(M_{\bullet h})\big)_{1 \le t \le z,\, 1 \le h \le k} \in \mathbb{R}_\perp^{z \times k}.$$

By symmetry, $M \simeq M'$ implies $\mathrm{Stat}(M) = \mathrm{Stat}(M')$.

Definition 5.4 (Within-node and level aggregation)**.** The *within-node aggregation* appends the statistics block to the raw values:

$$\mathrm{aug} : \mathcal{M}_k \to \widehat{\mathcal{M}}_k, \qquad \mathrm{aug}(M) := \begin{pmatrix} M \\ \mathrm{Stat}(M) \end{pmatrix} \in \mathbb{R}_\perp^{(n+z) \times k} \quad \text{for } M \in \mathbb{R}^{n \times k},$$

where $\widehat{\mathcal{M}}_k := \mathrm{aug}(\mathcal{M}_k)$ is the set of *augmented matrices*. Equivalently, aug maps each column $a = M_{\bullet h} \in \mathbb{R}^n$ to $\big(a_1, \dots, a_n, s_1(a), \dots, s_z(a)\big)^\top \in \mathbb{R}_\perp^{n+z}$. The *raw part* $\rho : \widehat{\mathcal{M}}_k \to \mathcal{M}_k$ removes the last z rows, so $\rho \circ \mathrm{aug} = \mathrm{id}$. We write aug and not α, which denotes the contribution of a row throughout.

The *level aggregation* of augmented matrices $\widehat{M}_1, \dots, \widehat{M}_m \in \widehat{\mathcal{M}}_k$ stacks their raw parts and aggregates the result:

$$\lambda(\widehat{M}_1, \dots, \widehat{M}_m) := \mathrm{aug}\Big(\biguplus_{i=1}^{m} \rho(\widehat{M}_i) \Big).$$

If $\widehat{M}_i \in \mathbb{R}_\perp^{(n_i+z) \times k}$ for each i, then

$$\lambda(\widehat{M}_1, \dots, \widehat{M}_m) \in \mathbb{R}_\perp^{(n_1 + \cdots + n_m + z) \times k};$$

the children's statistics blocks are discarded and recomputed, not stacked.

Definition 5.5 (Upward operator)**.** For a node $v \in V$, let $c_1, \dots, c_m$ be its children, listed in a fixed order of their labels. The *upward operator* assigns to v the augmented matrix

$$U(v) := \begin{cases} \mathrm{aug}\big(F_{X_v, W}(u_v)\big), & v \in E \quad \text{(leaf)}, \\ \lambda\big(U(c_1), \dots, U(c_m)\big), & v \in V \setminus E \quad \text{(internal node)}. \end{cases}$$

We write n_v for the number of rows of $\rho(U(v))$, so that $U(v) \in \mathbb{R}_\perp^{(n_v+z) \times k}$.

In the edge record $e_i = (\varphi_i, U_i, \ell_i)$, the reducer is $U_i = \mathrm{Stat}$. It maps the child's result table $\rho(U(v_i))$ to the fixed-shape contribution

$$U_i\big(\rho(U(v_i))\big) \in \mathbb{R}_\perp^{z \times k},$$

which the parent identifies by the label ℓ_i.

Definition 5.6 (Exhaustive branching)**.** An internal node v *branches exhaustively* if the fibers of its children $c_1, \dots, c_m$ partition its own fiber:

$$\mathrm{Fib}_{X_v}(u_v) = \bigsqcup_{i=1}^{m} \mathrm{Fib}_{X_{c_i}}(u_{c_i}),$$

i.e. every $r \in \mathrm{supp}(B_{\mathcal{R}})$ with $r[X_v] = u_v$ satisfies $r[X_{c_i}] = u_{c_i}$ for exactly one i.

If all children of v share the same key attributes X', then by Lemma 4.2 v branches exhaustively whenever the key values $u_{c_1}, \dots, u_{c_m}$ are pairwise distinct and include every $x \in \mathrm{supp}(B_{X'})$ with $x <: u_v$.

Proposition 5.7 (Shape and correctness)**.** *Let $v \in V$.*

1. *If v is a leaf, then $n_v = B_{X_v}(u_v)$; if v is internal with children $c_1, \dots, c_m$, then $n_v = \sum_{i=1}^{m} n_{c_i}$.*
2. *If every internal node in the subtree rooted at v branches exhaustively, then*

$$\rho\big(U(v)\big) \simeq F_{X_v,W}(u_v), \qquad U(v) \in \mathbb{R}_{\perp}^{(B_{X_v}(u_v)+z)\times k},$$

and the statistics block of $U(v)$ equals $\mathrm{Stat}\big(F_{X_v,W}(u_v)\big)$.

In particular, along a path $v_0 \to \cdots \to v_d$ in an exhaustively branching tree, the leaf carries $U(v_d) \in \mathbb{R}_{\perp}^{(B_{X_d}(u_d)+z)\times k}$, each upward step from level j to level $j-1$ yields $U(v_{j-1}) \in \mathbb{R}_{\perp}^{(B_{X_{j-1}}(u_{j-1})+z)\times k}$, and at the root $n_{v_0} = B_{\emptyset}(u_0)$ is the total number of row occurrences of $\mathcal{R}$.

Proof. Part 1 is immediate from Definitions 5.4 and 5.5. For part 2, argue by induction on the height of v; the leaf case holds by definition. Let v be internal. By the induction hypothesis, $\rho(U(c_i)) \simeq F_{X_{c_i},W}(u_{c_i})$, whose rows are the vectors $r[W]$ for $r \in \mathrm{Fib}_{X_{c_i}}(u_{c_i})$, each repeated $B_{\mathcal{R}}(r)$ times. Since the children's fibers partition $\mathrm{Fib}_{X_v}(u_v)$, the stack $\biguplus_i \rho(U(c_i))$ contains $r[W]$ exactly $B_{\mathcal{R}}(r)$ times for each $r \in \mathrm{Fib}_{X_v}(u_v)$ and no other rows; hence it is row equivalent to $F_{X_v,W}(u_v)$ and has $B_{X_v}(u_v)$ rows. The claim about the statistics block follows from the symmetry of $s_1, \dots, s_z$. □

Remark 5.8. Without exhaustive branching, $U(v)$ is still well defined, but $\rho(U(v))$ describes only the bag sum of the children's selections, which may omit or repeat rows of $\mathrm{Fib}_{X_v}(u_v)$. Without Assumption 5.3 (e.g. for range predicates or predicates over auxiliary attributes in $\mathcal{F}$), replace $B_{\mathcal{R}}$ by the node bag $B_v(r) := B_{\mathcal{R}}(r) \cdot [\,\Phi_v(r)\,]$ of $D(v)(\mathcal{R})$ and $F_{X_v,W}(u_v)$ by the value matrix of all row occurrences of B_v; Proposition 5.7 then holds with $n_v = |D(v)(\mathcal{R})|$, provided the children's predicates partition the rows of $D(v)(\mathcal{R})$.

5.3 From downward selection to upward aggregation

Proposition 5.7 compares the two passes on raw rows. This subsection compares them on summaries and thereby answers (P1). In the terms of Section 4.2, the two passes move attributes in opposite directions. The downward operator adds attributes to the kept ones: a node fixes the values of its key attributes, and the summary of the rows that reach it squashes all other attributes. The upward operator squashes attributes that were kept: it merges the summaries of the children into a summary for the parent, which no longer distinguishes the children. For the squash of a relation the merge is prescribed by the data, with one term for each key value below the parent (Proposition 4.8(1)). In a tree

it is prescribed by the edges. This subsection states exactly when the two agree, that is, when merging the children's summaries reproduces the summary of the rows that the downward operator delivers to the parent, and when the finalizer may be deferred to the node at which a value is reported.

Throughout, fix a commutative monoid $(S, \oplus, \mathbf{0}_S)$ and a row contribution $\alpha\colon \mathrm{Row}(\Sigma) \to S$ with extension $\widehat{\alpha}\colon \mathrm{Bag}(\mathrm{Row}(\Sigma)) \to S$ (Section 2.2), so that $\widehat{\alpha}(B) = \bigoplus_{r \in \mathrm{supp}(B)} B(r) \cdot \alpha(r)$ with the multiples $n \cdot s$ of (1).

Selection on bags. A predicate φ over Σ acts on bags by $\sigma_\varphi(B)(r) := B(r) \cdot [\varphi(r)]$ (Section 2.2). Parts (1) and (2) of the next lemma repeat Lemma 2.3 for bags of rows; part (3) describes a sum of selections, which is what a layer of a tree forms.

Lemma 5.9 (Selection calculus)**.** *Let $\varphi, \psi, \varphi_1, \ldots, \varphi_m$ be predicates over Σ and $B, B' \in \mathrm{Bag}(\mathrm{Row}(\Sigma))$.*

(1) *$\sigma_\varphi(B \uplus B') = \sigma_\varphi(B) \uplus \sigma_\varphi(B')$ and $\sigma_\varphi(\varnothing_{\mathrm{bag}}) = \varnothing_{\mathrm{bag}}$; that is, σ_φ is a monoid endomorphism of $\mathrm{Bag}(\mathrm{Row}(\Sigma))$.*
(2) *$\sigma_\varphi \circ \sigma_\psi = \sigma_{\varphi \wedge \psi}$; in particular, selections are idempotent and commute* (cf. Lemma 6.1).
(3) *$(\biguplus_{i=1}^m \sigma_{\varphi_i}(B))(r) = B(r) \cdot \#\{\, i \mid \varphi_i(r) \,\}$ for every row r. Consequently, $\biguplus_i \sigma_{\varphi_i}(B) = \sigma_{\varphi_1 \vee \cdots \vee \varphi_m}(B)$ if and only if no row of $\mathrm{supp}(B)$ satisfies two of the φ_i.*

Proof. All three claims are pointwise identities between multiplicities:

$$(B(r) + B'(r))\,[\varphi(r)] = B(r)\,[\varphi(r)] + B'(r)\,[\varphi(r)], \qquad [\varphi(r)]\,[\psi(r)] = [\varphi(r) \wedge \psi(r)],$$
$$\textstyle\sum_i [\varphi_i(r)] = \#\{i \mid \varphi_i(r)\},$$

and the last number equals $[\varphi_1(r) \vee \cdots \vee \varphi_m(r)]$ exactly when it is at most 1. □

Lemma 5.10 (Selection–aggregation exchange)**.** *For a predicate φ over Σ, the* masked contribution $\alpha_\varphi\colon \mathrm{Row}(\Sigma) \to S$ *is*

$$\alpha_\varphi(r) := [\varphi(r)] \cdot \alpha(r) = \begin{cases} \alpha(r), & \varphi(r), \\ \mathbf{0}_S, & \text{otherwise.} \end{cases}$$

It satisfies $\widehat{\alpha} \circ \sigma_\varphi = \widehat{\alpha_\varphi}$.

Proof. By Lemma 5.9(1), $\widehat{\alpha} \circ \sigma_\varphi$ is a monoid homomorphism $\mathrm{Bag}(\mathrm{Row}(\Sigma)) \to S$. On a singleton it takes the value $\widehat{\alpha}(\{\{r\}\}) = \alpha(r)$ if $\varphi(r)$ and $\widehat{\alpha}(\varnothing_{\mathrm{bag}}) = \mathbf{0}_S$ otherwise, which is $\alpha_\varphi(r)$. Since $\mathrm{Bag}(\mathrm{Row}(\Sigma))$ is free on $\mathrm{Row}(\Sigma)$, a homomorphism is determined by its values on singletons. □

Lemma 5.10 says that aggregating a selection is again a mergeable aggregate of the *unfiltered* relation, with the filter moved into the contribution. This is the conditional aggregate `AGG(x) FILTER (WHERE` φ`)` of SQL, and it is what allows every node of a tree to be evaluated in one pass over $\mathcal{R}$. It is also the way in which a predicate on an attribute that is squashed takes effect. Such a predicate cannot be applied to the summaries (Proposition 4.9(2)), but it can be squashed together with its attribute: for every $X \subseteq \Sigma$, the squash of $\sigma_\varphi(\mathcal{R})$ onto X is the squash $G_X^{\alpha_\varphi}$ of $\mathcal{R}$ for the masked contribution, because $\sigma_{X=x} \circ \sigma_\varphi = \sigma_\varphi \circ \sigma_{X=x}$.

Two summaries per node. Recall from Definition 5.2 and Remark 5.8 that the relation $D(v)(\mathcal{R})$ reaching a node v has the bag $B_v := \sigma_{\Phi_v}(B_{\mathcal{R}})$, i.e. $B_v(r) = B_{\mathcal{R}}(r) \cdot [\Phi_v(r)]$.

Definition 5.11 (Direct summary, upward summary, leaf multiplicity)**.** Let $v \in V$ and let $L(v) := \{\, p \in E \mid v \text{ is a prefix of } p \,\}$ be the set of leaves below v.

- The *direct summary* of v is $s(v) := \widehat{\alpha}(B_v)$, the summary of the rows selected by the downward operator.
- The *upward summary* of v is defined by recursion on the height of v: with $c_1, \ldots, c_m$ the children of an internal node v,

$$\overline{U}(v) := \begin{cases} s(v), & v \in E \quad \text{(leaf)}, \\ \overline{U}(c_1) \oplus \cdots \oplus \overline{U}(c_m), & v \in V \setminus E. \end{cases}$$

- The *leaf multiplicity* of v is the function $\mu_v \colon \mathrm{Row}(\Sigma) \to \mathbb{N}_0$,

$$\mu_v(r) := \#\{\, p \in L(v) \mid \Phi_p(r) \,\},$$

  the number of leaves below v whose cumulative predicate selects r.

The leaf multiplicity depends on the predicates of the tree only, not on $\mathcal{R}$. Since E is prefix-free, the leaves below an internal node are partitioned by its children, so

$$\mu_p = [\Phi_p] \quad (p \in E), \qquad \mu_v = \mu_{c_1} + \cdots + \mu_{c_m} \quad (v \in V \setminus E), \tag{9}$$

and, because Φ_p implies Φ_v for $p \in L(v)$, $\mu_v(r) = 0$ whenever $\Phi_v(r)$ fails.

Under Assumption 5.3, $B_v = \sigma_{X_v = u_v}(B_{\mathcal{R}})$, and the direct summary is the entry at the key value of the node of the squash of $\mathcal{R}$ onto its key attributes (Definition 4.4):

$$s(v) = G_{X_v}(u_v). \tag{10}$$

The node keeps X_v, and its direct summary is the element of the summary space S into which X_v^{c} is squashed. The direct summaries of all nodes with the same key attributes X are entries of the one S-bag G_X. Without the assumption, the predicate of the node is squashed with the attributes that it reads: $s(v) = \widehat{\alpha_{\Phi_v}}(B_{\mathcal{R}})$ is the single entry of the squash onto $\emptyset$ for the masked contribution (Lemma 5.10).

Theorem 5.12 (Downward–upward exchange)**.** *Let E be a reduction tree over $\mathcal{R}$ and $v \in V$.*

(1) Downward. *The direct summary is a homomorphic image of the root bag:*

$$s(v) = \widehat{\alpha_{\Phi_v}}(B_{\mathcal{R}}) = \bigoplus_{r \in \mathrm{supp}(B_{\mathcal{R}})} \big(B_{\mathcal{R}}(r)\,[\Phi_v(r)]\big) \cdot \alpha(r).$$

(2) Upward. *The upward summary is the merge of the direct summaries of the leaves below v, and again a homomorphic image of the root bag:*

$$\overline{U}(v) = \bigoplus_{p \in L(v)} s(p) = \widehat{\mu_v \alpha}(B_{\mathcal{R}}) = \bigoplus_{r \in \mathrm{supp}(B_{\mathcal{R}})} \big(B_{\mathcal{R}}(r)\,\mu_v(r)\big) \cdot \alpha(r),$$

where $(\mu_v \alpha)(r) := \mu_v(r) \cdot \alpha(r)$.

(3) Exchange. *If*

$$\mu_v(r) \cdot \alpha(r) = \alpha(r) \qquad \text{for every } r \in \operatorname{supp}(B_{\mathcal{R}}) \text{ with } \Phi_v(r), \tag{11}$$

then $\overline{U}(v) = s(v)$. *Moreover,* $\overline{U}(v) = s(v)$ *holds for* every *relation* $\mathcal{R}$ *over* Σ *if and only if* $\mu_v(r) \cdot \alpha(r) = \alpha(r)$ *for every* $r \in \operatorname{Row}(\Sigma)$ *with* $\Phi_v(r)$.

Proof. (1) is Lemma 5.10 with $\varphi = \Phi_v$.

(2) The first equality follows by induction on the height of v from Definition 5.11, since the leaves below an internal node are partitioned by its children. For the second, apply (1) to each leaf and regroup by rows using (1):

$$\bigoplus_{p \in L(v)} \bigoplus_{r \in \operatorname{supp}(B_{\mathcal{R}})} (B_{\mathcal{R}}(r)[\Phi_p(r)]) \cdot \alpha(r) = \bigoplus_{r \in \operatorname{supp}(B_{\mathcal{R}})} \Big(B_{\mathcal{R}}(r) \sum_{p \in L(v)} [\Phi_p(r)] \Big) \cdot \alpha(r),$$

and $\big(B_{\mathcal{R}}(r)\mu_v(r)\big) \cdot \alpha(r) = B_{\mathcal{R}}(r) \cdot \big(\mu_v(r) \cdot \alpha(r)\big)$.

(3) By (1) and (2), $s(v)$ and $\overline{U}(v)$ are the values at $B_{\mathcal{R}}$ of the extensions of α_{Φ_v} and $\mu_v \alpha$. These contributions agree at every row at which Φ_v fails, where both are $\mathbf{0}_S$, and by (11) at every row of $\operatorname{supp}(B_{\mathcal{R}})$ at which Φ_v holds; hence the two sums agree term by term. For the converse, let $r \in \operatorname{Row}(\Sigma)$ with $\Phi_v(r)$ and take the relation with bag $\{\{r\}\}$: then $s(v) = \alpha(r)$ and $\overline{U}(v) = \mu_v(r) \cdot \alpha(r)$. □

Figure 3 displays the theorem as a diagram. Both paths are monoid homomorphisms out of the free commutative monoid $\operatorname{Bag}(\operatorname{Row}(\Sigma))$, so comparing them reduces to comparing their values on single rows; this is the content of part (3).

$$\begin{array}{ccc} \operatorname{Bag}(\operatorname{Row}(\Sigma)) & \xrightarrow{(\sigma_{\Phi_p})_{p \in L(v)}} & \operatorname{Bag}(\operatorname{Row}(\Sigma))^{L(v)} \\ \Big\downarrow{\scriptstyle \sigma_{\Phi_v}} & & \Big\downarrow{\scriptstyle \nabla \circ \widehat{\alpha}^{\,L(v)}} \\ \operatorname{Bag}(\operatorname{Row}(\Sigma)) & \xrightarrow[\widehat{\alpha}]{} & S \end{array}$$

Figure 3: Downward–upward exchange at a node v. Down, then right: select the rows of v and aggregate them (the direct summary $s(v)$). Right, then down: select the rows of every leaf below v, aggregate each leaf, and merge with $\nabla(s_p)_p := \bigoplus_p s_p$ (the upward summary $\overline{U}(v)$). By Theorem 5.12, the square commutes on $B_{\mathcal{R}}$ whenever (11) holds, and for every monoid and contribution exactly when each row reaching v reaches exactly one leaf below v.

Corollary 5.13 (Hierarchical consistency)**.** *For a node* v *of a reduction tree over* $\mathcal{R}$*, the following are equivalent:*

(a) $\mu_v(r) = [\Phi_v(r)]$ *for every* $r \in \operatorname{supp}(B_{\mathcal{R}})$*; that is, every row reaching* v *reaches exactly one leaf below* v*;*
(b) $\overline{U}(v) = s(v)$ *for every commutative monoid* S *and every contribution* $\alpha \colon \operatorname{Row}(\Sigma) \to S$.

Moreover, (a) *holds at every node of the subtree rooted at* v *if and only if every internal node* w *of that subtree, with children* $c_1, \dots, c_m$*, satisfies*

$$[\Phi_{c_1}(r)] + \cdots + [\Phi_{c_m}(r)] = [\Phi_w(r)] \qquad \text{for every } r \in \operatorname{supp}(B_{\mathcal{R}}). \tag{12}$$

Under Assumption 5.3, (12) *states that* w *branches exhaustively* (Definition 5.6).

Proof. (a)⇒(b) is Theorem 5.12(3), because $1 \cdot \alpha(r) = \alpha(r)$. For (b)⇒(a), take $S = \mathrm{Bag}(\mathrm{Row}(\Sigma))$ and $\alpha(r) = \{\{r\}\}$, so that $\widehat{\alpha}$ is the identity. By Theorem 5.12(1) and (2), $s(v)$ is the bag $r \mapsto B_{\mathcal{R}}(r)[\Phi_v(r)]$ and $\overline{U}(v)$ is the bag $r \mapsto B_{\mathcal{R}}(r)\mu_v(r)$; they coincide only if $\mu_v(r) = [\Phi_v(r)]$ wherever $B_{\mathcal{R}}(r) > 0$.

For the last claim, use (9). If (12) holds at every internal node, then $\mu_w = [\Phi_w]$ on $\mathrm{supp}(B_{\mathcal{R}})$ follows by induction on the height of w. Conversely, if $\mu_w = [\Phi_w]$ and $\mu_{c_i} = [\Phi_{c_i}]$ on $\mathrm{supp}(B_{\mathcal{R}})$, then $\sum_i [\Phi_{c_i}] = \sum_i \mu_{c_i} = \mu_w = [\Phi_w]$ there. □

Remark 5.14 (Static and dynamic exhaustiveness). If the identity in Corollary 5.13(a) holds for every $r \in \mathrm{Row}(\Sigma)$ rather than only on $\mathrm{supp}(B_{\mathcal{R}})$, it is a property of the predicates alone, and by Theorem 5.12(3) consistency then holds for every relation. Such a tree can be certified from its filter expressions before any data is read, which matters for the lazy and SQL execution modes. The condition on $\mathrm{supp}(B_{\mathcal{R}})$ is the data-dependent version; it is the one satisfied by a tree whose children enumerate the key values observed in $\mathcal{R}$.

When the leaves below a node have the same key attributes, the two summaries are two merges of the same S-bag.

Corollary 5.15 (Exchange as a merge)**.** *Suppose that Assumption 5.3 holds and that all leaves below a node v have the same key attributes X, so that $X <: X_v$. For $x \in \mathrm{Row}(X)$ let $m_v(x) := \#\{\, p \in L(v) \mid u_p = x \,\}$ be the number of leaves below v with the key value x. Then*

$$s(v) = \bigoplus_{\substack{x \in \mathrm{supp}(G_X) \\ x <: u_v}} G_X(x), \qquad \overline{U}(v) = \bigoplus_{\substack{x \in \mathrm{supp}(G_X) \\ x <: u_v}} m_v(x) \cdot G_X(x).$$

The direct summary is the merge that squashes the attributes $X \setminus X_v$ (Definition 4.6)*, and the upward summary is the same merge with the term of x taken $m_v(x)$ times. They agree if $m_v(x) \cdot G_X(x) = G_X(x)$ for every $x \in \mathrm{supp}(G_X)$ with $x <: u_v$, in particular if every such key value is the key value of exactly one leaf below v.*

Proof. By (10) and Proposition 4.8(1), $s(v) = G_{X_v}(u_v) = \big((\mathrm{up}^X_{X_v})_* G_X\big)(u_v)$, which is the first formula. A leaf $p \in L(v)$ has $s(p) = G_X(u_p)$, and $u_p <: u_v$ because Φ_p implies Φ_v. By Theorem 5.12(2), $\overline{U}(v) = \bigoplus_{p \in L(v)} G_X(u_p)$; collecting the leaves with the same key value gives the second formula, in which the key values with $G_X(x) = \mathbf{0}_S$ contribute nothing. □

Here $\mu_v(r) = m_v(r[X])$, so the condition of the corollary is (11), summed over the rows of each group. The data decide which key values have a nonzero entry, and the tree decides which of them it lists and how often. Hierarchical consistency is the statement that the tree lists what the data contain.

For particular classes of monoids, condition (11) is weaker than the partition condition, or its failure can be quantified.

Corollary 5.16 (Idempotent summaries tolerate overlap)**.** *Suppose that $s \oplus s = s$ for all $s \in S$. If every $r \in \mathrm{supp}(B_{\mathcal{R}})$ with $\Phi_v(r)$ satisfies $\mu_v(r) \geq 1$, i.e. the leaves below v* cover *the rows reaching v, then $\overline{U}(v) = s(v)$, whether or not the leaves overlap. Conversely, if $\overline{U}(v) = s(v)$ for every idempotent commutative monoid and every contribution, then the leaves below v cover the rows reaching v.*

Proof. In an idempotent monoid, $n \cdot s = s$ for all $n \geq 1$, so (11) holds. For the converse, take $S = (\mathcal{P}_{\mathrm{fin}}(\mathrm{Row}(\Sigma)), \cup, \varnothing)$ and $\alpha(r) = \{r\}$; then $s(v) = \{r \in \mathrm{supp}(B_{\mathcal{R}}) \mid \Phi_v(r)\}$ and $\overline{U}(v) = \{r \in \mathrm{supp}(B_{\mathcal{R}}) \mid \mu_v(r) \geq 1\}$ by Theorem 5.12. □

Among the aggregates of Table 1, the summaries of `MIN`, `MAX`, `COUNT(DISTINCT)`, and the sketch of `APPROX_COUNT_DISTINCT` are idempotent. They are therefore correct on trees with overlapping siblings, such as sliding windows or cumulative thresholds, as long as no row is dropped.

Corollary 5.17 (Additive summaries: the discrepancy)**.** *Suppose that S is a submonoid of an abelian group $(G, +)$. Then, in G,*

$$\overline{U}(v) - s(v) = \sum_{r \in \mathrm{supp}(B_{\mathcal{R}})} B_{\mathcal{R}}(r)\,(\mu_v(r) - [\Phi_v(r)]) \cdot \alpha(r).$$

A row that reaches v but no leaf below v is missing once, and a row that reaches $\mu_v(r) \geq 2$ leaves is counted $\mu_v(r) - 1$ times too often.

Proof. Subtract the expressions in Theorem 5.12(1) and (2). □

The corollary applies to `COUNT` and to the summaries of `AVG` and `VAR` in Table 1, which lie in $\mathbb{Z}^n$. It does not apply to $(\mathbb{N}_0, +)_\perp$ as such, because the adjoined identity $\perp$ makes 0 a second idempotent; for `SUM`, use the counting completion $(\mathbb{N}_0, +)^{\#}$ of Definition 4.16, the summary $(\mathrm{count}, \mathrm{sum})$, and report `NULL` when the count is zero.

With a count on the summary space and a counting contribution (Definition 4.14), the count of the upward summary is available at every node, and it decides whether the summary is correct.

Proposition 5.18 (Counts and exhaustive branching)**.** *Let $\#$ be a count on S, α a counting contribution, and v a node of a reduction tree over $\mathcal{R}$.*

(1) Counts. $\#s(v) = \sum_r B_{\mathcal{R}}(r)\,[\Phi_v(r)]$*, the number of row occurrences that reach v, and* $\#\overline{U}(v) = \sum_r B_{\mathcal{R}}(r)\,\mu_v(r)$*, the number of selections of row occurrences by the leaves below v.*

(2) No tolerance. *For a row r, condition* (11)*, $\mu_v(r) \cdot \alpha(r) = \alpha(r)$, holds if and only if $\mu_v(r) = 1$. Hence $\overline{U}(v) = s(v)$ for every relation over Σ if and only if every row with $\Phi_v(r)$ satisfies the predicate of exactly one leaf below v.*

(3) The count as a certificate. *Suppose that no row of* $\mathrm{supp}(B_{\mathcal{R}})$ *is selected by two leaves below v, or that every row of* $\mathrm{supp}(B_{\mathcal{R}})$ *reaching v is selected by at least one. Then the following are equivalent:*
   (a) $\overline{U}(v) = s(v)$*;*
   (b) $\#\overline{U}(v) = \#s(v)$*;*
   (c) $\mu_v(r) = [\Phi_v(r)]$ *for every* $r \in \mathrm{supp}(B_{\mathcal{R}})$.
   In the first case $\#s(v) - \#\overline{U}(v)$ is the number of row occurrences that reach v and no leaf; in the second, $\#\overline{U}(v) - \#s(v)$ is the number of surplus selections.

(4) Value part. *Let $S = T^{\#}$ and $\alpha = \alpha_T^{\#}$ for a commutative monoid T and a contribution α_T. Then* $\mathrm{pr}\,\overline{U}(v)$ *and* $\mathrm{pr}\,s(v)$ *are the upward and the direct summary of v for α_T. In particular, if T is idempotent and the leaves below v cover the rows reaching v, then* $\mathrm{pr}\,\overline{U}(v) = \mathrm{pr}\,s(v)$ *even when the counts differ.*

Proof. (1) Apply the homomorphism # to Theorem 5.12(1) and (2) and use $\#\alpha(r) = 1$. (2) Applying # to $\mu_v(r) \cdot \alpha(r) = \alpha(r)$ gives $\mu_v(r) = 1$; the second claim is then Theorem 5.12(3). (3) (c) implies (a) by the formulas of Theorem 5.12(1) and (2), and (a) implies (b). By (1), $\#\overline{U}(v) - \#s(v) = \sum_r B_{\mathcal{R}}(r)\big(\mu_v(r) - [\Phi_v(r)]\big)$. Under the first hypothesis $\mu_v(r) \le [\Phi_v(r)]$ on $\operatorname{supp}(B_{\mathcal{R}})$, because $\mu_v(r) \le 1$ and $\mu_v(r) = 0$ where Φ_v fails; under the second, $\mu_v(r) \ge [\Phi_v(r)]$. In both cases all terms have the same sign, so the sum vanishes only if every term does, which is (c); and the sum counts what is stated. (4) pr is a monoid homomorphism with $\operatorname{pr} \circ \widehat{\alpha_T^{\#}} = \widehat{\alpha_T}$ (Proposition 4.17(2)), and both summaries are built from leaf summaries by $\oplus$. The last claim is Corollary 5.16. □

The first hypothesis of part (3) can be guaranteed from the predicates alone, for every relation: whenever all layers below v consist of equalities on a single column with distinct values, and in every level-uniform tree (Corollary 6.6). For such trees one integer comparison per node, between the merged count and the count of the rows delivered by the downward operator, decides whether the upward summary is correct. Without either hypothesis the comparison is not conclusive for a single relation: if one row is selected twice and another row with the same contribution is omitted, the summaries agree, and so do the counts, although the leaves do not partition the rows. Part (2) is the price of counting: a summary with a positive count is never idempotent, because $\#(s \oplus s) = 2\,\#s$, so the tolerance of overlap in Corollary 5.16 holds for the value part only, as stated in part (4).

Example 5.19. Let $\Sigma = \{os, x\}$, $B_{\mathcal{R}} = \{\{[\text{linux}, 10], [\text{mac}, 20]\}\}$, and let the root ε have two leaf children with predicates φ_1 and φ_2. Write r_1, r_2 for the two rows; the direct summaries of the root are 30 for `SUM` and 20 for `MAX`.

(i) *Partition.* $\varphi_1 \equiv (os = \text{linux})$, $\varphi_2 \equiv (os \neq \text{linux})$. Then $\mu_\varepsilon(r_1) = \mu_\varepsilon(r_2) = 1$, and $\overline{U}(\varepsilon) = 10 + 20 = 30$ for `SUM` and $\max(10, 20) = 20$ for `MAX`.

(ii) *Overlap.* $\varphi_1 \equiv (os = \text{linux})$, $\varphi_2 \equiv (x \ge 10)$. Then $\mu_\varepsilon(r_1) = 2$ and $\mu_\varepsilon(r_2) = 1$. For `SUM`, $\overline{U}(\varepsilon) = 10 + 30 = 40$, in excess by $(2-1) \cdot 10$ as predicted by Corollary 5.17. For `MAX`, $\overline{U}(\varepsilon) = \max(10, 20) = 20$ remains correct by Corollary 5.16.

(iii) *Omission.* $\varphi_1 \equiv (os = \text{linux})$, $\varphi_2 \equiv (os = \text{bsd})$. Then $\mu_\varepsilon(r_2) = 0$, and both aggregates are wrong: $\overline{U}(\varepsilon) = 10$ for `SUM` and for `MAX`.

Deferred readout. We now connect the upward summary to the upward operator U of Definition 5.5, which transports raw rows and recomputes statistics at each level. To avoid a clash with the row contribution α, write $\text{aug}\colon \mathcal{M}_k \to \widehat{\mathcal{M}}_k$ for the within-node aggregation of Definition 5.4, so that $\rho \circ \text{aug} = \text{id}$ and $U(v) = \text{aug}\big(\biguplus_i \rho(U(c_i))\big)$ at an internal node. For a node v, let $F_v \in \mathbb{R}^{n \times k}$ be the value matrix of B_v: its rows are the vectors $r[W]$, each $r \in \operatorname{supp}(B_v)$ repeated $B_v(r)$ times. At a leaf, $U(v) = \text{aug}(F_v)$; under Assumption 5.3, $F_v = F_{X_v, W}(u_v)$.

For a map $\beta\colon \mathbb{R}^{1\times k} \to S$ and a matrix $M \in \mathcal{M}_k$ with rows $M_{1\bullet}, \dots, M_{n\bullet}$, put

$$\widehat{\beta}(M) := \beta(M_{1\bullet}) \oplus \cdots \oplus \beta(M_{n\bullet}),$$

with $\widehat{\beta}(M) = \mathbf{0}_S$ for the $0 \times k$ matrix. Then $\widehat{\beta}(M \uplus M') = \widehat{\beta}(M) \oplus \widehat{\beta}(M')$, and $M \simeq M'$ implies $\widehat{\beta}(M) = \widehat{\beta}(M')$ by commutativity.

Definition 5.20 (Mergeable statistics)**.** The statistics block Stat is *S-mergeable* if there are maps $\beta\colon \mathbb{R}^{1\times k} \to S$ and $\gamma\colon S \to \mathbb{R}_\perp^{z\times k}$ such that $\text{Stat} = \gamma \circ \widehat{\beta}$ on $\mathcal{M}_k$.

Every symmetric Stat is mergeable for the free choice $S = \mathrm{Bag}(\mathbb{R}^{1\times k})$, $\beta(a) = \{\{a\}\}$; the definition has content when S has fixed width. For example, the column means are $(\mathbb{N}_0 \times \mathbb{R}^k, +)$-mergeable with $\beta(a) = (1, a)$ and $\gamma(n, t) = t/n$, where $\gamma(0, t) = (\bot, \ldots, \bot)$.

Theorem 5.21 (Deferred readout)**.** *Let* $\mathrm{Stat} = \gamma \circ \widehat{\beta}$ *be S-mergeable, and let the upward summary* $\overline{U}$ *be formed with the contribution* $\alpha(r) := \beta(r[W])$*. For every reduction tree and every node v:*

(1) $\widehat{\beta}(\rho(U(v))) = \overline{U}(v)$.
(2) *The statistics block of* $U(v)$ *equals* $\gamma(\overline{U}(v))$*. In particular, at an internal node with children* $c_1, \ldots, c_m$ *it equals* $\gamma(\overline{U}(c_1) \oplus \cdots \oplus \overline{U}(c_m))$ *and is determined by the children's summaries, without their raw parts.*
(3) *If* (11) *holds at v, the statistics block of* $U(v)$ *equals* $\gamma(s(v)) = \mathrm{Stat}(F_v)$*, the statistics of exactly the rows that the downward operator delivers to v.*

Proof. For every node v, the definitions of F_v and $\widehat{\alpha}$ give

$$\widehat{\beta}(F_v) = \bigoplus_{r \in \mathrm{supp}(B_v)} B_v(r) \cdot \beta(r[W]) = \widehat{\alpha}(B_v) = s(v). \tag{13}$$

(1) By induction on the height of v. At a leaf, $\rho(U(v)) = \rho(\mathrm{aug}(F_v)) = F_v$, and (13) gives $\widehat{\beta}(F_v) = s(v) = \overline{U}(v)$. At an internal node, $\rho(U(v)) = \biguplus_i \rho(U(c_i))$, so

$$\widehat{\beta}(\rho(U(v))) = \bigoplus_i \widehat{\beta}(\rho(U(c_i))) = \bigoplus_i \overline{U}(c_i) = \overline{U}(v).$$

(2) The statistics block of $U(v) = \mathrm{aug}(\rho(U(v)))$ is $\mathrm{Stat}(\rho(U(v))) = \gamma(\widehat{\beta}(\rho(U(v))))$; apply (1).

(3) By Theorem 5.12(3), $\overline{U}(v) = s(v)$, and $\gamma(s(v)) = \gamma(\widehat{\beta}(F_v)) = \mathrm{Stat}(F_v)$ by (13). □

Remark 5.22. Proposition 5.7(2) is the instance $S = \mathrm{Bag}(\mathbb{R}^{1\times k})$, $\beta(a) = \{\{a\}\}$ of Theorem 5.21: there $\widehat{\beta}(M)$ is the bag of rows of M, part (1) identifies it with the upward summary, and under exhaustive branching Corollary 5.13 gives $\rho(U(v)) \simeq F_v$. Theorem 5.21 adds that, for a mergeable Stat, the level aggregation λ of Definition 5.4 may be replaced by the merge $\oplus$ of fixed-width summaries; the raw part then need not be transported above the leaves.

The fiber as the free upward summary. The free choice of summary in Remark 5.22 is the fiber of Definition 4.18, the summary into which nothing is squashed. It is the one upward summary from which the rows themselves, and not only a statistic, can be recovered.

Proposition 5.23 (Fibers as summaries)**.** *Suppose that Assumption 5.3 holds. Let $v \in V$, let $Y \subseteq \Sigma$ be a set of kept attributes, and form the direct and upward summaries with* $S = \mathrm{Bag}(\mathrm{Row}(Y))$ *and* $\alpha(r) = \{\{r[Y]\}\}$*.*

(1) Downward. $s(v) = \mathrm{Fib}^Y_{X_v}(u_v)$*: the downward operator delivers to v the fiber over its key value.*
(2) Universality. *Let S' be a commutative monoid and* $\alpha' \colon \mathrm{Row}(Y) \to S'$ *a contribution. The direct summary of v for the contribution $r \mapsto \alpha'(r[Y])$ is* $\widehat{\alpha'}(\mathrm{Fib}^Y_{X_v}(u_v))$ (Theorem 4.19(4))*. Every summary of v that reads only the attributes Y is a homomorphic image of the fiber.*

(3) Upward. *If every internal node of the subtree rooted at v branches exhaustively, then*

$$\overline{U}(v) = \mathrm{Fib}^{Y}_{X_v}(u_v) = \biguplus_{p \in L(v)} s(p) :$$

merging the summaries of the leaves, which are subfilters of v, restores the fiber of v. At the root, $\overline{U}(\varepsilon) = B_Y$, and for $Y = \Sigma$ the upward pass returns the relation $\mathcal{R}$ itself.

Proof. (1) By Assumption 5.3, $B_v = \sigma_{X_v = u_v}(B_{\mathcal{R}})$, and $\widehat{\alpha} = (\mathrm{up}^{\Sigma}_{Y})_*$. Hence, by Definition 4.18,

$$s(v) = (\mathrm{up}^{\Sigma}_{Y})_* \, \sigma_{X_v = u_v}(B_{\mathcal{R}}) = \mathrm{Fib}^{Y}_{X_v}(u_v).$$

(2) The contribution $r \mapsto \alpha'(r[Y])$ has extension $\widehat{\alpha'} \circ (\mathrm{up}^{\Sigma}_{Y})_*$, because both are homomorphisms that agree on singletons; apply (1).

(3) is Corollary 5.13 together with (1) and Theorem 5.12(2); at the root, $X_\varepsilon = \varnothing$ and $\mathrm{Fib}^{Y}_{\varnothing}$ has the single value B_Y. □

When all leaves below v have the same key attributes X, so that $X <: X_v$ (Definition 4.1), part (3) is Theorem 4.19(2) with $X' = X_v$: the leaves' fibers are stacked into the fiber of v. The raw part $\rho(U(v))$ of Definition 5.5 is the tuple form of this summary for $Y = W$, which is Proposition 5.7. The converse direction is not available: by Theorem 4.19(3), the fiber of v with values in W does not determine the fibers of its children.

A fixed-width summary preserves a statistic but not the rows. For example, the bags $\{\{1, 3\}\}$ and $\{\{2, 2\}\}$ have the same $(\mathrm{count}, \mathrm{sum})$ summary $(2, 4)$, so no map recovers the rows from it. The framework therefore keeps the free summary at the levels at which rows are still needed, and passes to a fixed-width quotient by $\widehat{\alpha'}$ where they are not; by part (2) this passage can be made at any level and commutes with the merges above it.

Remark 5.24 (What the fiber adds to annotated aggregation). In the semantics of Amsterdamer, Deutch, and Tannen [1], grouping by X returns one tuple per group, whose aggregated attribute is a tensor $\sum_i k_i \otimes m_i \in K \otimes M$; the other attributes of the group's rows are not part of the result. For $K = \mathbb{N}$ the tensor is the aggregate value itself, since $\mathbb{N} \otimes M \cong M$. For $K = \mathbb{N}[\mathcal{X}]$ with one token per tuple it records the pairs of token and aggregated value, which is what is needed to re-evaluate the aggregate under any valuation of the tokens, but it is a formal expression over the aggregated attribute, to be interpreted against the input database.

The fiber is data instead: $\overrightarrow{\mathrm{Fib}}^{Y}_{X}(x) \in \mathrm{Row}(Y)^{B_X(x)}$ keeps the rows of the group for any set Y of kept attributes, and by Theorem 4.19(1) the labelled fibers are the projection of $\mathcal{R}$ onto $X \cup Y$. Three consequences follow that the tensor does not provide: the upward pass can return the original relation (Proposition 5.23(3)); every aggregate of the group, including a holistic one, and the fibers of every superfilter X' with $X <: X'$ can still be computed afterwards (Proposition 5.23(2) and Theorem 4.19(2)); and it is known exactly which levels can be recovered from which (Theorem 4.19(3) and Remark 4.20). In the vocabulary of [1], the fiber is the aggregation monoid $M = \mathrm{Bag}(\mathrm{Row}(Y))$, the free one. The two approaches are complementary: there the annotation semiring K varies in order to track which tuples matter, while here $K = \mathbb{N}$ is fixed and M is kept free for as long as the rows are needed.

The finalizer γ is applied to a merged summary, never before a merge. The next proposition states when the alternative, merging already finalized values, is possible at all.

Proposition 5.25 (Hoisting the finalizer)**.** *Let $\gamma\colon S \to \Omega$ be a map into a set Ω. There is a map $g\colon \gamma(S) \times \gamma(S) \to \Omega$ with $\gamma(a \oplus b) = g(\gamma(a), \gamma(b))$ for all $a, b \in S$ if and only if the kernel of γ is a congruence, i.e.*

$$\gamma(a) = \gamma(a') \implies \gamma(a \oplus b) = \gamma(a' \oplus b) \qquad \text{for all } a, a', b \in S.$$

Proof. If g exists and $\gamma(a) = \gamma(a')$, then $\gamma(a \oplus b) = g(\gamma(a), \gamma(b)) = g(\gamma(a'), \gamma(b)) = \gamma(a' \oplus b)$. Conversely, define $g(\gamma(a), \gamma(b)) := \gamma(a \oplus b)$. This is well defined: if $\gamma(a) = \gamma(a')$ and $\gamma(b) = \gamma(b')$, then $\gamma(a \oplus b) = \gamma(a' \oplus b) = \gamma(b \oplus a') = \gamma(b' \oplus a') = \gamma(a' \oplus b')$. □

When γ is injective, as for `COUNT`, `SUM`, `MIN`, and `MAX`, the condition is trivial; these are the distributive aggregates of Gray et al. [8]. For `AVG` it fails: $\gamma_{\text{avg}}(2,1) = \gamma_{\text{avg}}(4,2) = 2$, but $\gamma_{\text{avg}}\big((2,1) + (0,1)\big) = 1$ and $\gamma_{\text{avg}}\big((4,2) + (0,1)\big) = 4/3$. Hence no function of the children's means computes the parent's mean, on any tree; this is the algebraic reason behind the last item of Remark 6.9.

A level as a grouped aggregation. The elementary level, one parent with its children, factors into the two steps announced in the introduction.

Proposition 5.26 (Two-step level merge)**.** *Let v be an internal node with children $c_1, \dots, c_m$ and labels $\ell_1, \dots, \ell_m$. Regard the labels as the values of an attribute λ_v, the* label attribute *of v, with $\mathrm{Dom}(\lambda_v) = \{\ell_1, \dots, \ell_m\}$, and let $T_v \in S\langle\{\lambda_v\}\rangle$, $T_v(\ell_i) := \overline{U}(c_i)$, be the* level table *of v: an S-bag that keeps the label attribute and nothing else.*

(1) *$\overline{U}(v) = \mathrm{tot}(T_v)$, the total of Definition 4.6, which is a monoid homomorphism from the S-bags over the label attribute to S. Thus the up-merge at v consists of child aggregation, which produces one fixed-width row per label, followed by the squash of the label attribute, that is, the contraction of the label axis.*

(2) *Suppose that no row of $\mathrm{supp}(B_v)$ satisfies two of $\Phi_{c_1}, \dots, \Phi_{c_m}$ and that $\overline{U}(c_i) = s(c_i)$ for all i. Let κ_v be the partial function on $\mathrm{supp}(B_v)$ sending r to the label ℓ_i of the unique child with $\Phi_{c_i}(r)$, if there is one. Then*

$$T_v(\ell) = \widehat{\alpha}\big(\sigma_{\kappa_v = \ell}(B_v)\big) \qquad \text{for every label } \ell,$$

so T_v is the grouped aggregation of $D(v)(\mathcal{R})$ by the key function κ_v (Section 3.3), restricted to the rows on which κ_v is defined: with κ_v as a derived attribute (Remark 4.12), it is the squash of these rows onto the label attribute. If v moreover branches exhaustively, κ_v is total and T_v is the result of `GROUP BY` κ_v on $D(v)(\mathcal{R})$.

The only difference from `GROUP BY` is that a label whose selection is empty keeps a row with summary $\mathbf{0}_S$ in T_v, whereas `GROUP BY` omits the group.

Proof. (1) is Definition 5.11. For (2), a row $r \in \mathrm{supp}(B_v)$ satisfies $\Phi_{c_i}(r)$ if and only if $\kappa_v(r) = \ell_i$, so $B_{c_i} = \sigma_{\kappa_v = \ell_i}(B_v)$ by Lemma 5.9(2), and $T_v(\ell_i) = s(c_i) = \widehat{\alpha}(B_{c_i})$. □

Under Assumption 5.3, when all children of v have the same key attributes X', put $Z := X' \backslash X_v$, the attributes that the level fixes on the way down. The key function κ_v is $r \mapsto r[Z]$ up to the naming of labels, so the label attribute stands for Z, and Proposition 5.26(2) reads: the level at v computes `GROUP BY` Z on $D(v)(\mathcal{R})$, restricted to the listed key values.

In the nested form of Lemma 4.11, the squash $G_{X'}$ is a bag over X_v whose summaries are S-bags over Z: the attributes Z have been moved from the key into the summary and are not yet squashed. Its entry at u_v is the level table of v, when v lists every key value below u_v and $\overline{U}(c_i) = s(c_i)$ for all children. The up-merge is the change of summary along tot, which squashes Z and gives G_{X_v} (Proposition 4.8(1)). The two steps of a level are thus the two steps of every merge: the attributes of the level leave the key, and then they are aggregated away.

Remark 5.27 (Semimodule reading). The bag $B_v = B_{\mathcal{R}} \cdot [\Phi_v]$ is the selection of the $\mathbb{N}$-relational algebra, and $n \cdot s$ is the canonical $\mathbb{N}$-semimodule structure of S used by Amsterdamer, Deutch, and Tannen [1]. In their terms, parts (1) and (2) of Theorem 5.12 evaluate one formal aggregate $\sum_r x_r \otimes \alpha(r)$ under two valuations of the row tokens, $x_r \mapsto B_{\mathcal{R}}(r)[\Phi_v(r)]$ and $x_r \mapsto B_{\mathcal{R}}(r)\mu_v(r)$, and Corollary 5.16 reflects the fact that the idempotent monoids are exactly the $\mathbb{B}$-semimodules, for which only the support of a bag matters. The theorem relies on edge predicates that read base attributes only, so that μ_v does not depend on $\mathcal{R}$. A predicate on an aggregated value, such as a `HAVING` clause or a top-k pruning of children, makes the selection depend non-monotonically on the rows and is outside its scope; it corresponds to the nested aggregation queries of [1].

5.4 Dense layers and dense trees: `ROLLUP`

A reduction tree allows arbitrary sibling predicates (Definition 5.1). The layer that occurs most often has one child for each value of a single column. We call it *dense*. For a dense layer the conditions of Section 5.3 hold by construction, and the layer is one step of the subfilter order: on the way down it adds one attribute to the kept ones, and on the way up it squashes this attribute.

Throughout this subsection Assumption 5.3 is in force. A *layer* is an internal node v together with its children. For an attribute $a \notin X_v$ and a value $c \in \mathrm{Dom}(a)$, write $(u_v, c) \in \mathrm{Row}(X_v \cup \{a\})$ for the key value that extends u_v by $a = c$; thus $(u_v, c) <: u_v$.

Definition 5.28 (Active domain, dense layer)**.** Let $v \in V$ and $a \in \mathcal{K} \setminus X_v$. The *active domain* of a at v is the set of values that a takes on the rows reaching v,

$$\mathrm{adom}_v(a) := \{\, r[a] \mid r \in \mathrm{supp}(B_{\mathcal{R}}),\ r[X_v] = u_v \,\} = \mathrm{supp}(\mathrm{Fib}^{\{a\}}_{X_v}(u_v)) \subseteq \mathrm{Dom}(a).$$

An internal node v is *dense on* a if there is a finite set C_v with

$$\mathrm{adom}_v(a) \subseteq C_v \subseteq \mathrm{Dom}(a)$$

such that v has exactly one child v_c for each $c \in C_v$, with edge predicate $a = c$ and label c. The layer is *tight* if $C_v = \mathrm{adom}_v(a)$ and *full* if $C_v = \mathrm{Dom}(a)$.

A tight layer has one child for each value of a that occurs in $D(v)(\mathcal{R})$; these are the groups of `GROUP BY` a. A full layer has one child for each value that a can take, whether or not it occurs, and requires $\mathrm{Dom}(a)$ to be finite. Every dense layer lies between the two.

Proposition 5.29 (Dense layer)**.** *Let v be dense on a, with children v_c, $c \in C_v$.*

(1) One subfilter step. *Every child has $X_{v_c} = X_v \cup \{a\}$ and $u_{v_c} = (u_v, c) <: u_v$. The step $X_v \cup \{a\} <: X_v$ is a covering: the only Z with $X_v \cup \{a\} <: Z <: X_v$ are $X_v \cup \{a\}$ and X_v. A key $x \in \mathrm{supp}(B_{X_v \cup \{a\}})$ satisfies $x <: u_v$ if and only if $x = (u_v, c)$ for some $c \in \mathrm{adom}_v(a)$. Hence the children of a tight layer are exactly the keys of $\mathrm{supp}(B_{X_v \cup \{a\}})$ below u_v, and the remaining children of a dense layer have empty fibers.*

(2) Exhaustive. *Every row r with $\Phi_v(r)$ satisfies*

$$\sum_{c\in C_v} [\Phi_{v_c}(r)] = [\, r[a] \in C_v \,] \le 1,$$

with equality for $r \in \mathrm{supp}(B_{\mathcal{R}})$, and for every r if the layer is full. Hence v branches exhaustively (Definition 5.6)*. Conversely, an internal node whose edge predicates are $a = c$ for pairwise distinct values $c \in C$ branches exhaustively only if* $\mathrm{adom}_v(a) \subseteq C$.

(3) Split and stack. *For every $Y \subseteq \Sigma$,*

$$\mathrm{Fib}^Y_{X_v}(u_v) = \biguplus_{c\in C_v} \mathrm{Fib}^Y_{X_v\cup\{a\}}(u_v, c), \qquad B_{X_v}(u_v) = \sum_{c\in C_v} B_{X_v\cup\{a\}}(u_v, c).$$

If $a \in Y$, then conversely $\mathrm{Fib}^Y_{X_v\cup\{a\}}(u_v, c) = \sigma_{a=c}(\mathrm{Fib}^Y_{X_v}(u_v))$ for every $c \in C_v$. The downward split by a and the upward stack are then inverse to each other.

(4) `GROUP BY`. *Let S be a commutative monoid and $\alpha\colon \mathrm{Row}(\Sigma) \to S$ a contribution, and suppose that $\overline{U}(v_c) = s(v_c)$ for all $c \in C_v$, as is the case when the children are leaves. Then the level table of v is*

$$T_v(c) = \widehat{\alpha}(\sigma_{a=c}(B_v)) \quad (c \in C_v), \qquad T_v(c) = \mathbf{0}_S \quad (c \notin \mathrm{adom}_v(a)),$$

that is, $T_v(c) = G_{X_v\cup\{a\}}(u_v, c)$: the level table is the entry at u_v of the squash onto $X_v \cup \{a\}$, nested at X_v (Lemma 4.11)*, with a as the label attribute. Squashing a gives $\overline{U}(v) = \mathrm{tot}(T_v) = s(v) = G_{X_v}(u_v)$. If the layer is tight, T_v is the result of `GROUP BY` a on $D(v)(\mathcal{R})$: one summary for each of the $|\,\mathrm{adom}_v(a)|$ groups, and no other row. If moreover S has a count and α is counting, then $\mathrm{supp}(T_v) = \mathrm{adom}_v(a)$ for every dense layer: the groups are read off the summaries.*

Proof. (1) The cumulative predicate of v_c is $\Phi_v \wedge (a = c)$, which holds at r if and only if $r[X_v \cup \{a\}] = (u_v, c)$. A set Z with $X_v \subseteq Z \subseteq X_v \cup \{a\}$ is one of the two. A key x over $X_v \cup \{a\}$ with $x <: u_v$ equals $(u_v, x[a])$, and $B(x) > 0$ if and only if some row of $\mathrm{supp}(B_{\mathcal{R}})$ reaching v has $r[a] = x[a]$.

(2) A row with $\Phi_v(r)$ satisfies $\Phi_{v_c}(r)$ if and only if $r[a] = c$, and it has one value of a. If $r \in \mathrm{supp}(B_{\mathcal{R}})$, then $r[a] \in \mathrm{adom}_v(a) \subseteq C_v$. Thus every row of $\mathrm{Fib}_{X_v}(u_v)$ lies in the fiber of exactly one child. For the converse, let $c' \in \mathrm{adom}_v(a)$ and choose $r \in \mathrm{supp}(B_{\mathcal{R}})$ with $r[X_v] = u_v$ and $r[a] = c'$; exhaustiveness gives a child whose predicate $a = c$ holds at r, so $c' = c \in C$.

(3) By (1), the keys $x \in \mathrm{supp}(B_{X_v\cup\{a\}})$ with $x <: u_v$ are the keys (u_v, c), $c \in \mathrm{adom}_v(a)$, and the fibers over (u_v, c) for $c \in C_v \setminus \mathrm{adom}_v(a)$ are empty; apply Theorem 4.19(2) with $X = X_v \cup \{a\}$ and $X' = X_v$, and count the elements. If $a \in Y$, then for $y \in \mathrm{Row}(Y)$

$$\mathrm{Fib}^Y_{X_v\cup\{a\}}(u_v, c)(y) = \sum_{\substack{r[X_v]=u_v \\ r[a]=c,\ r[Y]=y}} B_{\mathcal{R}}(r) = [\, y[a] = c \,] \sum_{\substack{r[X_v]=u_v \\ r[Y]=y}} B_{\mathcal{R}}(r),$$

because $r[Y] = y$ implies $r[a] = y[a]$.

(4) By (2), Proposition 5.26(2) applies with the key function $\kappa_v(r) = r[a]$, which is total on $\mathrm{supp}(B_v)$. A value $c \notin \mathrm{adom}_v(a)$ selects no row of B_v, so its summary is $\widehat{\alpha}(\varnothing_{\mathrm{bag}}) = \mathbf{0}_S$; for a counting contribution the converse holds by Proposition 4.15(1). The identity $\overline{U}(v) = s(v)$ is Corollary 5.13, and $s(v) = G_{X_v}(u_v)$ is (10). The groups of `GROUP BY` a on $D(v)(\mathcal{R})$ are the values in $\mathrm{adom}_v(a)$. □

By part (2), density is exactly what makes a layer of single-column equalities exhaustive. The disjointness of the children is a property of the predicates, since a row has one value of a. Their coverage depends on $\mathcal{R}$ for a tight layer and is again a property of the predicates for a full layer (Remark 5.14). By part (3), a dense layer loses nothing when the split column is kept in the fiber: the parent's fiber and the family of the children's fibers determine each other. With a fixed-width summary the way up is a squash of the split column and has no inverse (Proposition 4.9).

Dense trees. Stacking dense layers on distinct columns gives the tree of a multi-column grouping together with all its subtotals.

Definition 5.30 (Dense tree)**.** Let $\mathcal{R}$ be nonempty, let $a_1, \dots, a_d \in \mathcal{K}$ be distinct attributes, and put $X_j := \{a_1, \dots, a_j\}$ for $0 \le j \le d$. For $x \in \operatorname{supp}(B_{X_d})$ let

$$p_x := (e_1, \dots, e_d), \qquad e_i := (a_i = x[a_i],\ \mathrm{Stat},\ x[a_i]).$$

The *dense tree* on $(a_1, \dots, a_d)$ is $E(a_1, \dots, a_d) := \{\, p_x \mid x \in \operatorname{supp}(B_{X_d}) \,\}$.

Theorem 5.31 (Dense trees compute `ROLLUP`)**.** *Let $E = E(a_1, \dots, a_d)$.*

(1) Structure. *E is a reduction tree in which all paths have length d. For $0 \le j \le d$, the map $v \mapsto u_v$ is a bijection from the nodes of depth j onto* $\operatorname{supp}(B_{X_j})$*, and $X_v = X_j$. Every node of depth $j-1$ is tightly dense on a_j: it has one child for each value of a_j on the rows that reach it, and these are the groups of* `GROUP BY` *a_j on these rows. E is the only reduction tree with reducer* Stat *and these labels that has this property for $1 \le j \le d$. The number of nodes is*

$$\sum_{j=0}^{d} \bigl|\operatorname{supp}(B_{X_j})\bigr| \le 1 + d\,\bigl|\operatorname{supp}(B_{X_d})\bigr|.$$

(2) Summaries. *For every commutative monoid S, every contribution α, and every node v of depth j,*

$$\overline{U}(v) = s(v) = G_{X_j}(u_v) = \widehat{\alpha}(\sigma_{X_j = u_v}(B_{\mathcal{R}})).$$

Hence the summaries at depth j are the entries of the squash G_{X_j} of $\mathcal{R}$ onto X_j (Definition 4.4)*, that is,* `GROUP BY` *$a_1, \dots, a_j$, and the $d+1$ depths together are* `GROUP BY ROLLUP(`*$a_1, \dots, a_d$*`)`*: the leaves are the finest grouping and the root is the grand total. The upward pass is the chain of merges*

$$G_{X_d} \longmapsto G_{X_{d-1}} \longmapsto \cdots \longmapsto G_{X_1} \longmapsto G_{X_0} = \widehat{\alpha}(B_{\mathcal{R}}),$$

in which the step from depth j to depth $j-1$ squashes the column a_j. The level table of a node v of depth $j-1$ has the entries $G_{X_j}(u_v, c)$, $c \in \operatorname{adom}_v(a_j)$, where (u_v, c) extends u_v by $a_j = c$.

(3) Fibers. *For every $Y \subseteq \Sigma$, the upward summary with values in* $\mathrm{Bag}(\mathrm{Row}(Y))$ *is the fiber of v, restored from the leaves below it:*

$$\overline{U}(v) = \mathrm{Fib}^{Y}_{X_j}(u_v) = \biguplus_{\substack{x \in \operatorname{supp}(B_{X_d}) \\ x <: u_v}} \mathrm{Fib}^{Y}_{X_d}(x).$$

Proof. (1) E is finite, nonempty because $\mathcal{R}$ is, and prefix-free because its paths are distinct and have the same length. The depth-j prefix of p_x consists of the edge records for

$a_1, \dots, a_j$ and is determined by $x[X_j]$, with distinct values of $x[X_j]$ giving distinct prefixes; its cumulative predicate is $X_j = x[X_j]$. The values $x[X_j]$, $x \in \mathrm{supp}(B_{X_d})$, are the elements of $\mathrm{supp}(B_{X_j})$. Let v have depth $j-1$. Its children are the prefixes with key values $x[X_j] <: u_v$, one for each value in $\{x[a_j] \mid x \in \mathrm{supp}(B_{X_d}),\ x[X_{j-1}] = u_v\} = \mathrm{adom}_v(a_j)$, with edge predicate $a_j = x[a_j]$; so v is tightly dense on a_j. Conversely, in a tree with this property the set of nodes of depth j is determined by the set of nodes of depth $j-1$, and every node of depth less than d has a nonempty fiber and therefore a child, so the tree is E. The bound holds because $|\mathrm{supp}(B_{X_j})| \le |\mathrm{supp}(B_{X_d})|$ and $|\mathrm{supp}(B_{X_0})| = 1$.

(2) By (1) and Proposition 5.29(2), every internal node branches exhaustively, so $\overline{U}(v) = s(v)$ by Corollary 5.13, and $B_v = \sigma_{X_j = u_v}(B_{\mathcal{R}})$. By the bijection in (1), the nodes of depth j are the groups of $\mathcal{R}$ by X_j, and their summaries are the entries of G_{X_j} by (10); the entries at the other key values are $\mathbf{0}_S$. The step from G_{X_j} to $G_{X_{j-1}}$ is the merge of Proposition 4.8(1), and the level table is that of Proposition 5.29(4). The grouping sets of `ROLLUP` are $X_d, X_{d-1}, \dots, X_0$.

(3) is Proposition 5.23(3) and Theorem 4.19(2) with $X = X_d$ and $X' = X_j$. □

A layer that is dense on a set Z of several columns, with one child for each value of Z in the fiber of v, is therefore not a new notion: by Theorem 5.31 applied below v, its children are the nodes $|Z|$ levels below v in the dense tree on the columns of Z, taken in any order, and the intermediate nodes are the subtotals. Single-column dense layers generate every `GROUP BY`.

Remark 5.32 (Full density and the summary array). Suppose that $\mathrm{Dom}(a_1), \dots, \mathrm{Dom}(a_d)$ are finite and that every layer is full instead of tight. The leaves are then indexed by $\mathrm{Dom}(a_1) \times \cdots \times \mathrm{Dom}(a_d)$, and the leaf summaries form an order-d array

$$T\colon \mathrm{Dom}(a_1) \times \cdots \times \mathrm{Dom}(a_d) \to S, \qquad T(x) = \widehat{\alpha}\big(\sigma_{X_d = x}(B_{\mathcal{R}})\big),$$

with $T(x) = \mathbf{0}_S$ at every cell that contains no row. This array is the squash G_{X_d} of Definition 4.4, written out on all of $\mathrm{Row}(X_d)$. The summary of a node of depth j with key u is the contraction of the last $d-j$ axes, $\bigoplus_{x <: u} T(x)$, which squashes the columns $a_{j+1}, \dots, a_d$, and exhaustive branching is a property of the predicates (Remark 5.14). The tight tree is the sparse form of the same array: it stores T on $\mathrm{supp}(B_{X_d})$ only. The two forms have the same summaries at every node they share, because the omitted cells are $\mathbf{0}_S$. They differ in size, $\prod_j |\mathrm{Dom}(a_j)|$ against $|\mathrm{supp}(B_{X_d})|$ leaves, and in the empty cells, which the full tree reports with the readout $\gamma(\mathbf{0}_S)$ and the tight tree omits, as `GROUP BY` does.

6 Reordering the levels

This section answers (P2). The levels of a tree can be permuted when all nodes of a level split on the same attributes. The permuted tree has the same root and the same leaves, and a node at an intermediate level is determined by the set of levels above it and not by their order. For dense trees, the $d!$ orders are the maximal chains of the lattice of grouping sets, and together they compute `CUBE`.

6.1 Level-uniform trees and level permutations

Write $[d] := \{1, \dots, d\}$ and, for a permutation π of $[d]$, $\pi([j]) := \{\pi(1), \dots, \pi(j)\}$, with $\pi([0]) = \emptyset$.

Lemma 6.1 (Selections commute)**.** *For arbitrary predicates φ, ψ over Σ,*

$$\sigma_\varphi \circ \sigma_\psi = \sigma_{\varphi \wedge \psi} = \sigma_\psi \circ \sigma_\varphi.$$

Consequently, for predicates $\varphi_1, \ldots, \varphi_d$, every permutation π of $[d]$, and every $0 \le j \le d$,

$$\sigma_{\varphi_{\pi(j)}} \circ \cdots \circ \sigma_{\varphi_{\pi(1)}} = \sigma_{\bigwedge_{i \in \pi([j])} \varphi_i},$$

which depends on π only through the set *$\pi([j])$. In particular, the selections at the root ($j = 0$) and at the leaf ($j = d$) are invariant under reordering.*

Figure 4 shows the case of two predicates.

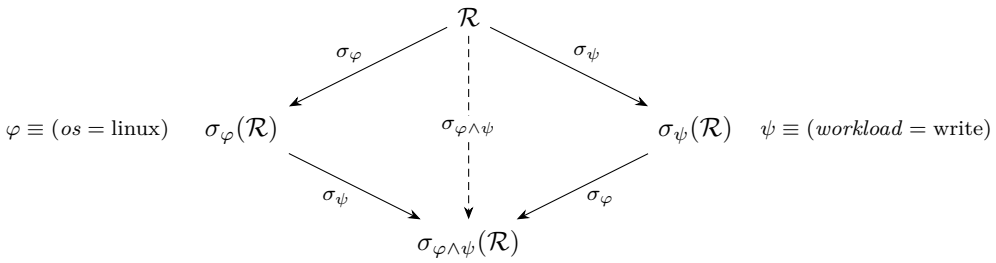


Figure 4: Selections commute (Lemma 6.1). The left path filters by *os* and then by *workload*, the right path in the opposite order. Both reach the relation selected by the conjunction, so a leaf does not depend on the order of its edge predicates; the intermediate relations $\sigma_\varphi(\mathcal{R})$ and $\sigma_\psi(\mathcal{R})$ differ.

Reordering a single path in isolation is not a tree operation, because paths share prefixes: the levels must be permuted in all paths simultaneously. This requires that all nodes of a level split on the same attributes.

Definition 6.2 (Level-uniform tree)**.** A reduction tree E is *level-uniform of depth d* if

1. every path in E has length d;
2. there are pairwise disjoint, nonempty attribute sets $Y_1, \ldots, Y_d \subseteq \mathcal{K}$ such that every edge record $e = (\varphi, U, \ell)$ occurring at position i of a path in E satisfies $\mathrm{attr}(\varphi) = Y_i$ and
$$\varphi(r) \iff r[Y_i] = c_e \qquad \text{for all } r \in \mathrm{Row}(\Sigma),$$
for some key value $c_e \in \mathrm{Row}(Y_i)$; and
3. distinct edge records occurring at the same position i have distinct key values c_e and distinct labels.

For a path $p = (e_1, \ldots, e_d) \in E$ and $I \subseteq [d]$, let

$$X_I := \bigcup_{i \in I} Y_i, \qquad \Phi_{p,I} := \bigwedge_{i \in I} \varphi_i,$$

and let $u_{p,I} \in \mathrm{Row}(X_I)$ be the unique key value with $u_{p,I}[Y_i] = c_{e_i}$ for all $i \in I$. Then

$$\Phi_{p,I}(r) \iff r[X_I] = u_{p,I}, \qquad u_{p,J}[X_I] = u_{p,I} \quad \text{for } I \subseteq J \subseteq [d].$$

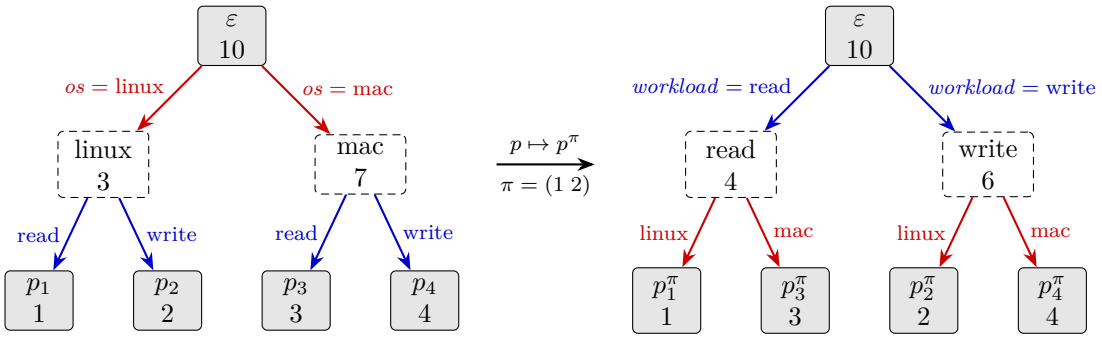
A level-uniform tree satisfies Assumption 5.3, with $X_j = X_{[j]}$ and $u_j = u_{p,[j]}$ along each path p.

Definition 6.3 (Level permutation)**.** Let E be level-uniform of depth d and let π be a permutation of $[d]$. For $p = (e_1, \dots, e_d) \in E$ put

$$p^\pi := (e_{\pi(1)}, \dots, e_{\pi(d)}), \qquad E^\pi := \{\, p^\pi \mid p \in E \,\}.$$

We write U^π for the upward operator of E^π (Definition 5.5), with the same W and the same statistics $s_1, \dots, s_z$.

Figure 5 shows a level-uniform tree of depth 2 and its permutation.



E: levels $Y_1 = \{os\}$, $Y_2 = \{workload\}$ $\qquad$ E^π: levels $Y_{\pi(1)} = \{workload\}$, $Y_{\pi(2)} = \{os\}$

Figure 5: Level permutation (Definition 6.3) of a level-uniform tree of depth 2. An edge at level i carries the predicate $Y_i = c$; at the lower level only the value c is shown. The color of an edge is the level of E from which its edge record comes: red for $Y_1 = \{os\}$ and blue for $Y_2 = \{workload\}$. Both trees consist of the same edge records; in E the red level is above the blue one, and in E^π the blue level is above the red one. Each node shows its key value and, as an example of a summary, the sum of a measure with the leaf sums $1, 2, 3, 4$. Shaded nodes are invariant (Theorem 6.4(4a,b)): the root is the same, and the leaves p_i and p_i^π select the same rows, here $p_1 = (\text{linux}, \text{read})$, $p_2 = (\text{linux}, \text{write})$, $p_3 = (\text{mac}, \text{read})$, $p_4 = (\text{mac}, \text{write})$. Dashed nodes differ: the nodes of depth 1 group by *os* in E and by *workload* in E^π, with $3 + 7 = 4 + 6 = 10$. For dense trees these are the two orders of Theorem 6.8: $E = E(os, workload)$ and $E^\pi = E(workload, os)$.

Theorem 6.4 (Level commutativity)**.** *Let E be level-uniform of depth d, let π be a permutation of $[d]$, let $p \in E$ and $0 \le j \le d$, and put*

$$I := \pi([j]), \qquad v := (e_{\pi(1)}, \dots, e_{\pi(j)}),$$

the depth-j node of E^π on the path p^π.

1. Well-formedness. *The tree E^π is a reduction tree. It is level-uniform with the attribute sets $Y_{\pi(1)}, \dots, Y_{\pi(d)}$, and $p \mapsto p^\pi$ is a bijection $E \to E^\pi$ between the leaves.*
2. Downward. *$D(v) = \sigma_{\Phi_{p,I}}$, $X_v = X_I$, and $u_v = u_{p,I}$. Hence the nodes of E^π at depth j correspond bijectively to the key values $\{u_{q,I} \mid q \in E\}$, and $D(v)$, X_v, u_v depend on π only through I.*
3. Upward.
$$\rho(U^\pi(v)) \simeq \biguplus_{\substack{q \in E \\ u_{q,I} = u_v}} F_{X_{[d]}, W}(u_{q,[d]}).$$
Hence, given E, the raw part of $U^\pi(v)$ is determined up to row equivalence, and its statistics block exactly, by the pair (I, u_v).

4. Invariants.
   (a) *Leaves:* $U^\pi(p^\pi) = U(p)$.
   (b) *Root:* $\rho(U^\pi(\varepsilon)) \simeq \rho(U(\varepsilon))$, *with identical statistics blocks.*
   (c) *If* $\pi([j]) = [j]$, *then the depth-*j *nodes of* E *and* E^π *with equal key values carry row-equivalent raw parts and identical statistics blocks.*
   (d) *For* $1 \le j < d$, *the adjacent transposition* $\pi = (j\ \ j{+}1)$ *changes only the nodes at depth* j.
5. Exhaustiveness. *If every internal node of* E *branches exhaustively, then so does every internal node of* E^π, *and* $\rho(U^\pi(v)) \simeq F_{X_I,W}(u_v)$.

Proof. (1) E^π is finite and nonempty, and all its paths have length d, so it is prefix-free. Sibling edges leaving a node of depth $j-1$ in E^π are distinct edge records from position $\pi(j)$ of E, so they have distinct key values and labels by Definition 6.2(3). The map $p \mapsto p^\pi$ is injective because π is invertible.

(2) The first claim is Lemma 6.1; the others follow from Definition 6.2. Two paths p^π, q^π share their depth-j node iff p and q carry the same edge records at all positions in I, which by Definition 6.2(3) holds iff $u_{p,I} = u_{q,I}$.

(3) Unfolding Definitions 5.4 and 5.5 by induction on the height shows that, in any reduction tree, the raw part at a node is row equivalent to the stack of the fiber value matrices of the leaves below it. By (2), the leaves below v are the paths q^π with $u_{q,I} = u_v$, and the leaf q^π has key attributes $X_{[d]}$ and key value $u_{q,[d]}$. The statistics block is determined by symmetry of $s_1, \dots, s_z$.

(4) For (a), take $j = d$: then $I = [d]$, both leaves have key attributes $X_{[d]}$ and key value $u_{p,[d]}$, and both results equal $\mathrm{aug}\big(F_{X_{[d]},W}(u_{p,[d]})\big)$. For (b), take $j = 0$: the stack in (3) ranges over all $q \in E$ for both trees. For (c), apply (3) to E (with the identity permutation) and to E^π. For (d), the transposition satisfies $\pi([i]) = [i]$ for all $i \ne j$.

(5) Since E branches exhaustively and the root fiber is $\mathrm{Fib}_\emptyset(u_\varepsilon) = \mathrm{supp}(B_\mathcal{R})$, repeated application of Definition 5.6 shows that the leaf fibers $\mathrm{Fib}_{X_{[d]}}(u_{q,[d]})$, $q \in E$, partition $\mathrm{supp}(B_\mathcal{R})$. Let $L(v) := \{q \in E \mid u_{q,I} = u_v\}$. For $q \in L(v)$, the leaf fiber of q is contained in $\mathrm{Fib}_{X_I}(u_v)$, because $u_{q,[d]}[X_I] = u_{q,I} = u_v$. Conversely, each $r \in \mathrm{Fib}_{X_I}(u_v)$ lies in the leaf fiber of exactly one $q \in E$, and then $u_{q,I} = r[X_I] = u_v$, so $q \in L(v)$. Thus the leaf fibers of $L(v)$ partition $\mathrm{Fib}_{X_I}(u_v)$. The same holds for every child of v in E^π, and $L(v)$ is the disjoint union of the sets $L(c)$ over the children c of v; hence the children's fibers partition the fiber of v. The last claim follows from Proposition 5.7 applied to E^π. □

Remark 6.5 (Lattice view). By Theorem 6.4, a node is determined by a pair (I, u) with $I \subseteq [d]$ and $u \in \mathrm{Row}(X_I)$, not by the order in which the levels in I were applied. The index sets form the Boolean lattice $(2^{[d]}, \subseteq)$, and by Lemma 4.2 $I \subseteq J$ implies $X_J <: X_I$, so that the grouping by X_J refines the grouping by X_I. An ordering of the levels is a maximal chain

$$\emptyset = \pi([0]) \subset \pi([1]) \subset \cdots \subset \pi([d]) = [d]$$

in this lattice. All $d!$ chains share the bottom element (the root) and the top element (the leaves), which is why these two levels are invariant; two orderings share an intermediate level j exactly when their chains pass through the same set, i.e. $\pi([j]) = \pi'([j])$. Figure 6 shows the lattice for $d = 3$ with three orderings.

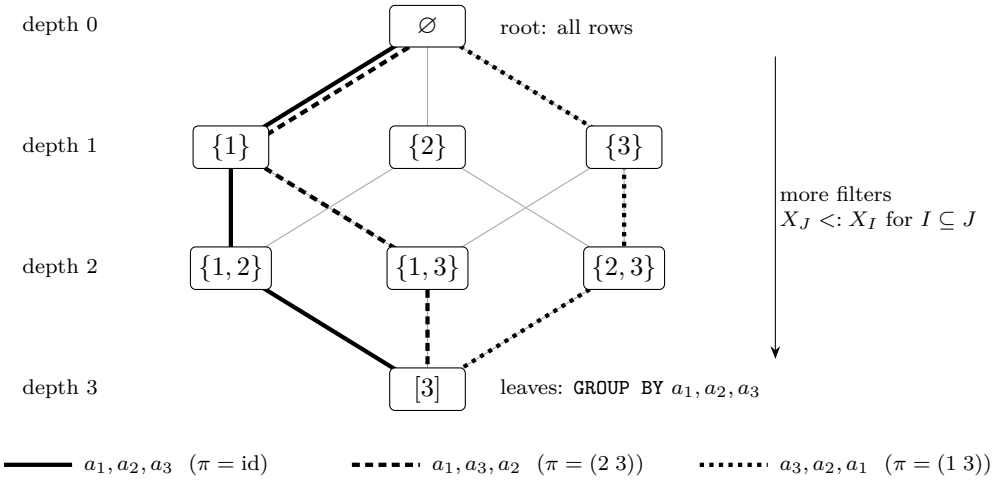


Figure 6: The level lattice for $d = 3$ (Remark 6.5). A node is an index set $I \subseteq [3]$ and stands for all tree nodes (I, u), the groups of `GROUP BY` X_I; a grey line is a covering step, which adds one level. An ordering of the levels is a maximal chain from $\varnothing$ to $[3]$, and three of the six are drawn. All chains share the root and the leaves. The solid and the dashed chain differ by the adjacent transposition $(2\ 3)$ and therefore only at depth 2 (Theorem 6.4(4d)); the dotted chain shares no intermediate level with either (Theorem 6.8(3)). Section 7 attaches the number $\nu(I)$ to each node and chooses the chain along which it rises earliest.

6.2 Summaries on the level lattice

Theorem 6.4 is stated for raw parts and statistics blocks. The same holds for the summaries of Section 5.3, for every summary monoid.

Corollary 6.6 (Summaries on the level lattice)**.** *Let E be level-uniform of depth d, let π be a permutation of $[d]$, and let v be the depth-j node of E^π on the path p^π, with $I = \pi([j])$. Write $\overline{U}^\pi$ for the upward summary of E^π. Then*

$$\overline{U}^\pi(v) = \bigoplus_{\substack{q \in E \\ u_{q,I} = u_v}} s(q), \qquad \mu_v(r) \le 1 \ \text{ for all } r \in \mathrm{Row}(\Sigma).$$

Hence the upward summary at v depends on π only through the pair (I, u_v), for every commutative monoid S, and $\overline{U}^\pi(v) = \widehat{\alpha}(\sigma_{X_I = u_v}(B_{\mathcal{R}}))$ whenever (11) holds at v; this is the case, in particular, when E branches exhaustively.

Proof. By Theorem 6.4(2), the leaves of E^π below v are the paths q^π with $u_{q,I} = u_v$, and $\Phi_{q^\pi} = \Phi_q$ by Lemma 6.1, so $s(q^\pi) = s(q)$; the formula is Theorem 5.12(2). Distinct leaves of a level-uniform tree differ in an edge record at some position i and therefore require different values of $r[Y_i]$ (Definition 6.2(3)), so at most one leaf predicate holds at any row. The last claim is Theorem 5.12(3) with $\Phi_v \equiv (X_I = u_v)$, together with Theorem 6.4(5) and Corollary 5.13. □

When E branches exhaustively, the summary at the node (I, u) is therefore the entry $G_{X_I}(u)$ of the squash of $\mathcal{R}$ onto X_I (Definition 4.4), which keeps the levels in I and squashes all others. The lattice of Remark 6.5 therefore carries one S-bag G_{X_I} at each index set I, and for $I \subseteq J$ the merge along the upcast squashes the levels in $J \setminus I$ and takes G_{X_J} to G_{X_I} (Proposition 4.8(1)). An ordering of the levels selects the S-bags along one

maximal chain: it is the order in which the levels are kept on the way down, and squashed, in reverse, on the way up. The two ends of every chain are the same, the squash $G_{X_{[d]}}$ that keeps every level and the squash $G_\emptyset$ that keeps none, and since a squash may be carried out in any number of steps (Lemma 4.7(2)), every chain leads from the first to the second. For summaries, this is the invariance of the leaves and of the root in Theorem 6.4(4). The intermediate S-bags differ because they keep different levels, and when the contribution reads no attribute of the levels, none of them is determined by another unless its index set is contained in the other's (Proposition 4.9).

6.3 All orders of a dense tree: `CUBE`

For the dense trees of Definition 5.30 every condition above holds by construction, and the summaries on the lattice are the grouping sets of the cube.

Corollary 6.7 (Dense trees are level-uniform)**.** *Let $E = E(a_1, \dots, a_d)$ and let π be a permutation of $[d]$.*

(1) *E is level-uniform of depth d* (Definition 6.2) *with $Y_i = \{a_i\}$, so that $X_I = \{a_i \mid i \in I\}$ for $I \subseteq [d]$.*
(2) *$E^\pi = E(a_{\pi(1)}, \dots, a_{\pi(d)})$* (Definition 6.3)*: permuting the levels of a dense tree gives the dense tree of the permuted columns. In particular, every layer of E^π is tightly dense, and E^π has one dense layer for each covering step of the chain*

$$X_{[d]} = X_{\pi([d])} <: X_{\pi([d-1])} <: \cdots <: X_{\pi([1])} <: X_{\pi([0])} = \varnothing.$$

Proof. (1) All paths have length d; the edge records at position i are $(a_i = c, \mathrm{Stat}, c)$, so they have attribute set $\{a_i\}$, and distinct records have distinct values and labels. (2) The path p^π_x has the edge record $\big(a_{\pi(i)} = x[a_{\pi(i)}], \mathrm{Stat}, x[a_{\pi(i)}]\big)$ at position i; it is the path of x in the dense tree on $(a_{\pi(1)}, \dots, a_{\pi(d)})$, which is defined from the same set $\mathrm{supp}(B_{X_d})$. The layers are tightly dense by Theorem 5.31(1). □

Theorem 6.8 (The orders together compute `CUBE`)**.** *Let $E = E(a_1, \dots, a_d)$, let π be a permutation of $[d]$, and let $E^\pi = E(a_{\pi(1)}, \dots, a_{\pi(d)})$ be the dense tree of the permuted columns.*

(1) *The nodes of depth j of E^π are the pairs (I, u) with $I = \pi([j])$ and $u \in \mathrm{supp}(B_{X_I})$, and the summary at the node $v = (I, u)$ is*

$$\overline{U}^\pi(v) = s(v) = G_{X_I}(u) = \widehat{\alpha}(\sigma_{X_I = u}(B_{\mathcal{R}}))$$

for every commutative monoid and every contribution. It depends on π only through the set $\pi([j])$.
(2) *The nodes of the trees E^π, over all π, are the pairs (I, u) with $I \subseteq [d]$ and $u \in \mathrm{supp}(B_{X_I})$. Together the $d!$ dense trees compute `GROUP BY CUBE`$(a_1, \dots, a_d)$, and each of them computes the `ROLLUP` along one maximal chain*

$$\varnothing = \pi([0]) \subset \pi([1]) \subset \cdots \subset \pi([d]) = [d]$$

of the 2^d grouping sets.
(3) *All orders have the same root, $I = \varnothing$, and the same leaves, $I = [d]$. Two orders π, π' have the same nodes of depth j if and only if $\pi([j]) = \pi'([j])$. For $I \subseteq J$, the summaries of X_I are merges of the summaries of X_J:*

$$G_{X_I}(u) = \bigoplus_{\substack{x \in \mathrm{supp}(B_{X_J}) \\ x <: u}} G_{X_J}(x). \tag{14}$$

Proof. (1) is Theorem 5.31(1) and (2) for the columns $a_{\pi(1)}, \ldots, a_{\pi(d)}$, whose first j form the set $X_{\pi([j])}$; by Corollary 6.7(2) this tree is the level permutation E^π of E, so the statement agrees with Corollary 6.6. (2) Every $I \subseteq [d]$ equals $\pi([j])$ for $j = |I|$ and a suitable π, and the grouping sets of `CUBE` are the sets X_I. (3) The first two claims follow from (1); they are the invariants of Theorem 6.4(4), stated for summaries. Equation (14) is Proposition 4.8(1). □

A node is thus a pair (I, u) and not a path: the order in which the levels in I were applied leaves no trace in it. By Proposition 4.9, when the contribution reads no key column, an intermediate level of one order is not determined by a level of another unless its index set is contained in the other's. But by (14) every level of every order is a merge of the leaves, and the leaves are one `GROUP BY` on all columns. Section 7 rests on this.

The cube of Gray et al. [8] is thus the union of the dense trees over all column orders, glued along the nodes they share (Remark 6.5): all of them share the root and the leaves, and two orders share the nodes of depth j exactly when their first j columns are the same set.

Figure 7 shows the two trees of Figure 5 glued along their shared nodes.

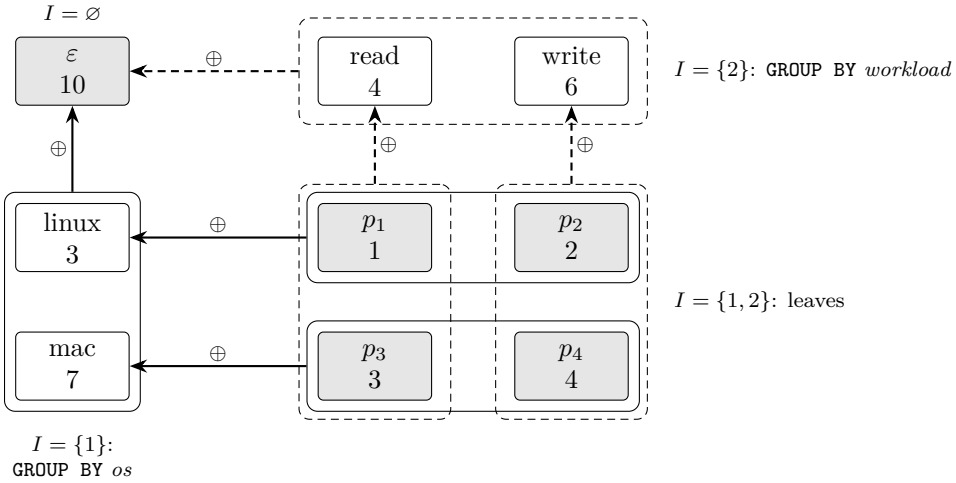


Figure 7: The two trees of Figure 5 glued along the nodes they share. Solid boxes and arrows are the upward merges of E (first the rows, then the row margin), dashed ones those of E^π (first the columns, then the column margin). The shaded nodes, the leaves and the root, belong to both trees and carry the same summaries, $1+2+3+4 = 3+7 = 4+6 = 10$; each margin belongs to one tree only. The four levels are the grouping sets of the cube for $d = 2$: the leaves form a view of rank 2 and each margin a view of rank 1.

Remark 6.9 (Limits of commutativity). Reordering therefore preserves the leaves and the root but, in general, *not* the intermediate levels: for $0 < j < d$ with $\pi([j]) \neq [j]$, the depth-j nodes of E^π aggregate by $X_{\pi([j])}$ instead of $X_{[j]}$. The theorem also relies on the following conditions.

- *Level uniformity.* If siblings split on different attributes, or paths have different lengths, E^π need not be a reduction tree; only level-uniform subtrees can be permuted, each independently. Figure 8 gives an example.
- *Uniform, symmetric reducers.* All nodes use the same symmetric statistics. With edge-specific reducers U_i, the result at v depends on which edge enters v, not only

on (I, u_v); with order-sensitive statistics, equality holds only for a fixed row order.

- *Recomputation from raw values.* The level aggregation λ discards the children's statistics blocks. If statistics were instead aggregated from the children's statistics (e.g. a mean of means), even the root would in general depend on the ordering.

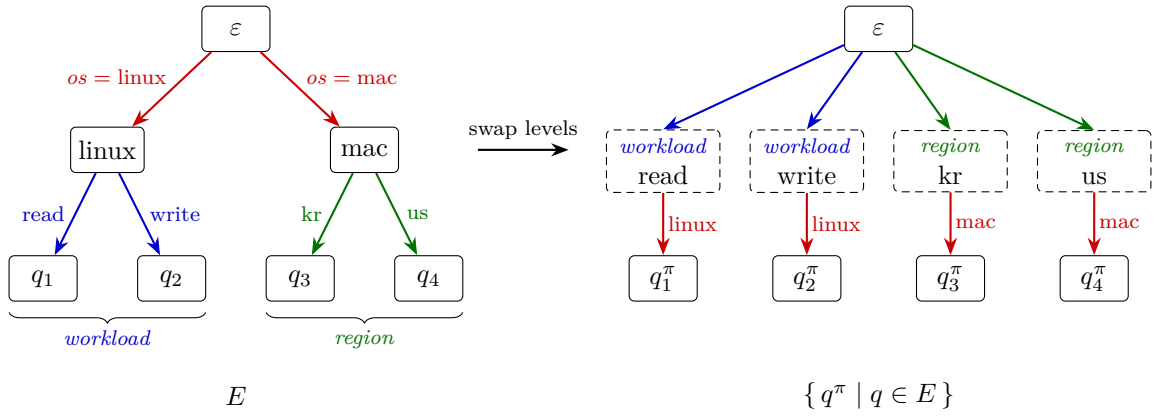


Figure 8: A tree that is not level-uniform (Remark 6.9): at the second level the children of the node linux split on *workload* and those of the node mac on *region*, so there is no single attribute set Y_2 (Definition 6.2). Edges are colored by the attribute of their edge record: red for *os*, as in Figure 5, blue for *workload* and green for *region*. The second level of E thus has two colors, whereas each level of a level-uniform tree has one. Swapping the two levels path by path gives the paths on the right, where the blue and green edges are above the red ones. The children of the root (dashed) now split on two different attributes and overlap: a row with *workload* = read and *region* = kr reaches two of them, and their nodes are not the groups of any key set. Theorem 6.4 does not apply.

7 Choosing the order of levels

By Theorem 6.8, the dense trees on a set of columns, taken in all orders, have the same root and the same leaves, and together they compute the cube. The order of the levels therefore changes nothing at the two ends of a tree and decides everything that is seen in between: it is the order in which the columns are squashed on the way up, and an intermediate level shows the columns that are still kept. This section measures what is seen. It defines the measure (Section 7.1) and then answers the three questions of the introduction: the order (Q1) in Section 7.2, the column (Q2) in Section 7.3, and the views (Q3) in Section 7.4. Section 7.5 combines the results into two algorithms.

Throughout, $\mathcal{R}$ is nonempty, $a_1, \dots, a_d \in \mathcal{K}$ are distinct columns, $X_I := \{a_i \mid i \in I\}$ for $I \subseteq [d]$, and $w \in \mathcal{V}$ is a real-valued value attribute, the *measure*. We use the second-order summary $(\mathbb{N}_0 \times \mathbb{R}^2, +)$ with contribution $\alpha(r) = (1, r[w], r[w]^2)$. This is the counting completion $(\mathbb{R}^2, +)^{\#}$ of Definition 4.16, with the first component as count, and α is counting (Definition 4.14). For $I \subseteq [d]$ and a key $u \in \operatorname{supp}(B_{X_I})$, the summary of the cube node (I, u) is

$$(n_u, t_u, q_u) := \widehat{\alpha}(\sigma_{X_I = u}(B_{\mathcal{R}})), \qquad m_u := t_u / n_u,$$

the number of rows of the group, the sum and the sum of squares of the measure, and its mean. This is the entry $G_{X_I}(u)$ of the squash of $\mathcal{R}$ onto X_I (Definition 4.4), which keeps the columns of I and squashes the measure together with all other columns into the summary space $\mathbb{N}_0 \times \mathbb{R}^2$, and $n_u = B_{X_I}(u) \geq 1$ by Proposition 4.15(1). For the root we

write (n, t, q) and m. By (14) these summaries are additive along the subfilter order: for $J \supseteq I$,

$$(n_u, t_u, q_u) = \sum_{\substack{x \in \mathrm{supp}(B_{X_J}) \\ x <: u}} (n_x, t_x, q_x). \tag{15}$$

7.1 Explained variance on the level lattice

Definition 7.1 (Explained variance)**.** Let $P, I \subseteq [d]$ be disjoint and $p \in \mathrm{supp}(B_{X_P})$. The *variance explained by I within p* is the weighted variance of the group means one level set below p,

$$\nu(I \mid p) := \sum_{\substack{u \in \mathrm{supp}(B_{X_{P \cup I}}) \\ u <: p}} \frac{n_u}{n_p} (m_u - m_p)^2.$$

For $P = \varnothing$ we write $\nu(I)$; it is the variance of the means of the groups of `GROUP BY` X_I, weighted by their sizes. The *total variance* of the measure is $\nu_{\mathrm{tot}} := q/n - m^2$.

For a single column, $\nu(\{i\} \mid p)$ is the variance of the means in the level table of the node with key p in a dense tree whose next column is a_i (Theorem 5.31(2)), that is, of the layer that is tightly dense on a_i at this node (Definition 5.28). This is the quantity meant by the variance of the aggregated values.

Theorem 7.2 (Variance on the level lattice)**.** *Let $P, I \subseteq [d]$ be disjoint and $p \in \mathrm{supp}(B_{X_P})$.*

(1) From summaries. $\nu(I \mid p) = \dfrac{1}{n_p} \displaystyle\sum_{u <: p} \frac{t_u^2}{n_u} - m_p^2$, *the sum being over $u \in \mathrm{supp}(B_{X_{P \cup I}})$.*

(2) Total variance. *The variance of the measure within p splits into an explained and a residual part:*

$$\frac{q_p}{n_p} - m_p^2 = \nu(I \mid p) + \frac{1}{n_p} \sum_{u <: p} \Big(q_u - \frac{t_u^2}{n_u} \Big),$$

where the last term is the mean squared deviation of the rows from the mean of their group. In particular $\nu_{\mathrm{tot}} = \nu(I) + \frac{1}{n} \sum_u (q_u - t_u^2/n_u)$.

(3) Chain rule. $\nu(P \cup I) = \nu(P) + \displaystyle\sum_{p \in \mathrm{supp}(B_{X_P})} \frac{n_p}{n} \nu(I \mid p)$.

(4) Monotone and bounded. *If $I \subseteq J \subseteq [d]$, that is, $X_J <: X_I$, then*

$$0 = \nu(\varnothing) \le \nu(I) \le \nu(J) \le \nu([d]) \le \nu_{\mathrm{tot}}.$$

Here $\nu(I) = \nu(J)$ if and only if $m_x = m_{x[X_I]}$ for every $x \in \mathrm{supp}(B_{X_J})$, and $\nu([d]) = \nu_{\mathrm{tot}}$ if and only if the measure is constant on the rows of every leaf.

Proof. (1) By (15), $\sum_{u <: p} n_u = n_p$ and $\sum_{u <: p} t_u = t_p$. Hence

$$\sum_{u <: p} n_u (m_u - m_p)^2 = \sum_{u <: p} \frac{t_u^2}{n_u} - 2 m_p t_p + n_p m_p^2 = \sum_{u <: p} \frac{t_u^2}{n_u} - n_p m_p^2.$$

(2) Add the two terms and use (1) and $\sum_{u <: p} q_u = q_p$. The term $q_u - t_u^2/n_u$ is the sum of $(r[w] - m_u)^2$ over the row occurrences of the group u.

(3) Every $u \in \mathrm{supp}(B_{X_{P\cup I}})$ lies below exactly one $p \in \mathrm{supp}(B_{X_P})$, namely $u[X_P]$. By (1),

$$\sum_p \frac{n_p}{n}\,\nu(I \mid p) = \frac{1}{n}\sum_u \frac{t_u^2}{n_u} - \frac{1}{n}\sum_p \frac{t_p^2}{n_p} = \big(\nu(P\cup I) + m^2\big) - \big(\nu(P) + m^2\big).$$

(4) Apply (3) with $P := I$ and the level set $J \setminus I$: the difference $\nu(J) - \nu(I)$ is a sum of nonnegative terms, which vanish exactly when all means below each p equal m_p. The bound $\nu(J) \le \nu_{\mathrm{tot}}$ and the last claim follow from (2), whose residual term is nonnegative and vanishes exactly when every group is constant. □

Part (1) shows that ν needs only the counts and sums of the cube nodes, which are merges of leaf summaries. Part (2) says what a squash hides. The squash onto $X_{P\cup I}$ replaces the rows of each group by one element of the summary space, a count, a sum, and a sum of squares, and a display of the group means shows the variance between the groups, $\nu(I \mid p)$. The residual term is the variance within the groups, which the means no longer show. It is still recorded in the sums of squares, so the split is computed from the summaries, without the rows. Part (4) is the reason why the question of this section is about orders and not about sets: more columns always explain more, so the complete set $[d]$ is trivially the best, and what can be chosen is how early the variance appears.

7.2 Orders

Definition 7.3 (Profile of an order)**.** Let π be a permutation of $[d]$; it stands for the order $a_{\pi(1)}, \ldots, a_{\pi(d)}$ of the levels. The *profile* of π is

$$\nu_j(\pi) := \nu(\pi([j])) \quad (0 \le j \le d), \qquad \delta_j(\pi) := \nu_j(\pi) - \nu_{j-1}(\pi) \quad (1 \le j \le d),$$

the variance explained by the first j levels and the variance *revealed* by level j.

Proposition 7.4 (An order splits a fixed total)**.** *For every permutation π of $[d]$:*

(1) *$\delta_j(\pi) \ge 0$ and $\delta_1(\pi) + \cdots + \delta_d(\pi) = \nu([d])$, which does not depend on π.*
(2) *$\delta_j(\pi)$ is the average over the nodes v of depth $j-1$ of the dense tree $E(a_{\pi(1)}, \ldots, a_{\pi(d)})$, weighted by n_{u_v}/n, of the variance $\nu(\{\pi(j)\} \mid u_v)$ of the means in the level table of v.*
(3) *$\nu_j(\pi)$ depends on π only through the set $\pi([j])$.*

Proof. (1) is Theorem 7.2(4) and a telescoping sum; $\nu([d])$ is a function of the leaves, which all orders share (Theorem 6.8(3)). (2) is Theorem 7.2(3) with $P = \pi([j-1])$ and $I = \{\pi(j)\}$, together with Theorem 6.8(1), which identifies the nodes of depth $j-1$ with $\mathrm{supp}(B_{X_P})$. (3) holds by definition. □

Since both ends of the profile are fixed, two orders can only differ in how fast the profile rises. We compare them by the area under it.

Definition 7.5 (Most revealing order, greedy order)**.** The *area* of an order is

$$A(\pi) := \sum_{j=1}^{d-1} \nu_j(\pi) = \sum_{j=1}^{d-1} (d-j)\,\delta_j(\pi).$$

An order is *most revealing* if it maximizes A. An order is *greedy* if, for every j, the column $\pi(j)$ maximizes $\nu(\pi([j-1]) \cup \{i\})$ over $i \notin \pi([j-1])$.

The second expression for A follows from $\nu_j = \delta_1 + \cdots + \delta_j$: variance revealed at level j is visible on the $d - j$ intermediate levels from j on, and counts that many times.

Theorem 7.6 (Finding the order)**.** *Let ν and A be as in Definitions 7.1 and 7.5.*

(1) Exact. *Define $F(\varnothing) := 0$ and, for nonempty $I \subseteq [d]$,*

$$F(I) := \nu(I) + \max_{i \in I} F(I \setminus \{i\}).$$

Then $\max_\pi A(\pi) = F([d]) - \nu([d])$, and a most revealing order is read off backwards from the maximizing indices. The recursion uses ν on the 2^d grouping sets of the cube and $d\,2^{d-1}$ comparisons. It is valid for any real function on the subsets of $[d]$ in place of ν.

(2) Greedy. *A greedy order uses ν on at most $d(d+1)/2$ sets. If the maximizer is unique at every step, its profile $(\nu_1, \ldots, \nu_d)$ is the lexicographically largest among all orders.*

(3) Additive case. *Suppose that $\nu(I) = \sum_{i \in I} \nu(\{i\})$ for all $I \subseteq [d]$. Then the order by decreasing $\nu(\{i\})$ satisfies $\nu_j(\pi) \geq \nu_j(\pi')$ for all orders π' and all j; it is greedy and most revealing. The hypothesis holds if the columns are independent in $\mathcal{R}$, i.e. $B_{X_{[d]}}(x)/n = \prod_i B_{\{a_i\}}(x[a_i])/n$, and the leaf means are additive, $m_x = \sum_i f_i(x[a_i])$ for some functions f_i.*

(4) In general *a greedy order need not be most revealing* (Example 7.7).

Proof. (1) By induction on $|I|$, $F(I)$ is the maximum of $\nu(I_1) + \cdots + \nu(I_k)$ over the chains $\varnothing \subset I_1 \subset \cdots \subset I_k = I$ with $|I_j| = j$: such a chain consists of a chain for $I \setminus \{i\}$, for the element i added last, followed by I. The chains for $I = [d]$ are the sets $\pi([j])$ of the orders π, and $\nu_d(\pi) = \nu([d])$ for all of them. Monotonicity of ν is not used.

(2) Step j evaluates $d - j + 1$ sets. The greedy order maximizes ν_1; among the orders with the same first column it maximizes ν_2; and so on.

(3) Under the hypothesis, $\nu_j(\pi)$ is the sum of j of the numbers $\nu(\{i\})$, which is largest for the j largest of them. For the sufficient condition, put $p_i(c) := B_{\{a_i\}}(c)/n$ and subtract from each f_i its mean $\sum_c p_i(c) f_i(c)$, collecting the means in a constant f_0. By independence, the leaves $x <: u$ below a key $u \in \operatorname{supp}(B_{X_I})$ have the weights $\prod_{i \notin I} p_i(x[a_i])$ within u, so $m_u = f_0 + \sum_{i \in I} f_i(u[a_i])$ and $m = f_0$; and $n_u/n = \prod_{i \in I} p_i(u[a_i])$. Hence $\nu(I) = \sum_u \prod_{i \in I} p_i(u[a_i]) \big(\sum_{i \in I} f_i(u[a_i])\big)^2 = \sum_{i \in I} \sum_c p_i(c) f_i(c)^2$, because the mixed terms have mean zero. □

Example 7.7 (An interaction that the greedy order finds last)**.** Let a_1, a_2, a_3 take the values $0, 1$, let $\mathcal{R}$ have one row for each of the eight combinations, and let $w = [a_1 \neq a_2] + \varepsilon\, a_3$ with $\varepsilon = \frac{1}{10}$. Then

$$\nu(\{1\}) = \nu(\{2\}) = 0, \qquad \nu(\{3\}) = \nu(\{1,3\}) = \nu(\{2,3\}) = \tfrac{1}{400},$$
$$\nu(\{1,2\}) = \tfrac{1}{4}, \qquad \nu([3]) = \nu_{\text{tot}} = \tfrac{101}{400}.$$

A greedy order starts with a_3 and has the profile $(\frac{1}{400}, \frac{1}{400}, \frac{101}{400})$ and the area $\frac{1}{200}$. The order a_1, a_2, a_3 has the profile $(0, \frac{1}{4}, \frac{101}{400})$ and the area $\frac{1}{4}$, which is the maximum. Neither a_1 nor a_2 explains anything alone; together they explain almost everything. Figure 9 shows the two profiles.

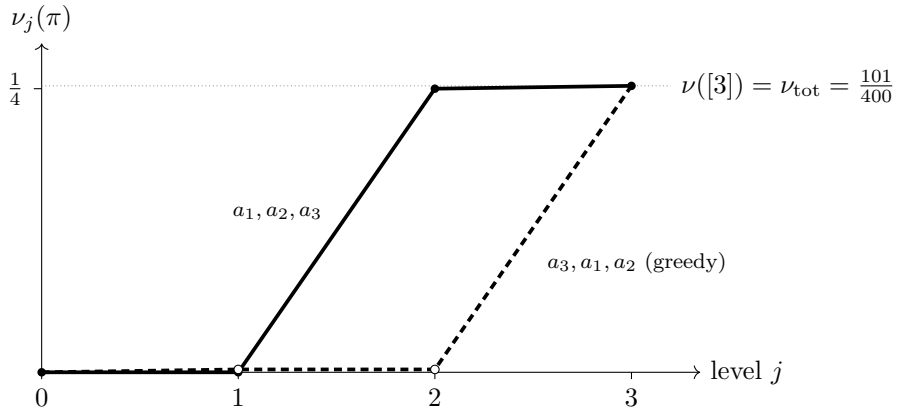


Figure 9: The profiles of two orders in Example 7.7. Both start at 0 and end at $\nu([3])$, as every profile does (Proposition 7.4). The greedy order (dashed) reveals $\frac{1}{400}$ at the first level and nothing at the second; the order a_1, a_2, a_3 (solid) reveals nothing at the first level and $\frac{1}{4}$ at the second. The area $A(\pi)$ adds the heights at the intermediate levels 1 and 2: $\frac{1}{200}$ against $\frac{1}{4}$.

7.3 The most affecting column

Question (Q2) asks for one column. The variance that a column reveals depends on the columns that precede it, so the question has one answer for each position.

Definition 7.8 (Effects of a column)**.** For $i \in [d]$ and $P \subseteq [d] \setminus \{i\}$, the *increment* of i after P is $\delta(i \mid P) := \nu(P \cup \{i\}) - \nu(P)$. The *main effect*, the *total effect*, and the *Shapley effect* of i are

$$\mu_i := \delta(i \mid \varnothing) = \nu(\{i\}), \qquad \tau_i := \delta(i \mid [d] \setminus \{i\}), \qquad \varphi_i := \frac{1}{d!} \sum_{\pi} \delta(i \mid \pi([\pi^{-1}(i) - 1])),$$

where the sum is over all orders and $\pi([\pi^{-1}(i) - 1])$ is the set of columns that precede i in π.

Proposition 7.9 (Effects)**.** *Let $i \in [d]$.*

(1) *In every order π, the variance revealed by the level of column i is $\delta(i \mid P)$ with P the set of columns that precede it. It equals μ_i if i is first and τ_i if i is last, and μ_i/ν_{tot} is the fraction of the total variance that a display of column i alone shows.*
(2) *$\varphi_i \geq 0$ and $\varphi_1 + \cdots + \varphi_d = \nu([d])$.*
(3) *In the additive case of Theorem 7.6(3), $\mu_i = \tau_i = \varphi_i$ for every i.*
(4) *In general the three rankings differ. In Example 7.7, $(\mu_1, \mu_2, \mu_3) = (0, 0, \frac{1}{400})$, while $(\tau_1, \tau_2, \tau_3) = (\frac{1}{4}, \frac{1}{4}, \frac{1}{400})$ and $(\varphi_1, \varphi_2, \varphi_3) = (\frac{1}{8}, \frac{1}{8}, \frac{1}{400})$.*

Proof. (1) is Definition 7.3. (2) Each increment is nonnegative by Theorem 7.2(4), and for each order the increments of the d columns sum to $\nu([d])$ by Proposition 7.4(1). (3) Every increment of i equals $\nu(\{i\})$. (4) follows from the values in Example 7.7; for instance, column 1 is preceded by $\varnothing$, $\{2\}$, $\{3\}$, $\{2, 3\}$ in $2, 1, 1, 2$ of the six orders, with increments $0, \frac{1}{4}, 0, \frac{1}{4}$. □

The three effects answer three questions. The main effect is the right measure for the single axis to display first. The total effect measures what is lost when the column is squashed

and all others are kept. The Shapley effect is the share of a column that does not depend on an order; it is the Shapley value of the game $I \mapsto \nu(I)$, used as an importance measure for variance explained by Owen [12] and, for dependent inputs as they occur here, by Song, Nelson, and Staum [15]. A column reveals nothing in every order exactly when $\varphi_i = 0$. A vanishing main effect does not show this (Example 7.7), and neither does a vanishing total effect, which also occurs for a column that is a function of the others.

Remark 7.10 (Columns with many values). By Theorem 7.2(4), $\nu(I) = \nu_{\mathrm{tot}}$ whenever every group of X_I is constant, in particular when a column has a different value in every row. Raw explained variance therefore favours columns with many values, whether or not they affect the measure. With $k_I := |\operatorname{supp}(B_{X_I})|$ groups, the correction for the number of groups is the adjusted explained variance

$$\nu_{\mathrm{adj}}(I) := \nu_{\mathrm{tot}} - \frac{n-1}{n-k_I}\big(\nu_{\mathrm{tot}} - \nu(I)\big) \quad (k_I < n), \qquad \nu_{\mathrm{adj}}(I) := 0 \quad (k_I = n),$$

which rescales the residual variance by its degrees of freedom, as in the adjusted coefficient of determination. It is computed from the same summaries, with $\nu_{\mathrm{adj}}(\varnothing) = 0$ and $\nu_{\mathrm{adj}}(I) \le \nu(I)$. It is no longer monotone: adding a level can lower it, so the increments of an order may be negative, although their sum $\nu_{\mathrm{adj}}([d])$ is still the same for every order. The recursion of Theorem 7.6(1) and the three effects apply to ν_{adj} without change.

7.4 Views

Question (Q3) concerns displays. A display shows one or two axes, possibly after fixing the values of some other columns.

Definition 7.11 (View)**.** Let $P, I \subseteq [d]$ be disjoint, I nonempty, and $p \in \operatorname{supp}(B_{X_P})$. The *view* with pinned key p and axes I is the table

$$u \longmapsto \gamma(s(u)), \qquad u \in \operatorname{supp}(B_{X_{P\cup I}}),\ u <: p,$$

of the reported statistic on the groups one level set below p. Its *rank* is $|I|$: a view of rank 1 is a bar chart and a view of rank 2 is a heat map. The columns outside $P \cup I$ are squashed.

Proposition 7.12 (Views are levels of the cube)**.** *Let $P, I \subseteq [d]$ be disjoint, I nonempty, and $p \in \operatorname{supp}(B_{X_P})$.*

(1) Any axes. *The cells of the view $(p; I)$ are the cube nodes $(P \cup I, u)$ with $u <: p$. For every summary monoid, the summary of a cell is the direct summary of its group and the merge of the leaf summaries below it,*

$$s(u) = \widehat{\alpha}(\sigma_{X_{P\cup I}=u}(B_{\mathcal{R}})) = \bigoplus_{\substack{x \in \operatorname{supp}(B_{X_{[d]}}) \\ x <: u}} s(x).$$

Hence every view, for every choice of pinned columns and axes, is computed from the leaf summaries of one dense tree, without reading $\mathcal{R}$ again: the leaf summaries are the squash $G_{X_{[d]}}$, and a view keeps a part of its columns (Proposition 4.8(1), Corollary 4.10).

(2) Variance shown. *With the mean as the reported statistic, the weighted variance of the cells of the view $(p; I)$ is $\nu(I \mid p)$, and*

$$\nu(P \cup I) - \nu(P) = \sum_{p \in \operatorname{supp}(B_{X_P})} \frac{n_p}{n}\, \nu(I \mid p).$$

The variance revealed by the axes I after the columns P is the average of the variances shown by the views $(p; I)$.

(3) Flat views. *$\nu(I \mid p) = 0$ if and only if all cells of the view $(p; I)$ have the mean m_p; the axes I can then be squashed below p without changing any mean.*

Proof. (1) The view lists the keys of $\mathrm{supp}(B_{X_{P\cup I}})$ below p, which are nodes of the cube by Theorem 6.8(2); the summary at a node is the same in every dense tree that contains it and equals the direct summary by Theorem 6.8(1) and (10). The second equality is (14) for the index sets $P \cup I \subseteq [d]$. (2) is Definition 7.1 and Theorem 7.2(3). (3) A weighted sum of squares with positive weights vanishes exactly when every term does. □

Views of a fixed rank r are ranked by $\nu(I \mid p)$, or by its adjusted form: the largest values point to the axes along which the aggregated values differ most, and by part (2) a large increment $\nu(P\cup I)-\nu(P)$ guarantees that some pinned key p has a view with at least that variance. Descending into the key with the largest $\frac{n_p}{n}\nu(I \mid p)$ and repeating the search there is the drill-down of root-cause analysis.

Remark 7.13 (Orders and views). The first r levels of an order form a view of rank r, so an order is a nested family of views, one of each rank. The best views of different ranks need not be nested. In Example 7.7 the best view of rank 1 has the axis a_3 and the best view of rank 2 has the axes a_1, a_2; no order contains both, and the most revealing order gives up the first for the second. For a heat map on two columns the difference

$$\nu(\{i, j\}) - \nu(\{i\}) - \nu(\{j\})$$

measures what the heat map shows beyond its two bar charts. It is zero in the additive case, positive when the columns interact ($\frac{1}{4}$ for a_1, a_2 in Example 7.7), and negative when they are redundant: if a_j is a function of a_i, then $\nu(\{i, j\}) = \nu(\{i\})$ and the difference is $-\nu(\{j\})$.

Remark 7.14 (Other statistics). Theorem 7.2 is a statement about means. It extends to every quasi-arithmetic mean by transforming the measure first: for the geometric mean, replace w by $\log w$, and all results hold for the variance of the logarithms. For another reported statistic, such as a median or a quantile, Proposition 7.12(1) still holds whenever the statistic is mergeable (Theorem 5.21), with the bag of values as the summary when no smaller one exists, so every view is still available and exact. The weighted variance of the reported values can still be computed for each level set and used in the recursion of Theorem 7.6(1), but it need not be monotone, and no fixed total is split among the levels.

7.5 The algorithm

The results combine into Algorithm 1, which reads $\mathcal{R}$ once. In the listings, G_I stands for the squash G_{X_I} of $\mathcal{R}$ onto the columns X_I, stored as a table from the key values $u \in \mathrm{supp}(B_{X_I})$ to their summaries (n_u, t_u, q_u), and $u[X_I]$ is the restriction of a key value to X_I.

Correctness. Line 8 computes the leaves, which are the same for every order (Theorem 6.8(3)). By Proposition 4.8(1), line 11 yields the squash onto X_I whichever column $i \notin I$ is chosen, so the tables G_I are the 2^d grouping sets of the cube. Lines 15 to 22 are the recursion of Theorem 7.6(1) and its backtracking: $\ell(I)$ is the column that a best chain ending in I adds last, and $F([d]) - \nu([d])$ is the largest area. By Theorem 6.8(1), the level

Algorithm 1 Order, effects, and views from one `GROUP BY` (exact)

Input: relation $\mathcal{R}$; distinct columns $a_1, \dots, a_d \in \mathcal{K}$; measure $w \in \mathcal{V}$

Output: a most revealing order π, the levels $T_0, \dots, T_d$ of its dense tree and its profile; the effects μ_i, τ_i, φ_i of every column; the views of rank 1 and 2, ranked

1: **function** SQUASH(G, I) ▷ Proposition 4.8(1)
2: $H \leftarrow$ empty table
3: **for** each entry $u \mapsto s$ of G **do**
4: $H[u[X_I]] \leftarrow H[u[X_I]] + s$ ▷ a missing entry is $(0, 0, 0)$
5: **return** H
6: **function** VAR(G, n, m) ▷ Theorem 7.2(1)
7: **return** $\frac{1}{n} \sum_{(n_u, t_u, q_u) \in G} t_u^2 / n_u - m^2$

Leaves and cube

8: $G_{[d]} \leftarrow$ `GROUP BY` $a_1, \dots, a_d$ on $\mathcal{R}$ with $(\texttt{COUNT}, \texttt{SUM}(w), \texttt{SUM}(w^2))$ ▷ the only read of $\mathcal{R}$
9: **for** $k = d - 1$ **down to** 0 **do**
10: **for** each $I \subseteq [d]$ with $|I| = k$ **do**
11: $G_I \leftarrow$ SQUASH($G_{I \cup \{i\}}$, I) for some $i \notin I$ ▷ squashes the column a_i
12: $(n, t, q) \leftarrow$ the single entry of $G_\varnothing$; $m \leftarrow t/n$; $\nu_{\text{tot}} \leftarrow q/n - m^2$
13: **for** each $I \subseteq [d]$ **do**
14: $\nu(I) \leftarrow$ VAR(G_I, n, m)

(Q1) *Most revealing order*

15: $F(\varnothing) \leftarrow 0$
16: **for** $k = 1$ **to** d **do**
17: **for** each $I \subseteq [d]$ with $|I| = k$ **do**
18: $\ell(I) \leftarrow \arg\max_{i \in I} F(I \setminus \{i\})$ ▷ the column that is split last within I
19: $F(I) \leftarrow \nu(I) + F(I \setminus \{\ell(I)\})$
20: $I \leftarrow [d]$
21: **for** $j = d$ **down to** 1 **do**
22: $\pi(j) \leftarrow \ell(I)$; $I \leftarrow I \setminus \{\pi(j)\}$
23: **for** $j = 0$ **to** d **do**
24: $T_j \leftarrow G_{\pi([j])}$; $\nu_j \leftarrow \nu(\pi([j]))$ ▷ level j of E^π, Theorem 6.8(1)

(Q2) *Effects of the columns*

25: **for** $i = 1$ **to** d **do**
26: $\mu_i \leftarrow \nu(\{i\})$; $\tau_i \leftarrow \nu([d]) - \nu([d] \setminus \{i\})$
27: $\varphi_i \leftarrow \displaystyle\sum_{P \subseteq [d] \setminus \{i\}} \frac{|P|!\,(d - 1 - |P|)!}{d!} \left(\nu(P \cup \{i\}) - \nu(P)\right)$

(Q3) *Views*

28: **for** $r = 1, 2$ **do**
29: $V_r \leftarrow$ the sets $I \subseteq [d]$ with $|I| = r$, sorted by decreasing $\nu(I)$
30: **return** π, $(T_j)_{0 \le j \le d}$, $(\nu_j)_{0 \le j \le d}$, $(\mu_i, \tau_i, \varphi_i)_{1 \le i \le d}$, V_1, V_2

of depth j of the dense tree E^π is the squash onto the first j columns of π, so line 24 returns the tree without another pass. Line 27 evaluates the Shapley effect of Definition 7.8 by collecting the $d!$ orders according to the set P of the columns that precede i: exactly $|P|!\,(d-1-|P|)!$ orders have this set. A view with a pinned key $p \in \text{supp}(B_{X_P})$ needs no further table: by Proposition 7.12 its cells are the entries of $G_{P \cup I}$ below p, and $\nu(I \mid p)$ is VAR applied to these entries with the count and the mean of the entry p of G_P. The

Algorithm 2 Greedy order

Input: as in Algorithm 1

Output: a greedy order π, the levels $T_0, \dots, T_d$ of its dense tree and its profile

1: $L \leftarrow$ `GROUP BY` $a_1, \dots, a_d$ on $\mathcal{R}$ with $(\texttt{COUNT},\ \texttt{SUM}(w),\ \texttt{SUM}(w^2))$ ▷ the only read of $\mathcal{R}$
2: $T_0 \leftarrow \textsc{Squash}(L, \varnothing)$; $(n, t, q) \leftarrow$ the single entry of T_0; $m \leftarrow t/n$
3: $P \leftarrow \varnothing$; $\nu_0 \leftarrow 0$
4: **for** $j = 1$ **to** d **do**
5: **for** each $i \in [d] \setminus P$ **do**
6: $H_i \leftarrow \textsc{Squash}(L, P \cup \{i\})$; $e_i \leftarrow \textsc{Var}(H_i, n, m)$ ▷ $e_i = \nu(P \cup \{i\})$
7: $\pi(j) \leftarrow \arg\max_{i \in [d] \setminus P} e_i$
8: $T_j \leftarrow H_{\pi(j)}$; $\nu_j \leftarrow e_{\pi(j)}$; $P \leftarrow P \cup \{\pi(j)\}$
9: **return** π, $(T_j)_{0 \le j \le d}$, $(\nu_j)_{0 \le j \le d}$

adjusted variance of Remark 7.10 replaces line 14 by $\nu_{\mathrm{adj}}(I)$ with k_I the number of entries of G_I; the other lines do not change.

Algorithm 2 is the greedy order of Definition 7.5. Each candidate is squashed from the leaves: $G_{P \cup \{i\}}$ keeps a column that G_P has squashed, so it is not determined by G_P (Proposition 4.9). It finds a most revealing order in the additive case of Theorem 7.6(3) and can fail otherwise (Example 7.7).

Cost. Let $N := |\operatorname{supp}(B_{X_{[d]}})| \le |B_{\mathcal{R}}|$ be the number of leaves. Both algorithms read $\mathcal{R}$ once and afterwards touch only summaries. Every table G_I has at most N entries, so Algorithm 1 performs at most $(2^d - 1)\,N$ additions of summaries in lines 9 to 11, followed by $d\,2^{d-1}$ comparisons for the order and $d\,2^{d-1}$ increments for the Shapley effects. Algorithm 2 performs $d(d+1)/2$ squashes of the leaves, at most $\frac{1}{2}d(d+1)\,N$ additions. If only the order and the effects are wanted, G_I can be dropped once $\nu(I)$ is known and the tables of size $|I| - 1$ are built, and the $d - 1$ intermediate levels of the chosen order are squashed from the leaves at the end.

8 Conclusion

The order of the levels of a reduction tree changes nothing at the root and at the leaves and decides what is seen in between. Explained variance makes this choice a search on the lattice of grouping sets: every order splits the same total, the most revealing order is found exactly from 2^d merged summaries, and the main, total, and Shapley effects say how much a single column contributes at the first position, at the last, and on average. The cost is one `GROUP BY` on all columns, followed by merges.

The algebra that makes this exact is more general than the use we make of it. A squash keeps a set of attributes and reduces all others to a summary space, it can be continued and not undone, and every level of every tree is such a squash. The exchange theorem holds for arbitrary sibling predicates and every commutative monoid, and it says for each kind of summary what overlapping or incomplete children cost: nothing for idempotent summaries that are covered, an explicit discrepancy for additive ones, and a difference of counts that certifies the merge.

Two limits remain. The exact recursion and the Shapley effects need all 2^d grouping sets, which is practical for the handful of columns of a drill-down and not beyond; the greedy

order needs $d(d+1)/2$ sets but can miss interactions. And the fixed total that an order splits is a property of means: for other reported statistics the views are still exact, but their variance need not be monotone in the set of columns.

References

[1] Yael Amsterdamer, Daniel Deutch, and Val Tannen. "Provenance for Aggregate Queries". In: *Proceedings of the 30th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS '11)*. ACM, 2011, pp. 153–164.

[2] Richard S. Bird. "An Introduction to the Theory of Lists". In: *Logic of Programming and Calculi of Discrete Design*. Ed. by Manfred Broy. Berlin: Springer-Verlag, 1987, pp. 3–42.

[3] Jeffrey Dean and Sanjay Ghemawat. "MapReduce: Simplified Data Processing on Large Clusters". In: *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04)*. San Francisco, CA, USA: USENIX Association, 2004, pp. 137–150.

[4] DuckDB Developers. *WITH Clause: CTE Materialization — DuckDB Documentation*. `https://duckdb.org/docs/current/sql/query_syntax/with`. Accessed: 2026-11-11.

[5] Kento Emoto, Sebastian Fischer, and Zhenjiang Hu. "Generate, Test, and Aggregate: A Calculation-based Framework for Systematic Parallel Programming with MapReduce". In: *Programming Languages and Systems (ESOP 2012)*. Vol. 7211. Lecture Notes in Computer Science. Springer, 2012, pp. 254–273. DOI: 10.1007/978-3-642-28869-2_13.

[6] Philip J. Fleming and John J. Wallace. "How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results". In: *Communications of the ACM* 29.3 (1986), pp. 218–221. DOI: 10.1145/5666.5673. URL: https://doi.org/10.1145/5666.5673.

[7] Jeremy Gibbons. "The Third Homomorphism Theorem". In: *Journal of Functional Programming* 6.4 (1996), pp. 657–665. DOI: 10.1017/S0956796800001908.

[8] Jim Gray et al. "Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals". In: *Data Mining and Knowledge Discovery* 1.1 (1997), pp. 29–53. DOI: 10.1023/A:1009726021843. URL: https://doi.org/10.1023/A:1009726021843.

[9] Ralf Lämmel. "Google's MapReduce Programming Model – Revisited". In: *Science of Computer Programming* 70.1 (2008), pp. 1–30. DOI: 10.1016/j.scico.2007.07.001.

[10] Jimmy Lin. *Monoidify! Monoids as a Design Principle for Efficient MapReduce Algorithms*. 2013. arXiv: 1304.7544 [cs.DC].

[11] Erik Meijer, Maarten Fokkinga, and Ross Paterson. "Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire". In: *Functional Programming Languages and Computer Architecture (FPCA 1991)*. Vol. 523. Lecture Notes in Computer Science. Springer, 1991, pp. 124–144. DOI: 10.1007/3540543961_7. URL: https://doi.org/10.1007/3540543961_7.

[12] Art B. Owen. "Sobol' Indices and Shapley Value". In: *SIAM/ASA Journal on Uncertainty Quantification* 2.1 (2014), pp. 245–251. DOI: 10.1137/130936233.

[13] Rob Pike et al. "Interpreting the Data: Parallel Analysis with Sawzall". In: *Scientific Programming* 13.4 (2005), pp. 277–298.

[14] Polars Developers. *polars.collect_all — Polars Documentation*. https://docs.pola.rs/api/python/dev/reference/api/polars.collect_all.html. Accessed: 2026-11-11.

[15] Eunhye Song, Barry L. Nelson, and Jeremy Staum. "Shapley Effects for Global Sensitivity Analysis: Theory and Computation". In: *SIAM/ASA Journal on Uncertainty Quantification* 4.1 (2016), pp. 1060–1083. DOI: 10.1137/15M1048070.

[16] Yuan Yu, Pradeep Kumar Gunda, and Michael Isard. "Distributed Aggregation for Data-Parallel Computing: Interfaces and Implementations". In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP '09)*. ACM, 2009, pp. 247–260. DOI: 10.1145/1629575.1629600.